

Build with AI

개발자, 제품 관리자, 비즈니스 리더를 위한 실용 가이드

12 Parts · Complete Series · 2025

목차

Part 0 AI로 만들기: 비개발자를 위한 실전 가이드

Part 1 AI가 답답하게 느껴지는 이유 — 그리고 모든 것을 바꾸는 사고 전환

Part 2 AI가 실제로 하는 일 (그리고 하지 못하는 일)

Part 3 AI 레고 스택

Part 4 진짜 병목은 당신의 데이터

Part 5 진짜 기술은 코딩이 아니다

Part 6 프롬프팅은 프로그래밍이다

Part 7 AI 에이전트

Part 8 바이브 코딩

Part 9 AI 코드 어시스턴트

Part 10 데모에서 프로덕션으로

Part 11 위험, 할루시네이션과 책임 있는 AI

Part 12 다음 물결: AI 에이전트, 물리 AI, 그리고 그 이후

AI로 만들기: 비개발자를 위한 실전 가이드

7 min read · 소개

"AI로 만들기" 시리즈 소개

> **핵심 요약:** *AI로 만들기*는 AI를 단순히 대화 상대로 쓰는 것을 넘어 실제로 작동하는 무언가를 만들고 싶은 비개발자 — 창업자, 운영 관리자, 컨설턴트, 도메인 전문가 — 를 위한 무료 12부작 시리즈다. 코딩 경험은 필요 없다. 유일한 전제 조건은 당신의 문제, 데이터, 사용자에 대해 신중하게 생각하려는 의지다.

무언가가 바뀌었다.

기술 자체가 아니라, AI는 수년째 개선되어 왔다. 바뀐 건 누구를 위한 것인가다.

최근 역사의 대부분 동안, AI로 의미 있게 작업하는 것은 기술적 능통함을 요구했다. API, 훈련 파이프라인, 모델 아키텍처를 이해해야 했다. 진짜 가치를 얻는 사람들은 엔지니어, 연구자, 데이터 사이언티스트였다. 다른 모든 사람은 채팅 인터페이스를 받고 "탐색해보세요"라는 말을 들었다.

그 시대는 끝났다.

2026년, 도구들은 컨설턴트, 창업자, 교사, 소규모 사업주 — 명확한 문제가 있고 그것에 대해 신중하게 생각할 의향이 있는 누구든 — 이 2년 전에는 엔지니어링 팀이 필요했을 소프트웨어를 만들고, 워크플로우를 자동화하고, AI 기반 솔루션을 배포할 수 있을 만큼 성숙했다.

간극은 더 이상 기술 능력이 아니다. 이해다. 대부분의 사람들이 AI에 대해 어떻게 생각해야 할지 모른다 — AI가 실제로 무엇을 하는지, 어디서 진정으로 도움이 되는지, 어디서 안정적으로 실패하는지, 어떻게 일관된 결과를 얻는지. 데모를 봤다. 도구를 시도해봤다. 인상적임과 답답함 사이 어딘가에 갇혀 있다.

이 시리즈가 그 다리다.

이 시리즈가 무엇인가

"**AI로 만들기**"는 AI로 진짜 것들을 만들고 싶은 사람들을 위해 쓰인 12편의 시리즈다 — 그냥 그것을 멋진 검색 엔진이나 약간 더 나은 자동완성으로 사용하는 게 아니라.

비전은 있지만 기술 공동창업자가 없는 창업자를 위해 쓰였다. 매일 같은 비효율성을 보고 소프트웨어가 고칠 수 있다는 걸 아는 운영 매니저. 고객이 필요한 정확한 도구를 만들고 싶은 컨설턴트. 교사, 디자이너, 어떤 종류든 도메인 전문가 — 문제를 깊이 이해하고 그 이해를 작동하는 것으로 변환하고 싶은 사람.

이 시리즈를 읽는 데 코딩 방법을 알 필요 없다. 기본 채팅 이상으로 AI를 사용해본 적이 없어도 된다. 신중하게 생각할 의향이 필요하다 — 당신의 문제, 데이터, 사용자, 실제로 만들고 싶은 것에 대해.

그게 전제조건이다.

이 시리즈가 아닌 것

프롬프트 핵 모음이 아니다. "상위 10가지 AI 도구" 목록이 아니다. AI가 언젠가 할 수 있을 것에 대한 과대표장이 아니다.

모든 편은 실제 사람들이 실제 것들을 만드는 데 사용하는 도구로, 2026년 지금 실제로 작동하는 것에 근거한다. 기술이 진정한 한계를 가지는 곳에서는 솔직하게 명명한다. 무언가가 실질보다 과대표장인 곳에서는 그렇게 말한다.

목표는 당신을 AI에 대해 흥분시키는 게 아니다. 목표는 당신을 AI로 효과적으로 만드는 것이다.

누가 쓰고, 왜 썼는가

이 시리즈는 송재희의 *AI Development Guide*에서 나왔다 — 20년 이상 엔터프라이즈 데이터 플랫폼을 구축해왔고, 수백 명의 학생에게 AI 개발과 바이트 코딩을 가르쳤으며, Seattle Partners를 통해 한국 기술 스타트업의 미국 시장 진출을 도운 실무자가 쓴 책.

이 책은 구체적인 좌절감 때문에 쓰였다: 기존 자료들이 너무 기술적이거나(엔지니어를 위해 쓰인) 너무 얕았다(팁을 원하는 사람들을 위해 쓰인). 개발자는 아니지만 진지하게 만들고 싶은 빌더를 위한 완전하고 솔직하고 실용적인 프레임워크가 없었다.

이 시리즈는 그 프레임워크의 핵심을 정리한다 — 편마다, 주제마다 — 각 편이 혼자 읽을 만한 충분한 실용적 깊이와, 열두 편 모두를 읽으면 전체 영역의 완전한 지도를 얻을 수 있는 충분한 넓이를 가지고.

열두 편

우리가 가는 곳:

AI 이해하기 (1~3편)

1편 — AI가 답답한 이유 대부분의 사람들이 결과를 얻지 못하는 진짜 이유: AI가 어떻게 작동하는지에 대한 그들의 생각과 실제 작동 방식 사이의 불일치. 모든 것을 바꾸는 사고 모델 전환 — 플러스 사람들이 뒤쳐진다고 느끼게 만드는 비교 나선과 그것에 대해 무엇을 해야 하는지.

2편 — AI가 실제로 하는 것과 못 하는 것 솔직하고 냉정한 분석: AI가 진정으로 탁월한 것(언어 변환, 패턴 합성, 규칙 기반 데이터 처리, 방대한 정보에서 찾기)과 안정적으로 실패하는 것(근거 없는 사실 정확도, 새로운 추론, 중요한 판단). MCP 도구 연결과 사고의 연쇄 추론 역량으로 업데이트됨.

3편 — AI 레고 스택 현대 AI 도구들이 세 레이어로 어떻게 맞물리는지 — 모델(두뇌), 오케스트레이션(조율자), 인터페이스(앞문). 2026년 현재 모델들, 스택 선택 방법, 그리고 왜 모델이 당신이 내릴 가장 덜 중요한 선택인지.

데이터와 프롬프트 작업 (4~6편)

4편 — 진짜 문제는 데이터다 AI에게 무엇을 넣느냐의 품질이 어떤 모델을 쓰느냐보다 결과의 품질을 결정하는 이유. AI 프로젝트를 망치는 네 가지 데이터 문제(일관성 없음, 불완전함, 오래됨, 맥락 붕괴)와 실용적인 데이터 준비도 체크리스트.

5편 — 기술보다 문제 정의가 성패를 가른다 Cursor, Bolt, Lovable, Claude Code가 설명할 수 있는 거의 모든 것을 만들 수 있는 시대에 — 병목은 완전히 원하는 것을 얼마나 정확하게 정의할 수 있느냐로 이동했다. 문제 정의 프레임워크와 여기서 10분의 명확성이 나중에 수시간의 재구축을 절약하는 이유.

6편 — 프롬프팅은 프로그래밍이다 프롬프트를 효과적으로 만드는 것의 80%를 커버하는 다섯 가지 핵심 프롬프트 패턴: 역할 + 작업 + 맥락, 포맷 지정, 제약과 안티패턴, 사고의 연쇄, 퓨샷 예시. 각각의 전후 예시와 함께.

구축하기 (7~9편)

7편 — AI 에이전트 에이전트를 만드는 것(챗봇이 아닌), 에이전트의 세 가지 레벨, n8n으로 첫 번째 에이전트 만들기, 그리고 지금 분야의 위치 — 에이전트 스웸, 결제 및 전화 역량을 가진 AI 직원, 컴퓨터 사용 에이전트, MCP가 이것을 어떻게 연결하는지.

8편 — 바이브 코딩 일상 언어로 실제 앱 만들기 — 도구들(Bolt, Lovable, Cursor, Claude Code, Windsurf, Replit), 실제 결과를 만드는 5단계 워크플로우, 리드 트래커 구축 단계별 워크스루, 바이브 코딩이 현재 잘 처리하는 것의 솔직한 한계.

9편 — AI 코드 어시스턴트 바이브 코딩과 전문 개발 사이의 다리: Cursor, Windsurf, GitHub Copilot, Claude Code, OpenAI Codex, Google Antigravity. 비개발자를 위한 다섯 가지 실용적 사용 사례, 실제로 작동하는 워크플로우, 플랜 모드와 CLAUDE.md 파일, 모델 티어링을 포함한 파워 유저 팁.

출시와 책임 (10~11편)

10편 — 데모에서 프로덕션으로 "데모에서 작동했다"가 "프로덕션에서 작동한다"와 같지 않은 이유. 프로덕션 준비도의 다섯 단계: 파일럿, 신뢰성 엔지니어링, 비용 아키텍처, 확장, 프로덕션 마인드셋. 실제 비용 수치, 모니터링 툴킷, 출시 전 체크리스트와 함께.

11편 — 리스크, 환각, 책임 있는 AI 빌더가 AI의 실패 모드에 대해 알아야 할 것: 네 가지 환각 유형, 낮은 이해관계에서 높은 이해관계까지 리스크 스펙트럼, 실용적인 안전장치 툴킷(RAG로 그라운드링, 신뢰도 신호, 범위 제한, 결과물 검증, 감사 로깅). 플러스 2026년의 법적, 윤리적 환경.

앞을 바라보며 (12편)

12편 — 다음 물결 기술이 실제로 향하는 곳: 자율 에이전트 네트워크, 피지컬 AI와 휴머노이드 로봇(지금 작동하는 것 vs. 아직 몇 년 남은 것), 지평선의 자기 개선 AI, 그리고 이것이 오늘의 빌더에게 의미하는 것. 추측이 아닌 현재 2026년 데이터에 근거.

이 시리즈를 읽는 방법

AI가 완전히 처음이라면 1편부터 순서대로 읽어라. 각 편은 이전 것 위에 쌓인다.

AI를 써왔지만 일관된 결과를 얻지 못하고 있다면 1편, 5편, 6편이 가장 레버리지가 높다. 사고 모델, 문제 정의, 프롬프팅 패턴.

특정 것을 자동화하고 싶다면 4편, 7편, 8편이 당신의 경로다. 데이터 준비도 → 에이전트 → 구축.

무언가를 출시하려 한다면 10편과 11편은 필수다. 프로덕션 준비도와 책임 있는 구축.

상황이 어디로 향하는지 이해하고 싶다면 12편부터 시작해서 뒤로 작업해라.

각 편은 완전하고 독립적인 읽기로 설계됐다 — 다른 것들을 읽지 않고도 어느 편에나 들어와서 가치를 얻을 수 있다. 하지만 함께하면 완전한 프레임워크를 형성하고, 편들 사이의 연결이 유용함의 일부다.

변화의 속도에 대한 메모

AI는 빠르게 움직이고 있다. 모델 버전이 바뀐다. 도구들이 등장하고 성숙해진다. 오늘 기적처럼 보이는 역량이 6개월 후에는 기본이 될 것이다.

기본은 변하지 않는다.

문제에 대해 어떻게 생각하는지. 그것을 정밀하게 정의하는 방법. 데이터 품질을 평가하는 방법. 효과적으로 프롬프팅하는 방법. 레이어로 구축하고 진행하며 테스트하는 방법. 신뢰성을 위해 설계하고 책임감 있게 구축하는 방법. 모든 코드 줄을 이해할 필요 없이 만들고 있는 것의 형태를 이해하는 방법.

이것들이 이 시리즈가 실제로 다루는 것들이다. 전반에 걸쳐 명명된 특정 도구들은 2026년 중반 현재 정확하다 — 하지만 기본 사고는 내구성이 있다. 당신이 읽을 때 이 시리즈의 모든 모델 이름이 구식이 되더라도, 프레임워크는 여전히 작동할 것이다.

시작하자

데모 간극 — 바이럴 영상에서 AI가 하는 것과 직접 얻는 것 사이의 거리 — 는 실재한다. 하지만 좁힐 수 있다. 그 간극의 반대편에 있는 사람들은 다른 도구를 쓰는 게 아니다. 그들은 그냥 문제에 대해 다르게 생각하고, AI와 더 정밀하게 소통하고, 더 규율 있게 구축하는 법을 익혔을 뿐이다.

그건 배울 수 있다. 그게 이 시리즈가 가르치는 것이다.

1편은 좌절감으로 시작한다 — 대부분의 사람들이 실제로 있는 곳이기 때문에 — 그리고 왜 그것이 일어나고 있는지, 그것에 대해 무엇을 해야 하는지 정확하게 설명한다.

시작하자.

"AI로 만들기"는 영어와 한국어로 제공된다. 이 시리즈는 송재희의 *AI Development Guide*에서 나왔으며, *Apple Books*, *Google Play Books*, 모든 주요 플랫폼에서 구매 가능하다.



Apple Books



Google Play Books



전 플랫폼 구매 (Books2Read)

AI가 답답하게 느껴지는 이유 — 그리고 모든 것을 바꾸는 사고 전환

7 min read · AI 이해하기

"AI로 만들기" 시리즈 1편

> **핵심 요약:** AI가 답답하게 느껴지는 이유는 대부분의 사람이 AI를 자판기처럼 — 요청을 넣으면 완성된 답이 나오는 것처럼 — 대하기 때문이다. 실제 AI는 맥락을 모르지만 똑똑한 협업자에 가깝고, 좋은 브리핑이 필요하다. 부러워하는 화제의 데모들은 수많은 실패한 시도와 정교한 브리핑을 감추고 있다. 해법은 더 좋은 도구가 아니라 더 명확한 사고다.

그 포스팅들을 봤을 것이다.

링크드인에서 누군가 주말 이틀 만에 SaaS 제품을 만들었다. 유튜브 영상에선 프롬프트 하나로 코드가 짜이고, 화면이 디자인되고, 앱이 배포된다. 10분짜리 영상이다. 트위터(X)에선 "AI로 이걸 5분 만에 했어요"라는 스레드가 수천 개의 리트윗을 받는다.

그리고 당신은 ChatGPT나 Claude를 열고, 필요한 걸 입력한다. 결과물이 온다. 뭔가 어긋나 있다. 조금 뻘하고, 살짝 틀리고, 엉뚱한 부분에서 자신감이 넘친다. 프롬프트를 수정하고 다시 시도한다. 조금 나아졌다. 결국 탭을 닫는다 — 해결책을 얻은 게 아니라, 막연한 실망감과 함께. 다들 뭔가를 알고 있는데 나만 모르는 것 같은 그 느낌을 안고.

이제 답답함이 두 겹이 됐다.

하나는 결과물 자체 — 원하는 게 안 나왔다. 다른 하나는 인정하기 더 어렵다. *다들 엄청난 결과를 얻고 있는 것 같은데, 나만 못 하고 있는 건 아닐까. 이미 뒤쳐진 건 아닐까.*

두 번째 답답함이 더 위험하다. 문제가 나 자신에게 있다는 느낌을 주기 때문이다 — 내 능력, 내 감각, 내가 AI를 "이해"하는지의 여부. 이 감각이 사람을 두 방향으로 몰아간다. 새로운 툴이 나올 때마다 쫓아가며 과잉 투자하거나, 아니면 "AI는 결국 과장이야"라며 손을 놓거나.

실제로 무슨 일이 일어나고 있는지 짚어보자.

데모와 현실 사이의 간극은 실재한다 — 그리고 의도적으로 보이지 않는다

바이럴 AI 데모는 진짜다. 그 결과물도 실제로 가능한 것들이다. 하지만 보이지 않는 것이 있다. 그 결과를 만들어낸 보이지 않는 작업들 — 정확한 브리핑, 공개된 것 전에 실패한 다섯 번의 시도, 무언가를 입력하기 전에 원하는 것을 이미 명확히 알고 있던 그 사람의 사고 과정.

공유되는 건 결과물이다. 그 전에 있었던 생각은 공유되지 않는다.

그래서 이미지가 왜곡된다. 마술사의 마지막 기술만 보고, 수년간의 연습은 보지 못한다. "주말에 만든 제품"을 보고, 무엇을 만들어야 하는지 정확히 알고 있었던 창업자의 수년간의 도메인 전문성은 보지 못한다. 완벽한 프롬프트 하나를 보고, 그 전에 실패한 열 개는 보지 못한다.

데모와 현실 사이의 간극은 당신의 AI 능력 차이가 아니다. 맥락(context)의 차이이다. 그리고 맥락은 쌓을 수 있는 것이다.

잘못된 사고 모델: AI를 자판기처럼 대하는 것

대부분의 사람들은 AI를 자판기처럼 대한다. 요청을 넣으면 결과물이 나와야 한다고 기대한다. 그게 안 되면 답답하다. 그리고 다들 코드를 해독한 것처럼 보이는 상황에서, 그 답답함은 배가 된다.

하지만 AI는 자판기가 아니다. 오히려 **오늘 처음 팀에 합류한, 엄청나게 똑똑하지만 맥락을 전혀 모르는 협업자**에 가깝다.

이런 사람을 상상해보자. 모든 책을 읽었고, 모든 프레임워크를 배웠고, 여러 언어에 능통하다. 하지만 당신을 모르고, 당신의 프로젝트를 모르고, 당신의 기준과 독자를 모르고, 지난 대화의 기억도 없다.

그런 사람에게 이렇게 말할 건가? *"보고서 써줘."*

당연히 아니다. 브리핑을 할 것이다. 배경을 설명하고, 목표를 말하고, 독자가 누구인지, '잘된 결과물'이 어떤 모습인지 알려줄 것이다. 예시를 보여주고, 함께 다듬어갈 것이다.

이것이 전환점이다. **AI는 버튼이 아니다. 제대로 된 브리핑이 필요한 협업자다.**

그리고 AI에서 놀라운 결과를 얻는 사람들? 그들은 그 브리핑을 잘 쓰는 법을 익힌 것이다. 그게 기술이다 — 그리고 배울 수 있는 기술이다.

실전에서 이 전환이 왜 중요한가

이 사고 전환을 하고 나면, AI와 일하는 방식이 완전히 달라진다.

전환 전: "내 제품 마케팅 이메일 써줘." → 뻔하고 무난한 결과물. 실망. 다시 링크드인을 열면 누군가의 AI 성공 스토리가 올라와 있다. 나선이 더 조여든다.

전환 후: 내 협업자가 이 일을 잘 하려면 뭘 알아야 하지?

그래서 이렇게 쓴다:

> "한국 IT 스타트업의 미국 시장 진출을 돕는 시애틀 기반 컨설팅 회사의 마케팅 이메일을 써야 해. 독자는 30~45세 스타트업 창업자들인데, 컨설팅 업체에 대한 경계심은 있지만 동료 추천엔 열려 있어. 말투는 직접적이고 약간 편안하면서도 자신감 있게, 영업 냄새는 없어야 해. 목표는 30분 미팅 예약이야. 전에 반응이 좋았던 이메일 하나 붙여줄게: [예시]"

같은 도구다. 결과는 완전히 다르다.

AI 결과물의 품질은 거의 항상 입력의 품질을 그대로 반영한다. 기술 실력이 아니고, 어떤 도구를 쓰느냐도 아니다. 맞는 강의를 들었는지, 맞는 유튜브를 봤는지도 아니다. **문제에 대해 얼마나 명확하게 생각했느냐다.**

비교의 함정에는 고유한 패턴이 있다

이건 명확하게 짚고 넘어갈 필요가 있다. AI를 둘러싼 비교 압박은 특히 값아먹는 방식이 있기 때문이다.

AI는 빠르게 움직이고 있다 — 진짜로. 몇 주마다 새로운 툴, 새로운 기능, 새로운 벤치마크가 나온다. 이게 특별한 불안을 만든다. 단순히 "나는 이게 서툰어"가 아니라, "가속하는 무언가에서 뒤처지고 있어"라는 감각이다. 이미 달리고 있는 기차에 올라타려는 것 같은데, 다른 사람들은 가볍게 뛰어오르는 영상을 찍고 있다.

하지만 이 가속이 실제로 의미하는 것은 이렇다. **도구는 당신이 뒤처지는 것보다 빠르게 쉬워지고 있다.**

지금의 Cursor는 1년 전의 Cursor보다 훨씬 쓰기 쉽다. Claude, ChatGPT, Gemini — 모두 불완전하고 짧은 프롬프트도 훨씬 잘 이해하게 됐다. 바닥이 계속 올라가고 있다. 올바른 사고 모델로 오늘 시작하는 사람이, 잘못된 모델로 1년 전에 시작한 사람보다 더 멀리 간다.

변하지 않는 것 — 오히려 시간이 지날수록 복리로 쌓이는 것 — 은 생각의 명확성이다. 문제를 정확하게 정의하는 능력. 도메인 지식. 무엇을, 왜 만들어야 하는지에 대한 이해.

그게 투자할 자산이다. 새 툴 릴리즈를 따라가는 것이 아니라.

간단한 연습: 브리핑 테스트

다음에 AI에게 프롬프트를 보내기 전, 스스로에게 세 가지를 물어보라.

1. 이걸 읽은 똑똑한 신입이 내가 무엇을 원하는지 이해할 수 있을까? 구글 검색어가 아니라 사람에게 주는 업무 지시로서. 검색 키워드처럼 들린다면, 맥락이 더 필요하다.

2. '잘된 결과물'이 어떤 모습인지 알려줬나? 제약 조건은 선물이다. "짧게 써줘"는 제약이 아니다. "출퇴근 2분 독자 기준, 150자 이내, 첫 문장에서 훅이 있어야 해"가 제약이다. 성공 기준이 명확할수록 AI는 훨씬 잘한다.

3. 예시나 참고 자료를 줬나? 내가 상상하는 것과 AI가 만드는 것 사이의 간극을 가장 빠르게 줄이는 방법은, 원하는 것에 가까운 예시를 보여주는 것이다. 설명 단락 여러 개보다 예시 하나가 낫다.

지금까지 보냈던 프롬프트 세 개를 이 기준으로 다시 봐라. 왜 어떤 건 됐고 대부분은 안 됐는지 바로 보일 것이다.

AI는 지름길이 아니라 사고 파트너다

더 미묘한 함정이 있다. 생각을 건너뛰는 지름길로 AI를 쓰는 것이다. 문제를 그대로 던지고, 답이 돌아오길 기다린다.

이건 거의 항상 실패한다. 대부분의 일에서 어려운 부분은 실행이 아니라 명확성이기 때문이다. 정확히 어떤 문제인가? 좋은 해결책은 어떤 모습인가?

이렇게 해보자. AI에게 무언가를 *해달라고* 하기 전에, 먼저 문제를 *정의하는 것*을 도와달라고 하라.

> "우리 팀 주간 보고 프로세스를 개선하고 싶어. 뭔가 만들기 전에, 지금 프로세스의 어떤 부분이 실제로 불편한 건지, 더 나은 버전은 어떤 모습일지 같이 생각해줄 수 있어?"

대부분의 경우, 생각했던 문제가 진짜 문제가 아니었다는 걸 발견하게 된다.

빌더로서 이게 의미하는 것

2026년, 도구는 병목이 아니다. Cursor, Claude, Bolt, Lovable — 3년 전엔 상상도 못 했던 속도로 진화하고 있다. 실행 능력은 그 어느 때보다 저렴하고 빠르다.

병목은 명확성이다. 무엇을, 누구를 위해, 왜 만들지를 — 강력한 도구가 실행할 수 있을 만큼 정확하게 정의하는 능력이다.

데모 보는 걸 멈춰라. 링크드인 포스팅과 나를 비교하는 걸 멈춰라. AI에서 놀라운 결과를 얻는 사람들은 당신과 다른 도구를 쓰는 게 아니다. 그들은 그냥 더 정확하게 생각하는 법을 익혔을 뿐이다.

그게 전부다. 그리고 그게 이 시리즈의 출발점이다.

지금 바로 해볼 것

AI로 하고 싶었지만 해보고 실망했거나, 어디서 시작할지 몰라 미뤄뒀던 것 하나를 떠올려라.

아직 AI 도구를 열지 마라.

먼저 이것을 적어라:

- 정확히 무엇을 만들거나 달성하고 싶은가?
- 누구를 위한 것이고, 그들에게 무엇이 필요한가?
- 좋은 결과물은 어떤 모습인가? 어떤 결과물을 보고 "맞아, 이거야"라고 할 수 있는가?
- 참고할 수 있는 예시나 레퍼런스가 있는가?

이제 도구를 열어라. 차이를 느껴라.

이것만 기억하자

하나도 기억 못 하더라도 이것만은 가져가라.

데모와 현실의 간극은 구조적인 것이지, 당신의 문제가 아니다. 온라인에서 보는 놀라운 결과들은 진짜다. 하지만 그 뒤에 있는 브리핑, 실패한 시도들, 명확한 목표 의식은 절대 공유되지 않는다. 당신이 뒤흔친 게 아니다. 하이라이트 영상만 보고 있는 것이다.

AI는 자판기가 아니라 협업자다. 결과물이 나오길 기대하는 것을 멈춰라. 대신 이렇게 생각하라. *내 협업자가 이 일을 잘 하려면 뭘 알아야 하지?* 이 하나의 전환이 모든 것을 바꾼다.

결과물의 품질은 생각의 품질을 그대로 반영한다. 기술 실력이 아니다. 어떤 도구를 쓰느냐도 아니다. 첫 글자를 입력하기 전에 문제에 대해 얼마나 명확하게 생각했느냐다.

명확성은 복리로 쌓인다. 툴 쫓기는 그렇지 않다. 도구는 계속 바뀐다. 하지만 문제를 정확히 정의하고, 독자를 이해하고, '잘된 결과물'이 어떤 모습인지 전달하는 능력 — 이걸 쓸수록 날카로워진다.

뒤흔치는 느낌은 진짜다. 하지만 그 원인은 능력의 차이가 아니라 접근 방식의 차이다. 그리고 접근 방식은 오늘 당장 바꿀 수 있다.

더 깊이 배우고 싶다면

이 글은 기본 사고 전환을 다뤘다. 송재희의 *AI Development Guide*는 훨씬 깊이 들어간다 — AI가 실제로 어떻게 생각하는지(강점, 실패 패턴, 환각이 왜 일어나는지)부터, 코드 없이 프로덕션 수준의 솔루션을 만드는 것까지.

AI가 할 수 있다는 것과 내가 실제로 얻고 있는 것 사이의 간극을 느껴본 적 있다면, 이 책이 그 다리다.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

다음 편: "AI가 실제로 하는 것과 못 하는 것" — AI가 진짜 잘하는 것과 구조적으로 실패하는 것을 냉정하게 살펴봅니다. 한계와 싸우는 걸 멈추고, 강점과 함께 일하는 법을 다룹니다.

AI가 실제로 하는 일 (그리고 하지 못하는 일)

11 min read · AI 이해하기

"AI로 만들기" 시리즈 2편

> **핵심 요약:** AI는 언어를 대규모로 다루는 일 — 요약, 재작성, 번역, 핵심 추출 — 과 방대한 텍스트에서 패턴을 찾고 종합하는 일에 탁월하다. 반면 실시간 사실 확인, 정확한 계산, 진정한 판단이 필요한 일에는 반복적으로 실패한다. 이 경계를 알면 AI의 한계와 싸우는 대신 강점에 기댈 수 있다.

AI를 처음 써본 사람들이 대부분 비슷한 경험을 한다.

어느 순간, 거의 마법처럼 느껴지는 일이 일어난다. 40페이지짜리 보고서를 30초 만에 핵심만 정리해준다. 어설픈 문단을 깔끔하게 다듬어준다. 복잡한 계약 조항을 일상 언어로 풀어준다.

그런데 바로 다음 세션에서, 아무렇지도 않게 틀린 말을 한다. 어떤 회사가 1987년에 설립됐다고 자신 있게 말하는데, 실제로는 2003년이다. 완벽해 보이는 코드를 짜줬는데, 실행하면 바로 오류가 난다. 논문 인용 목록을 줬는데, 그 논문들이 존재하지 않는다.

같은 도구다. 둘 다 오류가 아니다.

이게 AI를 쓰기 헛갈리게 만드는 핵심이다. AI는 동시에 두 극단에서 작동한다 — 어떤 영역에선 숨막히게 뛰어나고, 다른 영역에선 놀랍도록 취약하다. 그리고 어느 쪽인지 알려주는 신호가 없다.

대부분의 AI 답답함은 이 선을 모르는 데서 온다. 너무 믿어서 낭패를 보거나, 너무 안 믿어서 엄청난 가치를 놓치거나. AI가 진짜 잘하는 것과 구조적으로 실패하는 것을 이해하는 것 — 이건 단순한 지식이 아니라, 실제 결과를 얻기 위한 기본기다.

AI가 탁월하게 잘하는 것

1. 언어를 대규모로 다루는 것

이건 AI의 홈그라운드다. 요약, 재작성, 번역, 재구성, 핵심 추출, 톤 조정, 다른 독자를 위한 콘텐츠 변형 — 텍스트를 한 형태에서 다른 형태로 바꾸는 모든 것. AI는 이것을 어떤 사람보다 빠르고 일관되게, 어떤

분량에서도 해낸다.

고객 문의 이메일 50개를 받았는데 주요 패턴 다섯 가지를 찾아야 한다면? AI가 몇 초 안에 정리해준다. 같은 내용을 기술 독자용과 비기술 독자용으로 각각 써야 한다면? 동시에 해준다. 거친 초안을 다듬어야 한다면? AI는 탁월한 첫 번째 편집자다.

처음부터 쓰는 걸 멈춰라. AI로 초안을 만들고, 그걸 다듬어라. 어설픈 초안이라도 빈 페이지보다 훨씬 빠르게 완성된다.

2. 패턴 인식과 종합

AI는 사람이 수천 년을 읽어도 다 못 볼 분량의 텍스트를 학습했다. 그 덕분에 패턴을 발견하고, 서로 다른 분야를 연결하고, 여러 출처의 정보를 하나의 흐름으로 종합하는 능력이 뛰어나다.

어떤 문제에 대한 다섯 가지 접근 방식을 비교해달라고 해보라. 성공 사례들의 공통점을 찾아달라고 해보라. 주장 속의 구조적 모순을 짚어달라고 해보라. 이런 종합 작업 — 방대한 지식 안에서 점들을 연결하는 것 — 이 AI가 진짜 가치를 내는 영역이다.

리서치 종합에 AI를 써라. 여러 출처나 관점을 주고, 공통 흐름, 모순, 빠진 부분을 찾아달라고 요청해라.

3. 구조화된 시작점 만들기

빈 페이지 앞에서 굳어버리는 경험, 누구나 해봤을 것이다. AI는 여기서 탁월한 도구다. 프로젝트 계획, 회의 안건, 제안서 개요, 인터뷰 질문 목록, 의사결정 프레임워크 — AI는 사람이 30분 걸려 만들 구조화된 시작점을 몇 초 안에 만들어낸다.

핵심 단어는 "시작점"이다. 복잡한 작업에서 AI의 첫 번째 결과물이 최종 답인 경우는 드물다. 하지만 유용한 발판은 거의 항상 된다.

어디서 시작해야 할지 막힐 때, 먼저 구조를 요청해라. "X를 위한 개요를 줘" → 그 다음에 "X를 써줘" 순서로.

4. 복잡한 것을 쉽게 설명하기

AI는 놀랍도록 인내심 있는 선생님이다. 같은 개념을 다른 수준으로 설명해달라고 해보라. "호기심 많은 초등학생에게 설명하듯이"와 "MBA 수료자에게 설명하듯이"를 구분해서 정확하게 맞춰준다. 기술 개념을 요리 비유로 설명해달라고 해도 된다. 이해할 때까지 단계별로 설명해달라고 해도 된다. 짜증 안 내고, 질문을 판단하지 않는다.

중요한 미팅이나 결정 전에 AI로 연습해라. 모르는 분야를 빠르게 파악하는 데 이것보다 좋은 도구는 없다.

5. 명확한 규칙이 있는 데이터 처리

컴퓨터는 원래 규칙을 정확하고 대규모로 따르는 데 뛰어났다. AI는 그 강점을 물려받으면서, 데이터 주변의 맥락까지 이해하는 능력을 더했다. 논리가 명확하게 정의된 작업에서 AI는 진짜 신뢰할 수 있다. 세

금 시나리오 계산, 재무 데이터 분석, 수천 개 행에 일관된 서식 규칙 적용, 특정 기준을 위반하는 항목 플래그 처리, 구조화된 비교 실행 같은 것들이다.

핵심 표현은 "명확한 규칙"이다. 논리가 애매하지 않을 때 — X면 항상 Y — AI는 피로 없이 일관되게 처리한다. 회계사가 500개 항목을 한 시간 동안 검증할 세금 계산을 AI는 몇 초 안에 한다. 동일한 규칙을 1만 개 레코드에 적용해야 하는 데이터 품질 검사? 마찬가지다.

규칙을 명확하게 적을 수 있는 작업 — 정밀한 로봇이 따를 수 있을 만큼 — 은 AI가 대규모로 처리할 수 있는 작업이다. 데이터 검증, 수식 적용, 구조화된 분류, 컴플라이언스 체크를 생각해보라.

6. 방대한 정보에서 원하는 것 찾기

사람은 대량의 텍스트를 검색하는 데 서툴다. 훑어보고, 놓치고, 지치고, 처음에 찾은 것에 고착된다. AI는 이런 한계가 없다. 방대한 문서, 레코드 데이터베이스, 긴 이메일 스레드, 보고서 모음이 주어지면 — AI는 특정 정보를 찾고, 관련 구절을 꺼내고, 빠진 것을 파악하고, 불일치를 발견한다. 눈에 띄는 것만이 아니라 전체 내용을 대상으로.

이건 요약과 다르다. 요약은 압축한다. 찾기는 특정 것을 표면으로 끌어낸다. "이 200페이지 계약서에 우리 표준 약관과 충돌하는 방식으로 책임을 제한하는 조항이 있나요?"는 찾기 작업이다 — 그리고 AI는 이걸 탁월하게 해낸다.

AI를 문서 위의 검색 및 추출 레이어로 써라. 긴 계약서, 리서치 보고서, 회의 녹취록, 메모 모음을 주고, 요약이 아니라 찾기를 요청해라.

AI가 여전히 부족한 것 — 그리고 무엇이 달라졌나

한계를 살펴보기 전에 중요한 전제가 하나 있다. AI가 못 하는 것의 경계가 빠르게 이동하고 있다.

현대 AI 시스템은 이제 MCP(Model Context Protocol) 같은 프레임워크를 통해 외부 도구에 연결할 수 있다. 웹 검색, 실시간 데이터베이스 쿼리, 실시간 재무 데이터 조회, 파일 읽기, 외부 서비스와의 상호 작용이 가능하다. 먼 미래의 이야기가 아니다. 제대로 구성됐을 때 Claude, ChatGPT 같은 도구들이 오늘 실제로 작동하는 방식이다.

이것이 의미하는 것: 1년 전엔 절대적 한계처럼 느껴졌던 것들 중 일부가 이제는 부드러운 한계이거나 올 바른 설정으로 해결 가능한 것이 됐다. 어떤 것이 달라졌고, 어떤 것이 그대로인지 짚어보겠다.

1. 실시간 및 구체적 사실 정보 — 도구 연결로 크게 개선됨

AI의 고전적 한계: 학습 데이터에서 응답을 생성하지, 실시간 검색을 하지 않는다. 그래서 틀린 날짜, 조작된 통계, 오래된 정보를 완전한 자신감으로 내놓곤 했다.

이건 도구 없이 단독으로 사용하는 기본 AI 모델에서는 여전히 사실이다. 하지만 웹 검색이나 실시간 데이터 소스에 도구로 연결되면, AI는 응답하기 전에 정확하고 최신 정보를 가져올 수 있다. 도구가 연결된

AI에게 오늘 환율, 특정 회사의 최신 실적, 최근 뉴스를 물어보면 — 추측이 아니라 실제 답을 가져올 수 있다.

달라지지 않은 것: 도구가 있어도, AI는 찾아봐야 한다는 걸 인식하지 못할 때 또는 접근할 수 있는 소스에 정보가 없을 때 여전히 환각을 일으킬 수 있다. 자신감 문제가 완전히 사라진 건 아니다.

구체적이고 검증 가능한 사실 — 특히 최근 것 — 은 검색이나 도구 접근이 활성화된 AI 시스템을 써라. 그리고 중요한 것은 여전히 1차 출처에서 확인해라.

2. 새롭고 다단계적인 논리 추론 — 개선됐지만, 인간의 추론과는 여전히 다르다

이건 솔직하게 업데이트가 필요한 부분이다. 추론은 AI에서 가장 빠르게 발전한 영역 중 하나이기 때문이다.

OpenAI의 o1/o3, Google의 Gemini, Claude의 확장 사고 모드 같은 현대 "추론 모델"들은 **사고의 연쇄(Chain of Thought, CoT)** 라는 기법을 사용한다. 최종 답을 내기 전에 중간 단계를 생성하며, 말 그대로 "소리 내어 생각"한다. 수학, 논리, 코딩, 구조화된 다단계 문제에서 이건 진짜 도약이었다. 이전 모델들이 완전히 막혔을 복잡성을 이제 다룰 수 있다.

그러니 "AI는 추론을 못 한다"는 말은 더 이상 정확하지 않다. 인식 가능한 패턴과 정의된 구조가 있는 문제에서 AI 추론은 이제 인상적으로 신뢰할 수 있다.

하지만 몇 가지 중요한 간극은 여전히 남아 있다. 그리고 이건 더 많은 학습 데이터의 문제가 아니라 구조적인 차이이기 때문에 이해할 가치가 있다.

AI는 한 방향으로만 추론한다. 인간은 되돌아간다. 인간의 추론은 재귀적이다. 가설을 세우고, 머릿속으로 검증하고, 뭔가 어긋나는 느낌이 들면 되돌아가서 전제를 수정하고, 다시 시도한다 — 종종 의식적으로 결정하지 않고도. AI의 사고 연쇄는 대체로 한 번의 전진 생성이다. 각 단계는 이전 것을 기반으로 쌓인다. "재고하는" 것처럼 보여도, 인간이 이전 믿음을 진짜로 재방문하는 것이 아니라 다음으로 가장 그럴듯한 토큰을 생성하는 것이다. 그래서 AI가 틀린 방향으로 자신 있게 나아가다 한 번도 방향을 수정하지 않을 수 있다.

AI는 맞아야 한다는 이해관계가 없다. 인간의 추론은 결과에 의해 형성된다. 책임을 져야 할 때, 틀렸을 때 실제 비용이 있을 때 우리는 더 신중하게 추론한다. AI에게는 그런 마찰이 없다. 오류에 대한 두려움도, 자존심 투자도 없다. 이게 AI를 일관되게 만들지만, 동시에 인간이라면 중간에 느꼈을 내부 저항 없이 틀린 결론으로 자신 있게 걸어 들어갈 수 있다는 뜻이기도 하다.

자신감이 정확성을 의미하지 않는다. 이게 실용적으로 가장 위험한 간극이다. 추론 모델은 길고 상세한 사고 연쇄를 생성하고, 완전히 틀린 결론에 도달하면서 — 완전히 맞을 때와 정확히 같은 자신감 있는 말투를 쓸 수 있다. 추론의 길이와 겉보기 철저함이 답이 맞는지를 신뢰할 수 있게 나타내지 않는다. 인간 전문가라면 깊은 추론과 높은 자신감이 어느 정도 상관관계가 있다. AI에서는 그렇지 않다.

선례 없는 새로운 상황에서는 여전히 흔들린다. AI 추론은 이전에 본 것과 유사한 문제에서 빛난다. 진짜 새로운 상황 — 이례적인 엠티 케이스, 물리적 직관이 필요한 결정, 명확한 선례가 없는 사회적 역학 —에서는 여전히 어려워한다. 인간의 추론은 몸으로 체득되고 적응적이다. AI의 추론은 패턴 완성이고, 완성할 패턴이 없으면 발판이 흔들린다.

구조화되고 잘 정의된 문제에서는 추론 모델을 자유롭게 써라, 진짜 잘한다. 새롭고 중요하거나 결과가 큰 추론에서는 AI를 고려사항을 표면화하는 사고 파트너로 다뤄라, 최종 권위자가 아니라. 논리는 항상 스스로 검토해라. 그리고 AI가 가장 자신감 있게 들릴 때 특히 주의해라: 바로 그때가 "자신 있게 틀린" 실패 모드를 잡아내기 가장 어려운 순간이다.

3. 자신이 모른다는 것을 모르는 것 — 여전히 가장 위험한 한계

AI는 불확실성에 대한 신뢰할 수 있는 내부 신호가 없다. 사람 전문가는 지식 범위 밖의 질문을 받으면 보통 주저하거나 "잘 모르겠다"고 한다. AI는 종종 그러지 않는다. 빈 곳을 그럴듯하게 들리는 내용으로 채운다 — 완전히 만들어낸 것일 수 있고, 맞는 내용을 말할 때와 정확히 같은 자신감 있는 말투로.

이게 AI 커뮤니티에서 말하는 "환각(hallucination)"이다. 도구 접근은 사실 질문에서 환각을 줄여준다. 찾아볼 수 있기 때문이다. 하지만 근본적인 경향을 없애지는 못한다 — 특히 틈새 주제, 복잡한 세부사항, 어떤 소스도 답하기 어려운 경계 질문에서.

질문이 구체적이고 틈새적일수록, 자신 있는 답변에 더 의심해야 한다. 낯선 영역에서 AI 결과물은 결론이 아닌 시작 가설로 다뤄라.

4. 실제 이해관계가 있는 판단 — 도구가 도움이 안 되는 영역

이건 달라지지 않았고, 도구도 바꾸지 못한다. AI는 결과에 직접적인 이해관계가 없다. 회사를 운영해본 적도, 어려운 클라이언트 관계를 헤쳐나간 적도, 틀렸을 때 진짜 결과가 따르는 결정의 무게를 느껴본 적도 없다. 프레임워크를 제시하고, 고려사항을 나열하고, 반론을 펼칠 수 있다 — 하지만 실제 경험이 있는 사람의 판단을 대체할 수 없다.

"이 사람을 해고해야 하나?", "이 파트너십을 추진할 가치가 있나?", "이 투자자의 돈을 받아야 하나?" — 이건 정보 문제가 아니다. 살아있는 맥락, 관계 이력, 진짜 책임감이 필요한 판단이다.

AI를 놓친 고려사항을 발견하는 데 써라. 결정을 내리는 데 쓰지 마라. "이것의 찬반 논거는 뭐야?"가 "어떻게 해야 해?"보다 나은 질문이다.

신뢰 단계

이 패턴을 이해하면, 언제 AI를 믿고 언제 검증할지에 대한 실용적 감각이 생긴다.

자유롭게 신뢰해도 되는 영역 — 언어 변환(요약, 재작성, 번역), 브레인스토밍, 구조와 개요 생성, 개념 설명, 커뮤니케이션 초안 작성. 여기서 작은 오류의 비용은 낮고, 효율 이득은 크다.

신뢰하되 검증해야 하는 영역 — 리서치 종합, 전략 프레임워크, 기술 설명, 코드 생성. AI가 진짜 유용하지만, 결과물을 실행하기 전에 항상 검토해라.

주의하며 사용해야 하는 영역 — 구체적 사실, 통계, 인용, 법적/의학적 세부사항, 정확한 역사적 정보. 이 영역에서 AI 결과물은 결론이 아닌 리서치의 출발점으로 다뤄라.

의존하지 말아야 하는 영역 — 중요한 결정에 대한 최종 판단, 각 단계가 중요한 복잡한 새로운 추론, 틀렸을 때 심각한 결과가 있는 모든 것.

지금 바로 해볼 테스트

최근에 AI를 쓴 것 하나를 골라, 스스로에게 물어보라: 그 작업은 어떤 단계에 해당했나?

AI에게 요약이나 재작성을 요청했다면 — 안전한 영역이었다.

AI에게 구체적인 사실이나 데이터를 요청했다면 — 검증했는가? 그렇지 않다면, 다시 확인해봐라.

AI에게 중요한 결정을 도와달라고 했다면 — 그 결과물을 여러 입력 중 하나로 다뤘는가, 아니면 답으로 다뤘는가? 후자라면, 다시 생각해볼 필요가 있다.

대부분의 사람들은 생각해보면, 세 번째 영역에서 AI를 지나치게 믿거나, 첫 번째 영역에서 AI를 충분히 활용하지 못하고 있다는 걸 깨닫는다. 이 인식만으로 이미 일하는 방식이 달라진다.

AI의 한계가 뜻하는 것

AI의 한계는 AI의 가치를 줄이지 않는다. 그냥 게임의 규칙을 정의할 뿐이다.

계산기는 엄청나게 유용하다 — 하지만 전략 메모를 써달라고 하진 않을 것이다. GPS는 없어서는 안 될 도구다 — 하지만 제안하는 경로가 말이 안 될 때는 알아차린다. AI도 마찬가지다. AI가 잘하는 것과 못하는 것을 이해하는 건 비관주의가 아니다. AI를 제대로 쓰기 위한 전제 조건이다.

AI에서 놀라운 결과를 얻는 사람들은 AI의 약점을 무시하는 게 아니다. 맞는 작업을 맡기고, 판단은 자신이 하는 법을 익혔다.

이것만 기억하자

AI는 동시에 두 극단에서 작동한다. 어떤 영역에선 놀랍도록 뛰어내고, 다른 영역에선 놀랍도록 취약하다. 어느 쪽인지 아는 것이 기술이다.

AI의 홈그라운드인 언어 변환이다. 요약, 재작성, 재구성, 설명, 종합 — 텍스트를 한 형태에서 다른 형태로 바꾸는 것. 여기서 진짜 가치가 나온다.

AI는 자신이 모른다는 것을 모른다. 틀린 사실을 맞는 사실과 같은 자신감 있는 말투로 내놓는다. 질문이 구체적일수록 더 의심해야 한다.

신뢰 단계 프레임워크를 써라. 언어 작업은 자유롭게 신뢰. 종합과 코드는 신뢰하되 검증. 구체적 사실은 주의. 중요한 판단은 의존하지 말 것.

AI를 가장 잘 쓰는 사람은 가장 낙관적인 사람이 아니다. 어디에 배치할지, 어디서 직접 운전할지를 정확히 아는 사람이다.

더 깊이 배우고 싶다면

이 글은 AI가 실제로 어떻게 작동하는지 — 솔직한 버전을 다뤘다. 송재희의 *AI Development Guide*는 이 기반 위에서, 프롬프팅과 데이터부터 에이전트, 바이브 코딩, 프로덕션 배포까지 AI로 만드는 전체 여정을 보여준다.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

다음 편: "AI 레고 스택" — 현대 AI 도구들이 어떻게 맞물려 작동하는지, 그리고 만들고 싶은 것에 맞는 조합을 어떻게 고르는지를 다룹니다.

AI 레고 스택

8 min read · AI 이해하기

"AI로 만들기" 시리즈 3편

> **핵심 요약:** 모든 AI 솔루션은 세 개의 레이어로 이루어진다. **모델**(두뇌 — Claude, ChatGPT, Gemini), **오케스트레이션**(작업을 조율하고 자동화하는 층 — n8n, 에이전트), 그리고 **인터페이스**(사람이 사용하는 층 — 챗, 앱, Cursor 같은 에디터)다. 어떤 도구가 어느 레이어에 속하는지 알면 수십 개의 AI 제품 사이를 헤매는 혼란이 끝난다.

요즘 자주 일어나는 장면이 있다.

누군가 "AI를 업무에 써봐야겠다"고 결심한다. ChatGPT를 연다. 몇 가지를 시도한다. 되는 것도 있고 안 되는 것도 있다. Claude가 좋다는 말을 듣고 옮겨간다. 코딩엔 Cursor가 좋다는 말에 설치한다. 동료가 n8n으로 자동화해야 한다고 한다. 이미지엔 Midjourney가 있다. Make.com도 있다. 링크드인에 새 톨 포스팅이 올라온다. 어느새 브라우저 탭이 열일곱 개가 열려 있고, 이걸 과연 다 파악할 수 있을 까 하는 막막함이 밀려온다.

익숙한 장면인가?

문제는 도구가 아니다. 이 도구들이 어떻게 맞물리는지 아무도 설명해주지 않았다는 것이다.

구조를 한 번 보고 나면 — 각 레이어가 무엇을 하는지, 어떻게 연결되는지, 어떤 일에 무엇을 고르는지 — 전체 풍경이 압도적으로 느껴지는 것을 멈추고 빌딩 블록처럼 보이기 시작한다. 혼돈이 아니라 레고처럼.

현대 AI 스택의 세 레이어

모든 AI 기반 솔루션은 세 개의 뚜렷한 레이어로 이루어져 있다고 생각하라. 서로 위에 쌓인 구조다. 각 레이어는 역할이 있다. 각각은 아래 레이어에 의존한다.

레이어 1: 모델

레이어 3: 인터페이스

← 사람이 솔루션

하나씩 살펴보자.

레이어 1: 모델 — 두뇌

이게 AI 자체다. 언어를 이해하고, 추론하고, 텍스트를 생성하고, 코드를 쓰고, 문서를 요약하고, 질문에 답하는 대형 언어 모델(LLM)이다.

알아야 할 주요 플레이어들:

OpenAI — GPT-5.2 현재 플래그십으로, 코딩과 에이전틱 작업에 최적화됐다. GPT-5 mini와 GPT-5 nano는 잘 정의된 작업에 더 가볍고 빠르고 저렴한 대안이다. ChatGPT나 API를 통해 접근 가능하다.

Anthropic — Claude Opus 4.6, Claude Sonnet 4.6 Claude Opus 4.6은 Anthropic의 가장 강력한 모델이다 — 긴 문서, 미묘한 글쓰기, 복잡한 지시 따르기, 확장된 추론에서 탁월하며, 100만 토큰 컨텍스트 윈도우를 제공한다. Claude Sonnet 4.6은 일상적인 최적점이다: 빠르고, 매우 유능하며, 대부분의 사용 사례에서 비용 효율적이다. claude.ai나 API를 통해 이용 가능하다.

Google — Gemini 3.1 Pro / Gemini 2.5 Pro Gemini 3.1 Pro는 Google의 최신 최고 성능 모델로(현재 프리뷰 단계), 복잡한 문제 해결과 에이전틱 작업을 위해 구축됐다. Gemini 2.5 Pro는 현재 안정적인 프로덕션용 플래그십 — 강력한 추론, 매우 긴 컨텍스트 윈도우, Google 생태계(Docs, Sheets, Gmail)와의 깊은 통합. 텍스트, 이미지, 오디오, 비디오를 기본으로 다루는 강력한 멀티모달 역량을 갖추고 있다.

Meta — Llama 4 (오픈소스) Meta의 최신 오픈소스 모델 패밀리다. 무료이고 로컬 또는 자체 서버에서 실행 가능하다. 데이터 프라이버시, 비용 통제, 또는 독점 모델이 허용하지 않는 커스터마이징이 필요할 때 중요하다. Llama 4는 선도적인 독점 모델들과의 격차를 크게 줄였다.

xAI — Grok 4.20 xAI의 최신 플래그십으로, 업계 최고 수준의 속도와 에이전틱 톨 호출 역량을 갖췄다. X 검색을 통한 X(트위터) 데이터 실시간 접근이 독특한 강점이다 — 소셜 리스닝, 트렌드 분석, 실시간 공개 대화가 선호인 모든 것에 유용하다.

하나를 골라 영원히 고수할 필요가 없다. 모델마다 강점이 다르다. 많은 빌더들이 작업에 따라 다른 모델을 쓴다 — 깊은 추론과 장문 문서엔 Claude Opus 4.6, 코딩과 에이전틱 작업엔 GPT-5.2, Google 생태계 워크플로우엔 Gemini 3.1 Pro. 모델은 교체 가능하다. 프롬프트, 로직, 워크플로우 — 이게 투자할 가치가 있는 자산이다.

레이어 2: 오케스트레이션 — 조율자

모델 단독으로는 강력하지만 한계가 있다. 한 번에 하나의 질문에만 답한다. 매일 아침 자동으로 이메일을 확인하고, CRM에서 데이터를 가져오고, 처리하고, Slack에 요약を送낼 수 없다. 누군가 거기 앉아서 각 프롬프트를 수동으로 입력하지 않는 한 다단계 워크플로우를 실행할 수 없다.

오케스트레이션 레이어가 하는 일이 바로 그것이다. 조율한다: 올바른 시간에 AI를 트리거하고, 올바른 데이터를 제공하고, 다른 도구에 연결하고, 출력에 자동으로 행동한다.

이게 대부분의 사람들이 건너뛰는 레이어다 — 그리고 진짜 레버리지가 있는 곳이다.

n8n — 시각적 워크플로우 자동화 도구다. 캔버스에 노드(트리거, 액션, AI 호출)를 연결한다. X가 일어나면 Y를 하고, AI를 호출하고, Z를 한다. 오픈소스이고 자체 호스팅 가능하며, 통제권을 원하는 빌더에게 탁월하다. 대부분의 진지한 AI 자동화 워크플로우를 구동하는 것이 이것이다.

Make.com — n8n과 비슷하지만 더 초보자 친화적이고, 클라우드 기반이며, 사전 구축된 커넥터 라이브러리가 많다. 많은 기술적 설정 없이 빠르게 무언가를 실행하고 싶다면 Make가 출발점이다.

LangChain / LlamaIndex — AI가 데이터 및 도구와 상호작용하는 방식에 대한 세밀한 제어를 원하는 개발자 중심 프레임워크다. 개발자와 함께 작업하거나 코딩을 배우고 있다면 이것이 섬세한 제어를 제공한다.

AI 에이전트 — 점점 자체 오케스트레이션 레이어가 되고 있어 별도로 언급할 가치가 있다. 에이전트는 일련의 단계를 계획하고, 도구를 사용하고, 최소한의 인간 개입으로 목표를 완수할 수 있는 AI다 — 본질적으로 자체 단계를 파악하는 워크플로우다. 이 시리즈 후반부에서 에이전트 전체 포스트를 다룰 예정이다.

레이어 3: 인터페이스 — 앞문

이게 사용자가 실제로 보고 만지는 것이다. 인터페이스는 사람이 당신이 만든 것과 상호작용하는 방식이다.

채팅 인터페이스 — 가장 단순하다. 텍스트 박스, 응답. Claude.ai, ChatGPT, API 위에 구축된 커스텀 챗봇. 열린 대화에 좋다.

웹 앱과 대시보드 — Bolt, Lovable, Vercel v0, 또는 전통적인 개발로 구축된다. 채팅 창이 아닌 진짜 제품처럼 느껴지는 제대로 된 UI다.

임베디드 AI — 사용자가 이미 쓰는 기존 도구나 워크플로우에 내장된 AI. Google Docs의 문서를 요약하는 버튼. 보고서를 실행하는 Slack 명령어. 자동 분석을 트리거하는 이메일. 사용자는 "AI를 쓰고 있다"고 생각하지 않는다 — 그냥 도구를 쓸 뿐이다.

음성 인터페이스 — 빠르게 성장하고 있다. 듣고 응답하는 AI. 고객 서비스 봇, 음성 어시스턴트, 회의 전사 및 요약 도구들이다.

인터페이스는 종종 마지막에 구축하지만, 사용자가 가장 먼저 판단하는 것이다. 훌륭한 AI 워크플로우도 어설픈 인터페이스가 있으면 버려진다. 단순한 워크플로우도 깔끔하고 직관적인 인터페이스가 있으면 매일 쓰인다. 이 레이어를 과소평가하지 마라.

레이어들이 연결되는 방식: 실제 예시

구체적으로 만들어보자. 경쟁사 웹사이트를 모니터링하고, 변경된 내용을 요약하고, 주간 дай제스트를 보내주는 도구를 만들고 싶다고 가정하자.

레이어 1 (모델): Claude Sonnet 4.6 또는 GPT-5.2 — 경쟁사 콘텐츠를 읽고 새로운 내용과 왜 중요한지에 대한 명확하고 간결한 요약을 생성한다.

레이어 2 (오케스트레이션): n8n — 매주 월요일 아침 실행되어 경쟁사 URL을 가져오고, 콘텐츠를 AI 모델에 전달하고, 출력을 포맷하고, 이메일이나 Slack으로 보낸다.

레이어 3 (인터페이스): 이메일 받은편지함 또는 Slack 채널 — дай제스트가 도착하는 곳이다. 화려한 UI가 필요 없다; 인터페이스는 이미 시간을 보내는 곳이다.

이걸 구축하는 데 필요한 코딩: n8n의 시각적 인터페이스와 사전 구축된 이메일 커넥터를 사용한다면 제로다. 로직은 당신 것이다. AI가 읽고 요약한다. 오케스트레이션이 스케줄링과 라우팅을 한다. 인터페이스는 이미 구축돼 있다 — 받은편지함이다.

이게 레고 모델의 실제 작동이다. 각 조각은 역할이 있다. 조립하면 된다. 블록을 처음부터 만들 필요는 없다.

스택 선택 방법

이 모든 걸 한꺼번에 마스터할 필요는 없다. 현재 위치에 따른 실용적인 시작 프레임워크다:

막 시작하는 단계라면:

- 모델: ChatGPT 또는 Claude (채팅 인터페이스 직접 사용)
- 오케스트레이션: 아직 없음 — 워크플로우를 먼저 이해하기 위해 수동으로 해라
- 인터페이스: 채팅 창

무언가를 자동화하고 싶다면:

- 모델: API를 통한 Claude Sonnet 4.6 또는 GPT-5.2
- 오케스트레이션: Make.com (더 쉬움) 또는 n8n (더 강력함)
- 인터페이스: 이메일, Slack, 또는 간단한 웹 폼

진짜 제품을 만들고 싶다면:

- 모델: API를 통한 Claude Opus 4.6 또는 GPT-5.2, 다른 작업에 잠재적으로 여러 모델
- 오케스트레이션: 복잡한 워크플로우를 위한 AI 에이전트가 포함된 n8n 또는 LangChain
- 인터페이스: 깊은 코딩 없이 빠른 UI 구축을 위한 Bolt, Lovable, 또는 v0

일을 처리할 수 있는 가장 단순한 버전으로 시작해라. 필요가 명확해지면 레이어를 추가해라. 대부분의 사람들은 문제를 이해하기 전에 스택을 과도하게 설계한다. 먼저 문제를 이해해라 (1편으로 돌아가서), 그 다음 스택을 선택해라.

대부분의 사람이 저지르는 실수

문제를 이해하기 전에 도구를 고른다.

n8n에 대해 듣고 무엇을 자동화할지 알기도 전에 자동화를 구축한다. 그 수준의 복잡성이 필요한지 알기도 전에 LangChain을 설정한다. 작업에 맞는 것이 아니라 트렌딩인 것을 기준으로 모델을 선택한다.

스택은 사용 사례를 따라야 한다. 그 반대가 아니라.

어떤 도구를 선택하기 전에 물어보라: **해야 할 일이 무엇인가? 그 일에서 AI가 어디에 가치를 더하는가? 결과가 필요한 사람에게 어떻게 전달되는가?** 이 세 가지 질문은 레이어 1, 2, 3에 직접 매핑된다. 먼저 답하라. 그 다음 레고 조각을 선택해라.

이것만 기억하자

모든 AI 솔루션에는 세 개의 레이어가 있다: 모델, 오케스트레이션, 인터페이스. 어느 레이어에서 작업하고 있는지 — 그리고 무엇이 필요한지 — 이해하면 도구 압도감이 즉시 해소된다.

모델은 두뇌지만 전체 솔루션은 아니다. 모델 단독으로는 질문에 답한다. 오케스트레이션이 자동으로 행동하게 만든다. 인터페이스가 사용 가능하게 만든다. 진짜 무언가를 위해서는 셋 다 필요하다.

오케스트레이션 레이어에 대부분의 레버리지가 있다 — 그리고 대부분의 사람이 건너뛴다. n8n이나 Make.com 같은 도구로 AI를 트리거, 데이터 소스, 액션에 연결하는 것이 채팅 상호작용을 작동하는 시스템으로 바꾸는 것이다.

다른 작업에 다른 모델 — 교체 가능하다. 하나의 모델에 과도하게 헌신하지 마라. 프롬프트, 로직, 워크플로우가 진짜 자산이다. 모델은 교체 가능한 부품이다.

단순하게 시작하고 필요에 따라 레이어를 추가해라. 먼저 문제를 이해해라. 그 다음 해결하는 최소 스택을 선택해라. 복잡성은 기능이 아니라 비용이다.

더 깊이 배우고 싶다면

이 글은 아키텍처를 다뤘다. 송재희의 *AI Development Guide*는 각 레이어가 실제로 어떻게 작동하는지 더 깊이 들어간다 — 1인 운영자부터 엔터프라이즈 워크플로우까지 특정 사용 사례를 위해 구축된 스

택의 실제 예시들을 담고 있다.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

다음 편: "진짜 문제는 데이터다" — 어떤 AI를 쓰느냐보다 AI에게 무엇을 먹이느냐가 왜 더 중요한지,
그리고 데이터가 실제로 준비됐는지 어떻게 아는지를 다룹니다.

진짜 병목은 당신의 데이터

6 min read · 데이터와 프롬프트

"AI로 만들기" 시리즈 4편

> **핵심 요약:** AI에서는 모델보다 데이터가 더 중요하다. "쓰레기를 넣으면 쓰레기가 나온다"는 원칙은 여전히 유효하다. 일관성 없고 불안정하고 지저분한 정보를 주면 AI는 그럴듯하지만 흔들리는 토대 위에 세워진 결과를 자신 있게 내놓는다. 올바른 모델을 고르는 것은 대략 10%의 문제이고, 데이터를 제대로 갖추는 것이 대부분이 놓치는 90%의 문제다.

자주 반복되는 장면이 있다.

팀이 몇 주를 들여 AI 모델을 평가한다. GPT-5.2와 Claude Opus 4.6을 비교 테스트한다. 어느 쪽이 자신들의 use case에 더 맞는지 토론한다. 벤치마크를 읽고 시험을 돌린다. 최종적으로 모델을 결정하고 워크플로우를 구축한다.

그리고 실제 데이터를 넣는다 — 스프레드시트, CRM 내보내기, 고객 메모, 내부 문서들. 결과는 엉망이다. 일관성이 없고, 신뢰하기 어렵고, 가끔 쓸모 있고, 종종 틀렸다.

모델을 탓한다. 다른 모델로 바꾼다. 결과는 같다.

모델이 문제가 아니었다.

실제로 무슨 일이 일어나고 있는가

데이터 엔지니어링에는 AI보다 수십 년 먼저 생긴 원칙이 있다. **쓰레기를 넣으면 쓰레기가 나온다 (Garbage in, garbage out)**. 나쁜 데이터를 시스템에 넣으면 시스템이 아무리 정교해도 나쁜 결과가 나온다. 1980년대 데이터베이스에서도 사실이었고, 머신러닝 모델에서도 사실이었던. 오늘날 AI에서는 특히 그렇다.

차이점은 AI가 이 문제를 숨기는 데 유난히 뛰어나다는 것이다. 언어 모델은 일관성 없고 지저분하고 불완전한 데이터를 받아 *들리기에* 일관되고 자신감 있는 무언가를 만들어낼 수 있다. 공백을 채우고, 모순을 매끄럽게 넘기고, 그럴듯하게 들리는 결과물을 생성한다. 그 말은 AI 결과물에 따라 행동하려 할 때 그것이 불안정한 기반 위에 세워진 것임을 깨달을 때까지 AI가 잘 작동한다고 오랫동안 생각할 수 있다는 뜻이다.

AI 도구를 평가하는 대부분의 사람들은 모델을 평가하고 있다. 데이터를 평가해야 한다.

여기서 "데이터"가 실제로 의미하는 것

AI 맥락에서 데이터를 말할 때 스프레드시트와 데이터베이스만 의미하는 게 아니다. AI가 일을 하도록 넣는 모든 정보를 의미한다. 여기에 포함되는 것들:

- **정형 데이터** — 스프레드시트, CSV 파일, 데이터베이스 내보내기, CRM 레코드
- **비정형 텍스트** — 이메일, 회의 메모, 고객 피드백, 내부 문서, PDF
- **프롬프트에 제공하는 맥락** — 배경 정보, 지침, 예시
- **외부 소스** — 모델에 전달하기 위해 가져오는 웹사이트, API, 문서

이 모두가 데이터다. 그리고 모두 AI가 할 수 있는 것에 영향을 미치는 품질 문제를 가지고 있다.

AI 프로젝트를 망치는 4가지 데이터 문제

1. 일관성 없음

가장 흔하고 가장 치명적이다. 같은 것이 다섯 가지 다른 방식으로 묘사된다: "서울", "Seoul", "서울시", "서울특별시", "SEOUL". 한 시스템에서는 "활성"인 고객 상태가 다른 시스템에서는 "active"다. 어떤 레코드에는 공백이 있고 다른 레코드에는 하이픈이 있는 제품명.

사람은 일관성 없음을 자동으로 처리한다 — 뇌가 힘들이지 않고 정규화한다. AI는 그렇지 않다. "활성"과 "active"를 잠재적으로 다른 것으로 취급한다. 패턴 변형을 알아차리고 혼란스러워하거나 하류에서 오류를 복합시키는 가정을 만든다.

테스트: 데이터의 핵심 필드 — 카테고리, 상태, 이름 — 를 골라 같은 값의 변형이 몇 개 존재하는지 세어 보라. 두세 개 이상이면 일관성 문제가 있다.

2. 불완전함

빠진 필드. 빈 셀. 값이 있어야 할 곳에 "N/A". 입력하는 사람이 바꿨거나 시스템이 요구하지 않아서 절반만 채워진 레코드들.

AI는 불완전한 데이터로 작업할 수 있다 — 하지만 공백을 가정으로 채울 것이다. 그 가정이 합리적인 경우도 있다. 종종 그렇지 않다. 그리고 어느 쪽인지 항상 알 수 있는 건 아니다.

불완전함이 *체계적*일 때 위험이 배가된다 — 같은 유형의 레코드에서 항상 같은 필드가 빠질 때. AI가 같은 잘못된 가정을 반복해서, 일관되게, 대규모로 만든다는 의미다.

테스트: 레코드의 몇 퍼센트가 모든 핵심 필드를 채웠는가? 80% 미만이면 AI가 사실보다 가정으로 더 많이 작동하게 된다.

3. 오래됨

데이터에는 유효기간이 있다. 2년 전 고객 레코드는 잘못된 주소, 예전 직함, 더 이상 작동하지 않는 전화 번호가 있을 수 있다. 2022년에 작성된 제품 설명은 더 이상 존재하지 않는 기능을 묘사할 수 있다. 18개월 전 시장 분석은 다른 시장을 설명하고 있다.

오래된 데이터를 AI에 넣고 현재의 것을 요청하면 — 추천, 제안서, 고객 상황 요약 — 그 오래된 정보로 작업해 권위 있게 들리지만 현실의 구식 그림 위에 세워진 결과물을 만들어낸다.

테스트: 각 주요 데이터 소스가 마지막으로 검증되거나 업데이트된 게 언제인가? 모른다면, 그게 답이다.

4. 맥락 붕괴

가장 미묘한 문제다. 데이터가 당신에게는 완벽하게 이해된다 — 데이터가 담지 못하는 수년간의 맥락을 가지고 있기 때문이다.

"고객 보류 중"은 팀에게 특정한 의미다: 서비스 일시정지를 요청했거나, 하드웨어 배송을 기다리거나, 계약 갱신 협상 중이다. 하지만 그 문구 단독으로 AI에게 전달하면 수십 가지 의미 중 하나일 수 있다. AI는 그것을 올바르게 해석할 조직적 맥락이 없다.

내부인에게는 완벽하게 이해되지만 외부 독자에게는 — AI를 포함해 — 불투명한 약어, 내부 용어, 줄임 말, 코드에서 특히 그렇다.

테스트: 레코드 샘플을 무작위로 골라 사업에 익숙하지 않은 사람에게 해석하도록 요청하라. 혼란스러워하는 모든 곳이 맥락 붕괴 위험이다.

데이터 준비도 체크리스트

데이터 위에 AI 워크플로우를 구축하기 전에 이 체크리스트를 실행하라:

일관성

- [] 핵심 범주형 필드가 표준화된 값을 사용함 (같은 것의 여러 변형이 아니라)
- [] 이름, ID, 참조가 시스템 간에 일치함
- [] 날짜 형식이 일관됨
- [] 텍스트 필드에 별도 필드에 있어야 할 정형 데이터가 없음

완전성

- [] 핵심 필드가 80% 이상 채워져 있음

- [] 누락된 값이 명시적으로 표시됨 (비어있지 않고, 일관성 없는 "N/A"나 "미상"이 아니라)
- [] 필수 필드가 습관적으로 비워지지 않음

신선도

- [] 각 데이터 소스가 마지막으로 검증된 시기를 알고 있음
- [] 오래된 레코드가 표시되거나 보관되어 있고 현재 것과 섞이지 않음
- [] 시간에 민감한 필드 (연락처, 가격, 상태)에 검토 주기가 있음

맥락

- [] 약어와 내부 용어가 어딘가에 문서화되어 있음
- [] 상태 코드와 카테고리에 정의가 있음
- [] 레코드가 내부 조직 맥락 없이도 이해 가능함

이 목록의 모든 것을 체크할 수 있다면 데이터가 AI 준비가 된 것이다. 절반 미만을 체크하고 있다면 어떤 모델을 쓰더라도 AI 결과물이 신뢰할 수 없을 것이다.

어떻게 해결할 것인가

좋은 소식이 있다. AI 자체가 데이터를 정리하고 준비하는 탁월한 도구다. 프로세스는 이렇다:

1단계: 먼저 감사하고, 그 다음 수정하라. 무작위로 데이터 정리를 시작하지 마라. AI를 사용해 감사하라 — 샘플을 넣고 일관성 없음, 누락된 패턴, 모호한 값을 식별하도록 요청하라. 어떻게 수정할지 결정하기 전에 문제를 표면화하도록 해라.

2단계: 영향이 큰 필드부터 먼저 표준화하라. 모든 것을 고칠 필요는 없다. AI 워크플로우가 실제로 사용할 필드에 집중하라. 고객 접근 도구를 구축한다면 고객 이름, 상태, 연락처 정보가 내부 메모보다 더 중요하다. 전체 데이터셋이 아니라 중요 경로를 수정하라.

3단계: 표준화한 것을 문서화하라. 각 필드에 허용되는 값, 각 상태의 의미, 약어가 뜻하는 것을 담은 짧은 참조 문서가 몇 시간의 정리보다 가치 있다. 문제가 재발하는 것을 방지하고 AI에게 참조할 것을 제공한다.

4단계: 신선도를 프로세스에 내장하라. 데이터 품질은 일회성 프로젝트가 아니다. 쇠퇴한다. 간단한 검토 주기를 구축하라 — 느리게 변하는 데이터는 분기별, 빠르게 변하는 것은 월별 — 같은 오래된 문제를 반복해서 해결하지 않도록.

올바른 모델보다 올바른 데이터

올바른 AI 모델 선택은 10% 문제다. 데이터를 올바르게 하는 것은 90% 문제다.

이건 AI에 대한 비판이 아니다. 정보 집약적인 작업에서 진짜 작업이 어디에 있는지를 반영한다. 모델은 도구다. 데이터는 원재료다. 나쁜 재료로 작업할 때 도구를 탓하는 장인은 없다.

AI에서 일관되게 좋은 결과를 얻는 빌더들이 반드시 최고의 모델을 쓰는 건 아니다. 그들은 깨끗하고 일관되며 잘 문서화된 데이터를 사용하고 있다 — 그리고 그것을 유지하는 습관을 구축했다.

이게 AI 데모 세계가 절대 보여주지 않는 멋없는 진실이다. 데모는 완벽하게 사전 정리된 예시 데이터를 사용한다. 당신의 현실은 더 지저분하다. 그 지저분함을 고치는 것이 AI를 어떤 것에 추가하기 전에 할 수 있는 가장 높은 레버리지 작업이다.

더 깊이 배우고 싶다면

이 글은 데이터 기반을 다뤘다. 송재희의 *AI Development Guide*는 더 깊이 들어간다 — 특정 AI 사용 사례를 위해 데이터를 구조화하는 방법, AI를 사용해 가진 것을 정리하고 풍부하게 하는 방법, 데이터 품질이 AI 스택의 모든 레이어와 어떻게 연결되는지.

 [Apple Books](#)  [Google Play Books](#)  전 플랫폼 구매 (Books2Read)

다음 편: "기술보다 문제 정의가 성패를 가른다" — 거의 무엇이든 만들 수 있는 도구들의 시대에 왜 병목이 얼마나 정확하게 원하는 것을 표현할 수 있느냐로 완전히 이동했는지를 다룹니다.

진짜 기술은 코딩이 아니다

7 min read · 데이터와 프롬프트

"AI로 만들기" 시리즈 5편

> **핵심 요약:** AI로 만들 때 진짜 기술은 코딩이 아니라 원하는 것을 정확히 정의하는 능력이다. 모호한 입력은 살짝 잘못된 문제에 대한 자신 있고 잘 정돈된 답을 만들어 내고, 그 오류는 이미 그 위에 무언가를 쌓은 뒤에야 드러난다. 문제 정의를 다듬는 데 쓴 10분이 나중의 몇 시간을 아껴 준다.

지난 2년 사이 놀라운 일이 일어났다.

소프트웨어를 만드는 도구들이 너무 강력해져서 이제 만드는 행위 자체가 더 이상 어려운 부분이 아니다. Cursor가 코드를 쓴다. Bolt가 앱의 뼈대를 잡는다. Lovable이 UI를 생성한다. Claude Opus 4.6이 아키텍처를 추론한다. 원하는 것을 설명하면 — 종종 몇 분 안에 — 작동하는 무언가가 나타난다.

이건 해방감을 줘야 한다. 많은 사람에게 실제로 그렇다.

하지만 이 진보 안에 숨어 있는 문제가 있다. 만드는 비용이 거의 0에 가까워지면서 다른 무언가가 병목이 됐다 — 항상 거기 있었지만, 만드는 것이 어려울 때는 무시하기 쉬웠던 것.

이제 병목은 이것이다: 무엇을 만들고 싶은지 명확하게 정의할 수 있는가?

기술적으로가 아니다. 코드로가 아니다. 평범한 언어로. 강력한 도구가 — 혹은 팀에 막 합류한 똑똑한 사람이 — 스무 개의 명확화 질문을 하지 않고도 실행할 수 있을 만큼 충분히 정밀하게.

그 능력은 들리는 것보다 어렵다. 그리고 AI에서 놀라운 결과를 얻는 사람과 계속 답답한 결과를 얻는 사람을 가르는 것이 바로 이것이다.

왜 모호한 입력이 모호한 출력을 만드는가

문제 정의의 정밀도와 AI가 만들어주는 것의 품질 사이에는 직접적인 관계가 있다.

이건 AI에만 국한된 게 아니다. 모든 창의적이거나 기술적인 협업에서 사실이다. 디자이너를 고용하고 "멋있게 만들어줘"라고 하면 당신의 멋있음이 아닌 그들의 해석을 받게 된다. 개발자에게 "주문을 처리하는 걸 만들어줘"라고 브리핑하면 예상치 못한 방식으로 주문을 처리하고 — 당신이 당연하다고 생각한 방식으로로는 처리하지 않는 시스템을 받게 된다.

AI는 이 역학을 증폭시킨다. 언어 모델은 공백을 채우는 데 탁월하다 — 하지만 채우는 공백은 당신의 특정 맥락, 당신의 사용자, 당신의 제약, 당신의 성공 정의가 아닌 학습된 모든 것의 패턴 매칭을 기반으로 한다.

모호한 문제를 AI에게 주면, AI는 당신이 실제로 가진 문제와는 약간 다른 문제에 대한 자신감 있고 잘 구조화된 답을 준다.

그리고 여기에 함정이 있다. 맞아 보인다. 맞게 들린다. 표면 검사를 통과한다. 그 위에 구축하기 시작한다 — 그리고 세 단계 후에 기반이 어긋났다는 걸 깨닫는다.

잘 정의된 문제가 실제로 어떻게 생겼는가

대부분의 사람들은 문제를 정의하는 것이 더 긴 설명을 쓰는 것을 의미한다고 생각한다. 그렇지 않다. 길이는 정밀도가 아니다.

잘 정의된 문제에는 네 가지 구성 요소가 있다:

구체적인 상황 — "영업 프로세스를 자동화하고 싶어"가 아니라 "잠재 고객이 응답하지 않았을 경우 영업 통화 48시간 후 우리가 논의한 내용을 기반으로 개인화된 후속 이메일을 자동으로 보내고 싶어."

구체적인 사용자 — "고객들"이 아니라 "지난 30일 이내에 가입했고 아직 첫 번째 데이터 소스를 연결하지 않은 소규모 사업주들."

구체적인 제약 — 경계는 무엇인가? 할 수 없는 것은? 항상 해야 하는 것은? "주당 한 번 이상 후속 발송을 해서는 안 된다"는 제약이다. "자동화된 것처럼 느껴지지 않고 개인적으로 느껴져야 한다"는 제약이다.

구체적인 성공 조건 — 어떻게 작동했다는 걸 알 수 있는가? "후속 이메일의 응답률이 높아진다"는 성공 조건이다. "주당 두 시간을 절약해준다"는 성공 조건이다. "우리를 당황스럽게 만들지 않는다"도 성공 조건이다.

네 가지 없이는 방향은 있지만 정의는 없다. 그리고 방향은 구축하기에 충분하지 않다.

전과 후: 같은 목표, 다른 결과

실제 예시로 구체적으로 만들어보자.

모호한 버전: "팀이 고객 불만을 관리하는 데 도움이 되는 도구를 만들어줘."

무언가를 만들어낼 것이다. 인상적으로 보일 수도 있다. 하지만 "고객 불만 관리"가 티켓 시스템, AI 자동 응답기, 분류 도구, 대시보드, 에스컬레이션 워크플로우, 또는 스무 가지 다른 것을 의미할 수 있기 때문에 거의 확실히 진짜 문제를 놓칠 것이다.

정밀한 버전: "우리는 매주 이메일로 약 50건의 고객 불만을 받는다. 현재 팀이 각각을 수동으로 읽고 인간 응답이 필요한지 단순 확인만 필요한지 결정하고 적절한 담당자에게 배정한다. 이게 하루에 약 3시간이 걸린다. 수신 불만 이메일을 읽고 긴급도(긴급 / 표준 / 참고)로 분류하고 인간 검토를 위한 첫 번째 응답 이메일 초안을 작성하고 카테고리에 따라 적절한 팀원에게 라우팅하는 도구가 필요하다. 자동으로 아무것도 보내서는 안 된다 — 모든 것이 발송 전 인간 승인을 거쳐야 한다."

같은 목표. 완전히 다른 브리핑. 두 번째는 개발자, AI 에이전트, Bolt나 n8n을 가진 비기술 빌더 누구에게나 건네도 실제로 문제를 해결하는 무언가를 만들어낼 것이다.

차이는 기술 능력이 아니다. 사고 능력이다.

왜 이게 어렵고 (왜 대부분의 사람이 건너뛰는가)

문제를 정밀하게 정의하는 것은 불편한 작업이다. 모든 답을 알기 전에 결정을 내리도록 강요한다. 만들고 있다고 깨닫지 못했던 가정들을 표면화한다. 자신이 가졌다고 생각한 문제가 실제 문제가 아닐 수 있다는 걸 드러낸다.

대부분의 사람이 건너뛰는 이유:

만드는 것은 진전처럼 느껴진다. 정의하는 것은 지연처럼 느껴진다. Cursor나 Lovable을 열고 원하는 것을 설명하는 순간, 무언가가 일어나고 있다. 화면이 채워진다. 생산적으로 느껴진다. 어떤 도구도 건드리기 전에 30분을 문제 정의 작성에 쓰는 것은 낭비된 시간처럼 느껴진다 — 특히 도구들이 이렇게 빠를 때.

이건 정확히 반대다. 10분의 명확한 문제 정의가 잘못된 것에 대한 수시간의 반복을 절약한다. 도구는 빠르게 만든다; 비용은 만든 것을 맞닥뜨리고, 정확하지 않다는 것을 깨닫고, 다시 만드는 것이다.

모호함이 더 안전하게 느껴지기도 한다. 모호한 브리핑은 결과가 너그럽게 해석될 여지를 남긴다. 구체적이지 않으면 구체적으로 틀릴 수 없다. 정밀도는 헌신을 요구한다 — 그리고 헌신은 실제로 원하는 것에 대한 자신감을 요구한다.

그리고 진짜 문제는 종종 불편하다. 문제를 정밀하게 정의하는 과정이 진짜 문제가 생각했던 것이 아님을 드러낼 때가 있다. "더 나은 불만 관리 도구가 필요해"는 때때로 "지원 인력이 너무 부족해"를 의미한다. "보고를 자동화해야 해"는 때때로 "우리가 실제로 무엇을 측정하려는지 모른다"를 의미한다. 정밀한 문제 정의는 기술적 문제보다 직면하기 더 불편한 조직적 진실을 표면화할 수 있다.

문제 정의 프레임워크

어떤 AI 도구도 열기 전에 이 다섯 가지 질문에 답하라. 답을 적어라 — 머릿속에만 두지 마라.

1. 현재 상황은 무엇인가? 지금 일어나고 있는 것을 구체적이고 명확한 용어로 설명하라. 가능한 곳에서 숫자로. 누가 무엇을, 얼마나 자주, 얼마나 오래 하는가?

2. 고통은 무엇인가? 구체적으로 무엇이 잘못되고, 너무 오래 걸리고, 너무 많은 비용이 들거나, 일어나야 하는데 일어나지 않는가? "비효율적이야"가 아니라 — 구체적인 비효율성은 무엇인가?

3. 누가 이것을 경험하는가? 구체적인 역할, 구체적인 맥락. "사용자들"이 아니라 — 어떤 사용자들이, 무엇을 하면서, 언제?

4. 이상적인 결과는 어떻게 생겼는가? 이게 완벽하게 해결된다면 구체적인 시나리오를 설명하라. 단계별로 걸어가라. 지금은 일어나지 않는 무엇이 일어나는가?

5. 하드 제약은 무엇인가? 변할 수 없는 것은? 솔루션이 항상 하거나 절대 하지 말아야 하는 것은? 기술적으로 맞는 솔루션도 여전히 받아들이 수 없게 만드는 것은?

다섯 가지 모두 명확하게 답할 수 있을 때 문제 정의가 있다. 그 정의가 브리핑이다. Claude Opus 4.6, Cursor, Bolt에 넣어라 — 돌아오는 것의 품질이 얼마나 극적으로 달라지는지 보라.

도구는 전략이 아니다

Cursor, Claude, Bolt, Lovable이 설명할 수 있는 거의 모든 것을 만들 수 있는 시대에 경쟁 우위는 완전히 설명 품질로 이동했다. 사고 품질으로. 해결하는 문제에 대한 이해의 정밀도로.

이것은 2026년 가장 중요한 AI 기술이 프롬프트 엔지니어링이 아니라는 것을 의미한다. 어떤 모델을 쓸지 아는 것이 아니다. API나 자동화 도구를 이해하는 것이 아니다.

도구를 집기 전에 문제에 대해 명확하게 생각하는 능력이다.

그 능력은 항상 중요했다. 모든 훌륭한 제품, 모든 성공적인 자동화, 모든 유용한 도구는 해결하려 하기 전에 문제를 깊이 이해한 누군가로부터 시작됐다.

변한 것은 증폭이다. 만드는 것이 느리고 비쌀 때, 다소 모호한 문제 정의도 여전히 유용한 무언가를 만들었다 — 빌드 프로세스의 마찰이 길을 따라 명확화를 강요했기 때문에. 이제 도구는 즉시 만든다. 모호함이 끝까지 유지된다. 잘못된 질문에 대한 빠르고 잘 실행된 답을 받게 된다.

이기는 빌더는 처음에 속도를 늦추는 사람들이다 — 정밀하게 정의하기 위해 — 그래서 빌드 내내 자신감 있게 빠르게 움직일 수 있다.

연습: 한 단락 브리핑

AI로 만들거나 자동화하고 싶었던 것을 하나 골라라. 진짜인 것 — 가상의 것이 아니라.

위 프레임워크의 다섯 가지 질문 모두에 답하는 한 단락을 써라. 구체적으로. 숫자를 사용해라. 실제 사용자를 명명하라. 실제 제약을 명명하라.

그리고 다시 읽어라. 스스로에게 물어라: 나를 만난 적 없고, 내 사업을 본 적 없는 사람이 이 설명만으로 내가 필요한 것을 정확히 만들 수 있는가?

답이 아니라면 — 계속 다듬어라. 작업은 다듬는 데 있다.

답이 예라면 — 도구를 열어라. 준비됐다.

더 깊이 배우고 싶다면

이 글은 문제 정의 레이어를 다뤘다. 송재희의 *AI Development Guide*는 더 깊이 들어간다 — 날카로운 문제 정의를 효과적인 프롬프트, 워크플로우, 완전한 AI 솔루션으로 변환하는 방법을 다양한 도메인과 사용 사례에 걸친 실제 예시와 함께 담고 있다.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

다음 편: "프롬프팅은 프로그래밍이다" — 날카로운 문제 정의가 생기면 원하는 결과를 일관되게 얻는 프롬프트로 변환하는 방법을 다룹니다.

프롬프팅은 프로그래밍이다

8 min read · 데이터와 프롬프트

"AI로 만들기" 시리즈 6편

> **핵심 요약:** 프롬프팅은 프로그래밍이다. 기계에 정확한 지시를 내리는 일이지만, 프롬프트는 문자 그대로 실행되지 않고 *해석*되기 때문에 모든 빈틈을 AI가 추측으로 채운다. 다섯 가지 패턴이 효과적인 프롬프팅의 약 80%를 커버한다. 역할 + 작업 + 맥락, 형식 지정, 제약, 사고의 연쇄(chain of thought), 그리고 퓨샷 예시다.

끈질기게 퍼져 있는 오해가 있다. 프롬프팅은 그냥 "AI에게 말 거는 것"이라는 생각.

자연스럽게, 일상 언어로 뭔가를 입력하고 AI가 이해하면 이미 잘 하고 있는 거라는 생각. 프롬프팅은 쉬운 부분 — 생각 없이 누구나 할 수 있는 것이라는 생각.

이 오해가 사람들의 엄청난 시간을 낭비하고 평범한 결과를 만들어내고 있다.

프롬프팅은 말 거는 게 아니다. 프롬프팅은 *지시/하는* 것이다. 그리고 모든 정밀한 지시의 형태와 마찬가지로 — 법적 계약 작성, 디자이너 브리핑, 개발자를 위한 테스트 케이스 작성 — 지시의 품질이 결과의 품질을 결정한다.

AI의 일반 사용자와 파워 유저의 차이는 쓰는 도구나 선택하는 모델이 아니다. 프롬프트의 품질이다. 그리고 프롬프트 품질은 배울 수 있는 기술이다 — 알고 나면 모든 것을 바꾸는 소수의 패턴들과 함께.

이 글이 그 패턴들을 전한다.

왜 프롬프팅이 프로그래밍인가

개발자가 코드를 쓸 때, 기계에 정밀하고 모호하지 않은 지시를 내린다. 뭔가 모호하게 두면 코드가 깨지거나, 더 나쁜 경우 의도하지 않은 일을 한다.

프롬프팅도 같은 방식으로 작동한다 — 한 가지 중요한 차이점이 있다. 코드는 문자 그대로 실행된다. 프롬프트는 해석된다. AI가 패턴을 기반으로 공백을 채운다 — 그 패턴이 당신의 의도와 맞을 수도 있고 아닐 수도 있다.

이것이 의미하는 것:

- **정밀도가 중요하다** — 구체적인수록 AI가 추측해야 할 것이 적다
- **구조가 중요하다** — 프롬프트를 구성하는 방식이 AI가 처리하는 방식에 영향을 미친다
- **맥락이 중요하다** — 작업 전에 AI에게 말하는 것이 AI가 작업에 접근하는 방식을 형성한다
- **예시가 중요하다** — 보여주는 것이 말하는 것보다 더 신뢰할 수 있다

프롬프팅을 이렇게 생각하라: 쓰여진 모든 것을 읽었지만 당신, 당신의 맥락, 당신의 상황에서 "좋은 것"이 어떤 모습인지에 대해 특별히 아무것도 모르는 엄청난 유능한 인턴에게 지시를 쓰는 것. 남기는 모든 공백은 그들이 최선의 추측으로 채울 공백이다.

5가지 핵심 프롬프트 패턴

이건 트릭이나 꼼수가 아니다. 거의 모든 사용 사례 — 글쓰기, 분석, 코딩, 리서치, 의사결정 지원 — 에 걸쳐 작동하는 구조적 패턴이다. 이 다섯 가지를 익히면 프롬프트를 효과적으로 만드는 것의 80%를 커버한다.

패턴 1: 역할 + 작업 + 맥락

단일로 가장 영향력 있는 패턴이다. AI에게 누가 될지, 무엇을 할지, 무엇을 알아야 하는지를 말하라.

구조:

당신은 [특정 전문성을 가진 역할]입니다. 당신의 작업은 [구체적인 행동 동사 + 결과물]입니다. 맥락: [AI가

패턴 없이: > "우리 신제품 출시에 관한 링크드인 포스트를 써줘."

패턴 있어: > "당신은 기술 창업자들을 위한 글쓰기를 전문으로 하는 B2B SaaS 카피라이터입니다. > 당신의 작업은 무료 체험 가입을 유도하는 제품 출시 발표 링크드인 포스트를 쓰는 것입니다. > 맥락: 우리 제품은 Clearly(clearlyreqs.com)라는 AI 기반 요구사항 도구입니다. 우리 독자는 아이디어에서 스펙까지 걸리는 시간에 좌절하는 프로젝트 매니저와 스타트업 창업자들입니다. 말투는 직접적이고 자신감 있어야 합니다 — 과장 없이, 유행어 없이. 우리 일반적인 고객은 마케팅 주장에 회의적이고 구체성에 반응합니다."

두 번째 프롬프트는 급격히 더 유용한 무언가를 만들어낼 것이다. 더 길기 때문이 아니라 — 세 겹의 추측을 제거하기 때문이다.

패턴 2: 포맷 지정

AI는 프롬프트를 기반으로 합리적으로 보이는 포맷을 기본으로 설정한다. 그 기본값은 종종 당신의 목적에 맞지 않는다. 포맷을 명시적으로 지정하라.

약한: > "이 회의 녹취록을 요약해줘."

강한: > "이 회의 녹취록을 요약해줘. 다음과 같이 형식을 잡아줘: > - **결정된 사항** (불릿 목록, 각 1문장) > - **액션 아이템** (불릿 목록, 언급된 경우 담당자와 기한) > - **미결 질문** (불릿 목록, 후속 조치가 필요한 미해결 항목) > 전체 요약을 200단어 이내로 유지해줘."

포맷 지정이 특히 중요할 때:

- 결과물이 특정 곳(이메일, 슬라이드, 양식)으로 가야 할 때
- 결과물을 빠르게 훑어봐야 할 때
- AI가 간결함이 필요할 때 장황한 경향이 있을 때 (혹은 그 반대)
- AI 결과물을 연결할 때 — 한 프롬프트의 결과가 다른 프롬프트의 입력을 공급할 때

AI 결과물의 구조를 바꾸기 위해 매번 편집하고 있다면, 그건 프롬프트에 포맷 지정을 추가해야 한다는 신호다.

패턴 3: 제약과 안티패턴

AI에게 *하지 말아야 할* 것을 말하라. 이건 가장 많이 사용되지 않는 패턴 중 하나다 — 그리고 레버리지가 가장 높은 것 중 하나다.

받은 AI 결과물이 약간 어긋나고, 약간 일반적이고, 약간 잘못된 것처럼 느껴진 모든 경우는 대개 말하지 않은 제약을 위반한 것이다.

제약 예시: > "불릿 포인트를 사용하지 말고 — 흐르는 산문으로 써줘." > "경쟁사를 이름으로 언급하지 마." > "'레버리지', '시너지', '혁신적', '최첨단' 같은 단어를 사용하지 마." > "독자에게 기술적 배경이 있다고 가정하지 마." > "어떤 문장도 '저는'으로 시작하지 마." > "150단어를 초과하지 마."

안티패턴 예시 (피해야 할 것): > "어떤 회사에도 적용될 수 있는 일반적인 조언을 피해줘 — 모든 것이 우리 상황에 구체적이어야 해." > "답하기 전에 질문을 다시 진술하는 걸 피해줘." > "'고려할 가치가 있을 수도 있습니다' 같은 헤징 언어를 피해줘 — 직접적이고 구체적이어야 해."

제약은 제한처럼 느껴지지만 실제로는 정밀도의 한 형태다. "기술적으로 맞는"과 "실제로 내가 원했던 것" 사이의 간극을 좁힌다.

패턴 4: 사고의 연쇄 (먼저 추론하도록 요청하기)

복잡한 작업 — 분석, 전략, 디버깅, 결정 — 에서 답을 주기 전에 먼저 문제를 추론하도록 AI에게 요청하면 결과물 품질이 극적으로 향상된다.

이것이 작동하는 이유는 그럴듯하게 들리는 답으로 패턴 매칭하는 대신 결론을 향해 쌓아올리도록 모델에게 강요하기 때문이다. "작업을 보여줘"라고 요청하는 것과 같다.

CoT 없이: > "올해 일본 시장으로 확장해야 할까?"

CoT 있어: > "우리 B2B SaaS 제품을 올해 일본 시장으로 확장하는 것을 고려하고 있어. 추천을 주기 전에 다음을 추론해줘: > 1. 일반적으로 일본 진출 미국 SaaS 회사의 성공 또는 실패를 결정하는 요인은 무엇인가? > 2. 자신 있는 추천을 하기 위해 어떤 정보가 필요한가? > 3. 지금 확장하는 것에 대한 가장 강력한 논거는 무엇인가? > 4. 기다리는 것에 대한 가장 강력한 논거는 무엇인가? > 그런 다음 그 추론을 기반으로 추천을 해줘."

두 번째 프롬프트는 일관되게 더 미묘하고 더 신뢰할 수 있는 결과물을 만들어낸다 — 모델이 답을 주기 전에 문제의 복잡성과 씨름해야 하기 때문이다.

사고의 연쇄가 특히 유용한 경우:

- 의미 있는 이해관계가 있는 모든 결정
- 결론만큼 추론이 중요한 분석
- 디버깅 — 수정을 제안하기 전에 문제가 무엇을 일으킬 수 있는지 추론하도록 요청
- 자신 있게 들리는 답이 결국 틀렸을 때마다

패턴 5: 예시 (퓨샷 프롬프팅)

상상하는 것과 AI가 만드는 것 사이의 간극을 좁히는 가장 빠른 방법: 원하는 것의 예시를 보여줘라.

기술 문헌에서 "퓨샷 프롬프팅"이라고 불리지만 개념은 단순하다: 좋은 것이 어떻게 생겼는지 말로 설명하는 대신, 시연하라.

예시 없이: > "우리 회사 목소리로 제품 업데이트 이메일을 써줘."

예시 있어: > "우리 회사 목소리로 제품 업데이트 이메일을 써줘. > > 우리 톤에 맞는 이메일 두 가지 예시가 있어: > > [예시 1 — 실제로 보낸 이메일 붙여넣기] > > [예시 2 — 또 다른 실제 이메일 붙여넣기] > > 주목할 것: 우리는 문장을 짧게 유지하고, '사용자들'이 아닌 '당신'을 쓰고, 기능이 아닌 혜택으로 시작하고, 느낌표를 절대 쓰지 않는다."

예시는 설명 단락보다 더 많은 일을 한다. 리듬, 어휘, 구조, 톤을 보여준다 — 설명하기 매우 어렵지만 시연할 때 즉시 명확한 것들.

최고의 AI 결과물 소형 라이브러리를 유지하라. 다음에 비슷한 것이 필요하면 처음부터 스타일을 설명하는 대신 이전 결과물을 퓨샷 예시로 사용하라.

완성된 프롬프트

다섯 가지 패턴이 모두 적용될 때 잘 구성된 프롬프트가 어떻게 생겼는지:

작업: 잠재적 파트너에게 콜드 아웃리치 이메일 작성.

당신은 한국-미국 기술 파트너십을 전문으로 하는 비즈니스 개발 작가입니다.

당신의 작업은 미국 스마트시티 기술 회사의 이사에게 *Seattle Partners*를 소개하고 한국 스마트시티 스타트

맥락:

- *Seattle Partners*는 한국 기술 기업에 특화된 미국 시장 진출 회사입니다
- 스마트시티, 스마트빌딩, IoT 분야의 20개 이상 한국 스타트업과 함께 했습니다
- 수신자는 중규모 미국 스마트시티 기술 회사의 비즈니스 개발 이사입니다
- 그들은 많은 일반적인 파트너십 이메일을 받을 가능성이 높습니다; 우리 것이 즉시 주목을 받아야 합니다

포맷:

- 제목줄 (8단어 이내)
- 이메일 본문 (150단어 이내)
- 끝에 단일하고 명확한 행동 촉구

제약:

- "시너지"나 "혁신적"이라는 단어를 사용하지 마세요
- 모호하지 말고 - 우리가 다루는 최소 하나의 특정 기술 카테고리를 언급하세요
- "이 이메일이 잘 전달되길 바랍니다"나 그 변형으로 시작하지 마세요
- 톤은 직접적이고 자신감 있어야 합니다, 굴종적이지 않게

이전에 긍정적인 반응을 얻은 아웃리치 이메일 예시입니다: [여기에 예시 붙여넣기]

이 프롬프트는 일관되게 실제로 사용할 수 있는 무언가를 만들어낼 것이다 — 당신이 누구인지, 무엇을 원하는지, 좋은 것이 어떻게 생겼는지에 대한 AI의 추측을 거의 모두 제거하기 때문이다.

반복 마인드셋

캐주얼 사용자와 파워 유저를 가르는 한 가지 더: 불완전한 결과물에 어떻게 반응하는가.

캐주얼 사용자는 한 번 프롬프트를 시도하고, 정확하지 않은 무언가를 받고, 그냥 수용하거나 처음부터 다시 시작한다.

파워 유저는 프롬프팅을 반복적 프로세스로 취급한다. 첫 번째 결과물이 공백을 드러낸다. 프롬프트를 조이고, 제약을 추가하고, 잘못된 것의 예시를 준다. 두 번째 결과물이 더 좋다. 다시 다듬는다. 세 번째나 네 번째 반복에서 진정으로 탁월한 무언가를 가지게 된다 — 그리고 더 중요하게 재사용할 수 있는 프롬프트를 가지게 된다.

목표는 첫 번째 시도에서 완벽한 프롬프트를 쓰는 것이 아니다. 목표는 각 결과물에서 프롬프트가 무엇을 놓쳤는지 배우고 다음 버전에 그 정밀도를 추가하는 것이다.

다듬는 모든 프롬프트는 자산이다. 저장하라. 라이브러리를 구축하라. 그 라이브러리가 워크플로우를 위해 구축할 수 있는 가장 가치 있는 것 중 하나다.

더 깊이 배우고 싶다면

이 글은 핵심 패턴들을 다뤘다. 송재희의 *AI Development Guide*는 특정 사용 사례를 위한 고급 프롬프팅으로 더 깊이 들어간다 — 에이전트를 위한 프롬프팅 방법, 다단계 워크플로우에서 프롬프트를 연결하는 방법, 다양한 모델과 맥락에 이 패턴들을 적용하는 방법 포함.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

다음 편: "AI 에이전트 — 코딩 없이 나만의 미니 팀 만들기" — 단일 프롬프트를 넘어 당신이 자는 동안 작동하는 다단계 자율 워크플로우로 가는 방법.

AI 에이전트

12 min read · 구축하기

"AI로 만들기" 시리즈 7편

> **핵심 요약:** AI 에이전트는 스스로 행동하는 시스템이다. **트리거**(챗 창을 기다리지 않고 어떤 일이 일어나면 작동)를 가지고, **도구**(웹, 파일, 데이터베이스, API, 코드)를 사용하며, 목표를 향해 계획-실행-판단을 반복한다. 에이전트에는 세 가지 자율성 수준이 있고, 첫 번째 에이전트는 n8n으로 만들 수 있다.

이 시리즈에서 지금까지 프롬프팅에 대해 이야기했다 — AI에게 작업을 주고 결과를 받는 것. 프롬프트 하나, 응답 하나. 묻고 답하는 것.

유용하다. 하지만 한계도 있다. 실제 업무의 대부분은 단일 교환으로 일어나지 않기 때문이다. 시퀀스로 일어난다. 단계 A가 단계 B를 트리거한다. 단계 B의 결과가 단계 C로 들어간다. 외부의 무언가 — 이메일, 폼 제출, 캘린더 이벤트 — 가 전체를 시작한다. 그리고 마지막으로 결과가 어딘가로 간다: 받은편지함, 스프레드시트, Slack 채널.

바로 여기서 AI 에이전트가 등장한다.

에이전트는 질문에 답하는 AI가 아니다. **행동하는 AI**다 — 일련의 단계를 계획하고, 도구를 사용하고, 결정을 내리고, 최소한의 인간 개입으로 목표를 완수할 수 있는. 지시를 기다리는 스마트 어시스턴트와 목표를 받고 단계를 스스로 파악하도록 신뢰할 수 있는 어시스턴트의 차이이다.

그리고 2026년의 놀라운 것: 코드 한 줄 없이 에이전트를 만들 수 있다.

무엇이 "에이전트"를 만드는가

"에이전트"라는 단어가 많이 사용된다. 정밀하게 살펴보자.

기본 AI 상호작용은 이렇게 생겼다: **당신 → 프롬프트 → AI → 응답 → 당신**

에이전트 상호작용은 이렇게 생겼다: **트리거 → 에이전트 → [계획 → 도구 → 결정 → 도구 → 결정...] → 출력 → 목적지**

핵심 차이점:

에이전트는 트리거를 가진다. 채팅 창을 열 때까지 기다리지 않는다. 무언가가 일어날 때 활성화된다 — 새 이메일이 도착하거나, 폼이 제출되거나, 예약된 시간이 되거나, 스프레드시트에 새 행이 나타날 때.

에이전트는 도구를 사용한다. 웹을 검색하고, 파일을 읽고, 데이터베이스에 쓰고, 이메일을 보내고, API 를 호출하고, 코드를 실행할 수 있다. 컨텍스트 윈도우를 넘어 실제 세계로 뻗을 수 있다.

에이전트는 결정을 내린다. 분기점에서 — 만약 이것이면, 그러면 저것 — 상황을 평가하고 경로를 선택 한다. 텍스트만 생성하는 게 아니라 조건부 선택을 한다.

에이전트는 작업이 아닌 목표를 가진다. 작업은 "이 문서를 요약해줘"다. 목표는 "경쟁사 가격 페이지를 모니터링하고 변경될 때마다 무엇이 변했고 왜 중요할 수 있는지 요약과 함께 나에게 알려줘"다.

에이전트의 세 가지 레벨

모든 에이전트가 같지 않다. 세 가지 레벨로 생각하면 도움이 된다:

레벨 1 — 트리거된 자동화 무언가가 일어날 때 자동으로 실행되고, 고정된 일련의 단계를 실행하고, 결과를 전달하는 워크플로우. 진짜 의사결정 없음 — 정의된 프로세스의 신뢰할 수 있는 자동 실행만.

예시: 새로운 잠재 고객이 연락 폼을 작성할 때마다 LinkedIn에서 자동으로 찾아보고, 그들이 누구인지 개인화된 한 단락 요약을 생성하고, 팀이 리드를 보기 전에 CRM 노트에 추가한다.

도구: n8n 또는 Make.com, 생성 단계에 AI 포함.

레벨 2 — 의사결정 에이전트 AI가 무언가를 평가하고 발견한 것에 따라 다른 결과로 라우팅하는 조건부 논리가 포함된 워크플로우.

예시: 고객 지원 이메일이 도착하면 에이전트가 읽고, 분류하고(청구 문제 / 기술 문제 / 기능 요청 / 불만), 고객이 프리미엄 플랜에 있는지 확인하고, 그에 따라 라우팅한다 — 다양한 조합에 대해 다른 응답 템플릿으로.

도구: 여러 분기가 있는 n8n, 결정 지점에서 AI 분류.

레벨 3 — 목표 지향 에이전트 스크립트가 아닌 목표가 주어지는 에이전트 — 그리고 그것을 달성하기 위한 자체 단계를 파악한다. 계획하고, 순서대로 도구를 사용하고, 중간 결과를 평가하고, 접근 방식을 조정할 수 있다.

예시: "에너지 관리 분야 한국 스마트시티 스타트업 상위 5개를 조사하고, 제품 설명, 최근 펀딩, 핵심 차별점을 컴파일하고, 포맷된 비교 보고서를 만들어줘."

에이전트가 검색하고, 읽고, 추출하고, 구성하고, 작성한다 — 어떤 소스를 사용할지, 충분한 정보가 있을 때, 결과물을 어떻게 구성할지 결정하면서.

도구: 도구 접근이 있는 Claude Opus 4.6 또는 GPT-5.2, 또는 더 복잡한 오케스트레이션을 위한 Relevance AI나 Claude Code 같은 플랫폼.

첫 번째 에이전트 만들기: 실제 워크스루

실제 무언가를 만들어보자. 가장 강력한 무료/오픈소스 자동화 도구인 n8n을 사용해 레벨 1 에이전트를 만들겠다.

목표: 경쟁사 웹사이트 목록을 모니터링한다. 매주 월요일 아침, 가격 페이지가 업데이트됐는지 확인하고, 업데이트됐다면 변경된 내용 요약과 함께 Slack 메시지를 보낸다.

1단계: 트리거 n8n에서 Schedule 노드를 만든다. 매주 월요일 오전 8시에 실행하도록 설정한다. 이것이 트리거다 — 에이전트가 아무것도 하지 않아도 자동으로 깨어난다.

2단계: 데이터 소스 모니터링하려는 각 경쟁사 URL에 대해 HTTP Request 노드를 추가한다. 이것이 가격 페이지의 현재 콘텐츠를 가져온다.

3단계: 비교 오늘 콘텐츠를 지난 주 저장된 버전과 비교하는 Function 노드를 추가한다. 일치하면 — 중지. 일치하지 않으면 — 다음 단계로 계속.

4단계: AI 분석 AI 노드를 추가한다(n8n은 네이티브 Claude와 OpenAI 통합을 가지고 있다). 이 프롬프트와 함께 이전 및 새 콘텐츠를 전달한다:

> "이 경쟁사 가격 페이지의 두 버전을 비교하라. 다음을 식별하라: (1) 무엇이 변했는지, (2) 가격이 올랐는지 내렸는지, (3) 추가되거나 제거된 새 플랜이나 기능, (4) 그들이 이 변경을 했을 수 있는 이유에 대한 평가. 구체적이고 간결하게."

5단계: 출력 Slack 노드를 추가한다. 경쟁사 이름, 날짜, 요약으로 포맷된 AI 분석을 #competitive-intel 채널에 보낸다.

6단계: 새 버전 저장 다음 주와 비교하기 위해 오늘 콘텐츠를 저장한다.

총 설정 시간: 45-60분. 코드 없음. 그 후 매주 월요일 자동으로 실행된다 — 실제로 무언가 변할 때만 보게 된다.

이게 에이전트다. 깨어나고, 확인하고, 행동할지 결정하고, 지능적으로 행동하고, 필요한 곳에 결과를 전달한다.

지금 일어나고 있는 것: 에이전트의 최전선

위의 기본 에이전트 프레임워크는 오늘 당장 만들 수 있는 것들이다. 하지만 최전선은 빠르게 움직이고 있다 — 2026년에 등장하고 있는 것들은 아직 그 수준에서 만들 준비가 안 됐더라도 이해할 가치가 있다.

에이전트 스웸: 병렬로 작동하는 여러 에이전트

단일 에이전트는 하나의 작업 흐름을 처리한다. **에이전트 스웸**은 병렬로 작동하는 여러 전문화된 에이전트의 조율된 네트워크다 — 각각 다른 하위 작업을 처리하고 결과물을 결합한다.

프로젝트 팀처럼 생각하라. 한 명의 제너럴리스트 에이전트가 모든 것을 순차적으로 하는 대신:

- **리서치 에이전트** — 소스를 검색하고 읽는다
- **작성 에이전트** — 리서치를 기반으로 초안을 작성한다
- **비평 에이전트** — 검토하고 약점을 표시한다
- **편집 에이전트** — 최종 결과물을 다듬는다

각각 동시에 실행된다. 오케스트레이터가 결과를 결합한다. 전체 작업이 단일 순차적 에이전트보다 몇 배 빠르게 완료된다.

OpenClaw (에이전트 상호운용성을 위한 오픈 프로토콜), **LangGraph**, **CrewAI**가 스웸을 조율하는데 사용하는 도구들이다. CrewAI는 특히 비개발자에게 인기를 얻고 있다 — 각 에이전트의 역할, 도구, 목표를 일상 언어로 정의하면 프레임워크가 조율을 처리한다. Claude Code도 네이티브로 멀티 에이전트 설정을 지원한다.

실제 스웸 사례:

- **투자자 리서치:** 에이전트 1이 재무 데이터를 가져오고, 에이전트 2가 뉴스를 추적하고, 에이전트 3이 경쟁사 공시를 분석 — 모두 병렬로, 단일 브리핑으로 결합
- **콘텐츠 파이프라인:** 에이전트 1이 주제를 리서치하고, 2가 작성하고, 3이 사실 확인하고, 4가 SEO 최적화 — 전체 파이프라인이 자율적으로 실행
- **대규모 고객 지원:** 청구, 기술 문제, 계정 관리를 전문화된 에이전트들이 동시에 처리하고 라우팅 에이전트가 트래픽을 조율

AI 직원: 완전한 도구 접근

지금 가장 많은 대화를 만들어내고 있는 개념이다 — 그리고 가장 정당한 흥분을 자아내는. **AI 직원**은 단순한 데이터 접근이 아닌 실제 운영 능력을 부여받은 에이전트다:

- **결제 권한** — Stripe Agent Toolkit 같은 서비스를 통해 정의된 한도 내에서 승인된 거래를 실행할 수 있는 에이전트 (도구 구매, API 호출 결제, 구독 갱신)
- **전화와 음성** — 실제 전화를 걸고 받고, 음성 메일을 남기고, 전화 기반 시스템과 상호작용할 수 있는 전화번호를 가진 에이전트. **Bland AI**와 **Vapi** 같은 도구들이 실제 전화 통화를 처리하는 에이전트를 배포할 수 있게 한다 — 대부분의 사람에게 인간 상담원과 구별 불가능
- **이메일 아이덴티티** — 정의된 가드레일 내에서 당신을 대신해 서신을 수행할 수 있는 자체 이메일 주소를 가진 에이전트
- **브라우저와 컴퓨터 사용** — 웹사이트를 탐색하고, 양식을 작성하고, 버튼을 클릭하고, 소프트웨어를 조작할 수 있는 에이전트 — API만 호출하는 게 아니라 인간이 사용하는 것과 같은 인터페이스 실제 사용. **Anthropic**의 **Computer Use (Cowork)**, **OpenAI**의 **Operator**, **Google**의 **Project Mariner**가 모두 이것을 개척 중

- **코드 실행과 배포** — 코드를 작성하고, 테스트를 실행하고, 프로덕션 저장소에 변경 사항을 푸시할 수 있는 에이전트. Cognition의 **Devin** (AI 소프트웨어 엔지니어)이 이 범주의 대표 사례

Artisan, Relevance AI, 11x 같은 회사들은 이미 "AI SDR"(영업 개발 담당자)을 판매하고 있다 — 잠재 고객을 발굴하고, 리서치하고, 개인화된 아웃리치를 작성하고, 팔로업하고, 이의를 처리하고, 미팅을 예약하는 에이전트. 24/7 완전 자동화.

실제 함의: 공급업체 온보딩 프로세스를 처리하는 에이전트는 이제 이메일을 초안 작성할 뿐만 아니라 보내고, 팔로업하고, 청구서를 처리하고, 회계 시스템을 업데이트할 수 있다 — 사람이 모든 단계가 아닌 예외만 검토하면서 처음부터 끝까지.

중요한 가드레일: AI 직원 역량의 모든 진지한 구현에는 지출 한도, 액션 로그, 고위험 액션에 대한 승인 워크플로우, 즉각적인 취소 메커니즘이 포함된다. 목표는 감독 없는 자율성이 아니다 — 인간이 경계를 설정하고 예외를 검토하는 대규모 감독 자율성이다.

MCP: 에이전트를 연결하는 결합 조직

위의 모든 것 — 기본 레벨 1 자동화부터 완전한 AI 직원 설정까지 — 의 기반에는 2편에서 처음 논의한 MCP(Model Context Protocol)가 있다.

MCP는 에이전트가 불안정한 맞춤 통합이 아닌 표준화된 인터페이스를 통해 외부 서비스에 연결할 수 있게 한다. 에이전트가 Google Drive, Gmail, Slack, GitHub, CRM, 회계 도구에 네이티브로 접근할 수 있다 — 어떤 에이전트 프레임워크도 사용할 수 있는 성장하는 표준화된 커넥터 라이브러리를 통해.

에이전트 스위치가 실용화되고 있는 이유가 이것이다: 모든 에이전트가 같은 연결 프로토콜을 사용하면 다양한 서비스에 걸쳐 에이전트를 조율하는 것이 극적으로 단순해진다. 그리고 AI 직원 개념이 가속화되는 이유다 — 에이전트가 MCP를 통해 결제 시스템, 전화 API, 브라우저 인터페이스에 연결할 수 있으면 운영 능력이 빠르게 확장된다.

에이전트 도구나 플랫폼을 평가할 때 MCP를 지원하는지 물어봐라. 지원하는 것들은 커넥터 라이브러리가 성장함에 따라 시간이 지나면서 능력이 복리로 쌓인다.

메모리와 커뮤니케이션: 에이전트가 시간을 넘어 생각하는 방법

에이전트를 이해하고 나면 자연스럽게 생기는 질문이 있다. 에이전트는 뭔가를 기억하는가? 여러 에이전트가 함께 작동할 때 어떻게 서로 소통하는가?

맞는 질문들이다 — 그리고 답은 대부분의 입문 자료가 다루는 것보다 훨씬 미묘하다.

에이전트 메모리의 네 가지 유형

인컨텍스트 메모리 (단기) 에이전트의 활성 작업 메모리다 — 현재 컨텍스트 윈도우에 있는 모든 것. 현재 세션에서 보고, 하고, 말한 것들. 빠르고, 즉시 접근 가능하고, 세션이 끝나면 완전히 사라진다. 컴퓨터의 RAM처럼 생각하라. 실행 중에는 강력하다. 끄면 지워진다.

100만 토큰 컨텍스트 윈도우도 긴 다단계 워크플로우 동안 채워진다. 채워지면 오래된 내용이 삭제되거나 요약된다. 이것을 잘 관리하지 않는 에이전트는 이전 단계를 "잊기" 시작한다.

외부 메모리 (장기) 에이전트는 외부 저장소 — 데이터베이스, 파일, 스프레드시트, 또는 특화된 메모리 저장소 — 에서 읽고 쓸 수 있다. 세션 간에 유지되는 영속적 메모리다. 에이전트가 실행 간에 무언가를 기억해야 할 때 — 고객의 이력, 이전 결정, 지난 주에 배운 사실 — 외부 저장소에 쓰고 다음에 검색한다.

Mem0와 **Zep** 같은 도구들이 이를 위해 특별히 만들어졌다. 메모리를 검색 가능한 임베딩으로 저장해 에이전트가 모든 것을 로드하는 대신 *관련* 맥락을 검색할 수 있다. 이것이 "개인 어시스턴트" 에이전트가 시간이 지나면서 실제로 당신을 아는 것처럼 느끼게 만드는 것이다.

절차적 메모리 (내장된 기술) 이것은 모델의 훈련이다 — 본질적으로 할 수 있는 것. 코드 쓰기, 텍스트 요약, 데이터 분석. 런타임에 추가할 수 없다; 모델 버전별로 고정되어 있다. 하지만 에이전트가 행동할 수 있는 것을 확장하는 도구와 지침을 줌으로써 확장할 수 있다.

에피소딕 메모리 (행동 이력) 에이전트가 한 것의 로그 — 과거 도구 호출, 과거 결과물, 과거 결정들. 외부에 저장되고 새 세션 시작 시 검색되어 에이전트가 "지난 번에 무슨 일이 있었는지" 이해할 수 있도록. 중복 행동과 무한 루프를 피하는 데 핵심이다.

에이전트들이 서로 소통하는 방법

스웸이나 멀티 에이전트 시스템에서 에이전트들은 조율이 필요하다. 세 가지 주요 패턴:

공유 메모리 (블랙보드 패턴) 모든 에이전트가 공유 데이터 저장소에서 읽고 쓴다. 에이전트 1이 조사 결과를 블랙보드에 쓴다. 에이전트 2가 그것을 읽고 초안을 쓴다. 에이전트 3이 초안을 읽고 편집한다. 에이전트 간 직접 메시징 없음 — 공유 상태를 통해 조율한다. 단순하고, 신뢰할 수 있고, 디버그하기 쉽다. 대부분의 프로덕션 시스템에서 선호되는 패턴이다.

직접 메시징 (메시지 패싱) 에이전트들이 구조화된 메시지를 서로에게 직접 보낸다 — 팀원 간의 Slack DM처럼. 에이전트 1이 그것으로 무엇을 할지 명시적 지침과 함께 결과물을 에이전트 2에게 보낸다.

CrewAI와 **AutoGen** 같은 프레임워크가 이 모델을 사용한다. 에이전트들은 정의된 역할을 가지고 메시지 큐를 통해 작업을 넘긴다.

오케스트레이터 패턴 중앙 "관리자" 에이전트가 워크플로우를 제어한다. 워커 에이전트들에게 작업을 할당하고, 그들의 결과물을 수집하고, 품질을 평가하고, 다음 단계를 결정한다 — 기대에 미치지 못하면 수정을 위해 다시 보내는 것 포함. 이것은 훌륭한 인간 관리자가 팀을 운영하는 방식을 반영한다. 오케스트레이터는 전체 그림을 가진다; 워커들은 전문화된 하위 작업을 처리한다. 대부분의 엔터프라이즈급 에이전트 시스템이 이것의 어떤 형태를 사용한다.

실제로 문제가 되는 메모리 이슈

대부분의 사람들이 뭔가를 만들어보기 전까지 깨닫지 못하는 것: **에이전트는 자신이 이미 한 것을 자동으로 알지 못한다.**

다단계 워크플로우에서, 에이전트가 진행 상황을 명시적으로 기록하지 않으면 중간에 실패하면 처음부터 다시 시작해야 한다. 다음 주에 다시 실행되면 이미 처리한 것을 전혀 모른다 — 다시 처리한다.

이것은 에이전트에 고유한 버그 카테고리를 만든다:

- **중복 행동** — 에이전트가 처음 보낸 것을 기록하지 않았기 때문에 같은 이메일을 두 번 보낸다
- **잃어버린 맥락** — 에이전트가 워크플로우 앞에서의 관련 이전 결정을 모르고 결정을 내린다
- **무한 루프** — 에이전트가 실패했다는 기록 없이 실패한 것을 계속 재시도한다

해결책은 명시적 상태 관리다 — 모든 의미 있는 단계에서 외부 저장소에 체크포인트를 쓰는 것. 에이전트가 항상 답할 수 있도록: *무엇을 했는가, 아직 무엇을 해야 하는가, 무엇이 실패했는가?* 모든 진지한 에이전트 프레임워크에는 이를 위한 패턴이 있다. 화려하지 않다. 하지만 신뢰할 수 있는 프로덕션 에이전트와 실제 세계에서 망가지는 데모를 가르는 것이 바로 이것이다.

흔한 실수들

망가진 프로세스를 자동화하기 에이전트는 나쁜 프로세스를 열 배 속도로 충실히 실행한다. 자동화하기 전에 기본 프로세스가 실제로 맞는지 확인해라. 자동화는 증폭시킨다 — 좋은 것도 나쁜 것도.

중요한 출력에 인간 체크포인트 없음 고객, 파트너, 또는 외부 세계에 닿는 모든 것 — 항상 발송 전에 인간 검토 단계를 구축해라. 초안을 작성하는 에이전트는 강력하다. 검토 없이 발송하는 에이전트는 책임이다.

첫 번째 버전 과도하게 엔지니어링 레벨 1로 시작해라. 매주 두 시간을 절약하는 단순한 트리거된 자동화가 만드는 데 몇 주 걸리고 자주 망가지는 복잡한 레벨 3 에이전트보다 더 가치 있다. 단순한 버전이 작동하게 하고, 그것에서 배우고, 그 다음 복잡성을 추가해라.

실패 계획 없음 외부 서비스는 다운된다. API는 변한다. 페이지가 404를 낸다. 모든 에이전트에 오류 처리를 구축해라 — 단계가 실패하면 무슨 일이 일어나야 하는가? 최소한, 무언가 망가졌다는 것을 알 수 있도록 알림을 자신에게 보내라.

첫 번째 에이전트 도전

야심찬 것으로 시작하지 마라. 귀찮은 것으로 시작해라.

매번 같은 단계를 따르는 반복적으로 하는 무언가를 생각해보라. 15-30분이 걸리고, 최소 매주 일어나고, 사람이 필요하지 않아야 할 것 같은 것. 컴파일하는 보고서. 보내는 업데이트. 실행하는 점검. 작성하는 요약.

그게 첫 번째 에이전트 후보다.

먼저 종이에 단계를 매핑해라. 무엇이 트리거하는가? 순서대로 단계들은 무엇인가? 현재 인간 결정이 어디에서 일어나는가 — 그리고 규칙이 그 결정을 대체할 수 있는가? 결과물은 무엇이고 어디로 가는가?

그 지도가 생기면 n8n이나 Make.com을 열고 만들어라. 레벨 1로 시작해라. 실행되게 해라. 그 다음 한 달 동안 혼자 실행되는 것을 지켜봐라 — 그리고 추가하고 싶은 것을 주목해라.

더 깊이 배우고 싶다면

이 글은 에이전트의 기초를 다뤘다. 송재희의 *AI Development Guide*는 더 깊이 들어간다 — 멀티 에이전트 아키텍처, 특정 비즈니스 도메인을 위한 에이전트 워크플로우 설계 방법, 에이전트를 이전 포스트에서 다룬 프롬프팅 및 데이터 기반과 결합하는 방법 포함.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

다음 편: "바이프 코딩 — 일상 언어로 실제 앱 만들기" — 아이디어에서 코드 한 줄 없이 배포된 작동하는 애플리케이션으로 가는 방법.

바이브 코딩

9 min read · 구축하기

"AI로 만들기" 시리즈 8편

> **핵심 요약:** 바이브 코딩은 AI — 코드 어시스턴트와 생성형 UI 도구 — 를 사용해 일상 언어 설명만으로 실제 소프트웨어를 만드는 것이다. 당신이 설명하면 AI가 구현하고, 당신은 검토하고 반복한다. 무작정 하는 것이 아니라 원하는 것을 정확히 말하고, 체계적으로 테스트하며, 반복 가능한 워크플로우로 꾸준히 다듬을 때 가장 잘 작동한다.

2025년, "바이브 코딩"이 콜린스 사전의 올해의 단어가 됐다.

기술 용어로서는 이례적인 일이지만, 실제로 일어난 무언가를 포착했다. 원하는 것을 일상 언어로 설명하면 AI가 그것을 만든다는 아이디어가 약 2년 만에 공상과학에서 수백만 명의 일상 실천으로 이동했다.

하지만 과대포장이 말해주지 않는 것이 있다. 바이브 코딩을 시도하는 대부분의 사람들이 답답한 결과를 얻는다. 도구가 작동하지 않아서가 아니라, 챗봇에게 프롬프트하는 것과 같은 방식으로 접근하기 때문이다. 원하는 것을 느슨하게 설명하고, 거의 맞아 보이는 무언가를 받고, 끝없이 패치하다가, 결코 누군가에게 보여줄 수 있는 수준에 이르지 못한다.

이 글은 실제로 어떻게 하는지에 대한 것이다. 마인드셋, 워크플로우, 그리고 코드를 직접 작성하지 않고 아이디어에서 작동하고 배포 가능한 앱으로 가는 실용적 단계들.

바이브 코딩이 실제로 무엇인가

바이브 코딩은 AI를 사용해 — 구체적으로는 AI 코딩 어시스턴트와 생성형 UI 도구를 — 자연어 설명을 기반으로 코드를 대신 작성하게 하는 것이다. 당신이 설명한다. AI가 구현한다. 당신이 검토하고, 테스트하고, 반복한다.

이름은 그 느낌에서 왔다: *무엇을* 만들고 *왜* 만드는지를 생각하는 플로우 상태에 들어가고, *어떻게* 코드가 작동하는지는 생각하지 않아도 된다. 구현 레이어가 거의 사라진다.

하지만 "바이브"는 약간 오해를 불러일으킨다. 최고의 바이브 코더들은 그냥 눈치껏 하는 게 아니다. 원하는 것에 대해 정밀하고, 테스트에 체계적이고, 반복에 규율이 있다. 바이브는 경험에 있다. 접근 방식에 있는 게 아니다.

이 용어는 OpenAI 공동창업자이자 전 테슬라 AI 디렉터인 안드레이 카르파티가 만들었다. 그의 설명은 정밀했다: 원하는 것을 설명하면 AI가 코드를 생성하고, 실행하고, 무엇이 잘못됐는지 또는 다음에 무엇을 원하는지 설명하고, 반복한다. 코드를 쓰는 게 아니다. 지시하는 것이다.

이것을 가능하게 만든 것

세 가지가 합쳐져 바이브 코딩을 진짜로 실용적이게 만들었다:

모델이 코드에서 충분히 좋아졌다. 오늘날의 최전선 모델 — Claude Opus 4.6, GPT-5.2 — 은 수십 개의 언어로 프로덕션 품질의 코드를 작성하고, 자신의 실수를 디버깅하고, 아키텍처에 대해 추론한다. 자동완성만 하는 게 아니다. 아키텍처를 잡는다.

컨텍스트 윈도우가 충분히 커졌다. 전체 코드베이스가 이제 단일 컨텍스트 윈도우에 들어갈 수 있다. AI가 모든 것을 한꺼번에 볼 수 있다. 컴포넌트, 로직, 스타일을 전부 보면서 전체 시스템에 걸쳐 일관된 변경을 만든다.

목적에 맞는 도구들이 등장했다. Cursor, Bolt, Lovable, Replit, v0는 AI가 덧붙여진 텍스트 편집기가 아니다. 바이브 코딩 워크플로우를 위해 설계된 환경이다. 설명이 기본 입력이고 코드가 완전히 이해할 필요 없는 출력인.

이 조합은 "아이디어가 있다"와 "작동하는 것이 여기 있다" 사이의 간극을 몇 달에서 몇 시간으로 압축했다.

2026년의 도구들

환경이 상당히 성숙했다. 실제로 사용되고 있는 것들:

풀스택 웹 앱용 (설정 불필요):

- **Bolt.new** — 앱을 설명하면 풀스택 애플리케이션을 받는다. 빠른 프로토타이핑에 강하다. 가장 초보자 친화적인 옵션 중 하나.
- **Lovable** — 세련된 UI 집중으로 Bolt와 비슷하다. 시각적 품질이 중요한 소비자용 제품에 특히 좋다.
- **Vercel v0** — Vercel 배포 생태계와 긴밀하게 통합된 AI 기반 컴포넌트 생성. Next.js 세계에 있다면 최선의 선택.

코드 레벨 빌딩 (더 많은 제어):

- **Cursor** — AI가 깊이 통합된 VS Code 포크. 프롬프트를 쓰면 Cursor가 인라인으로 코드를 작성, 편집, 리팩터링한다. 개발자의 필수 도구지만 더 많은 제어를 원하는 비개발자들도 점점 더 사용하고 있다.

- **Claude Code** — Anthropic의 에이전틱 코딩 도구. 전체 코드베이스를 처리하고, 멀티파일 변경을 계획하고, 실행한다. 복잡한 프로젝트에 특히 강력하다.
- **Windsurf** — 에이전트 모드 코딩에서 다른 강점을 가진 강력한 Cursor 대안.

특정 사용 사례용:

- **Replit** — 브라우저 기반, 로컬 설정 없음. 학습과 빠른 실험에 좋다.
- **Glide / AppSheet** — 스프레드시트나 데이터베이스 위에 직접 구축된 데이터 중심 앱과 내부 도구.

올바른 선택: 기술적 설정 없이 빠르게 뭔가를 보고 싶다면 Bolt 또는 Lovable. 더 많은 제어를 원하고 조금 배울 의향이 있다면 Cursor. 복잡한 멀티파일 프로젝트라면 Claude Code.

바이브 코딩 워크플로우

대부분의 가이드가 부족한 부분이 여기도. 도구는 보여주지만 프로세스는 보여주지 않는다. 실제 결과를 만드는 워크플로우:

1단계: 만들기 전에 정의하라

5편으로 돌아가라. 어떤 도구도 열기 전에 한 단락 문제 정의를 써라. 알아야 할 것:

- 앱이 하는 것 (구체적으로)
- 누가 사용하는지 (구체적으로)
- 성공적인 첫 번째 버전이 어떻게 보이는지 (구체적으로)
- 아직 할 필요가 없는 것

MVP 함정: 대부분의 사람들이 첫 번째 시도에서 전체 비전을 만들려고 한다. 하지 마라. 진정으로 유용할 가장 작은 버전을 정의해라. 다른 모든 것은 버전 2다.

2단계: 먼저 구조를 잡아라

AI에게 전체 앱을 만들어달라고 요청하는 것으로 시작하지 마라. 먼저 계획을 요청해라.

> "나는 [한 단락 설명]을 만들고 싶어. 코드를 작성하기 전에 알려줘: > 1. 이 앱에 필요한 핵심 화면이나 페이지 > 2. 저장하고 표시해야 하는 주요 데이터 > 3. 첫 오픈부터 주요 액션 완료까지의 사용자 흐름 > 4. 구축 전에 생각해야 할 잠재적 복잡성"

이것을 검토하라. 틀린 것을 반박하라. 빠진 것을 추가하라. 여기서 5분이 나중에 수시간의 재구축을 방지한다.

3단계: 레이어로 구축하라

점진적으로 구축하라. 한 번에 하나의 기능, 다음으로 넘어가기 전에 테스트.

- 레이어 1: 뼈대 — 네비게이션, 기본 화면, 플레이스홀더 콘텐츠. 아직 로직 없음.
- 레이어 2: 데이터 — 무엇이 저장되는지, 어떻게 구성되는지, 실제 데이터의 기본 표시.
- 레이어 3: 핵심 액션 — 앱이 하는 가장 중요한 단 하나의 것. 끝까지 작동하는.
- 레이어 4: 부가 기능 — 다른 모든 것, 한 번에 하나씩 추가.

다음 레이어를 시작하기 전에 각 레이어를 확인한다. 이것이 작동하는 앱과 예상치 못한 것을 클릭하는 순간 망가지는 인상적인 데모를 구분하는 것이다.

4단계: 디버깅 마인드셋

것들은 망가진다. 이건 실패가 아니다. 소프트웨어의 본질이다.

무언가 망가지면 무작위로 패치하지 마라. 대신:

정밀하게 설명하라: > "제출 버튼을 클릭하면 아무 일도 일어나지 않아. 항목을 저장하고 성공 메시지를 보여주길 기대했어. 버튼이 UI에 나타나고 클릭 가능하지만 아무 효과가 없어."

정확한 오류를 보여줘라. 브라우저 콘솔에서 그대로 복사하라. 의역하지 마라.

수정하기 전에 설명을 요청하라: > "수정하기 전에, 무엇이 원인이라고 생각하는지, 왜 그런지 설명해 줘."

AI가 잘못 진단하면 잘못된 수정이 상황을 악화시키기 전에 오진을 잡아낸다. 그리고 코드가 어떻게 작동하는지에 대해 배운다.

5단계: 빌더가 아닌 사용자처럼 테스트하라

빌더는 해피 패스를 테스트한다. 사용자는 엣지 케이스를 찾는다.

각 기능 후:

- 필수 필드를 비워두면 어떻게 되는가?
- 같은 버튼을 빠르게 두 번 클릭하면 어떻게 되는가?
- 모바일 화면에서는 어떻게 보이는가?
- 아직 데이터가 없으면 어떻게 되는가?

이건 모호한 엣지 케이스가 아니다. 실제 사용자들이 처음 5분 안에 하는 것들이다.

실제 워크스루: 리드 트래커 만들기

컨설팅 회사를 위한 간단한 리드 추적 도구를 어떻게 만드는지. 아무것도 없는 상태에서 오후 안에 배포까지.

브리핑: 팀이 새 리드를 기록하고, 단계(신규 / 연락됨 / 미팅 예약 / 제안서 발송 / 완료)를 기록하고, 메모를 추가하고, 단계별로 정렬된 모든 리드를 테이블에서 볼 수 있는 웹 앱.

1단계: Bolt.new 열기

2단계: 구조 프롬프트 > "간단한 리드 트래커 웹 앱을 만들고 싶어. 코딩 전에 알려줘: 어떤 화면이 필요하고, 어떤 데이터를 저장해야 하고, 가장 단순한 첫 번째 버전은 뭔가?"

응답을 검토하라. 확인하고 진행하라.

3단계: 뼈대 프롬프트 > "뼈대를 만들어줘: 리드를 보여주는 테이블이 있는 메인 페이지, 리드 추가 버튼, 회사명, 연락처 이름, 이메일, 전화번호, 상태(드롭다운: 신규 / 연락됨 / 미팅 예약 / 제안서 발송 / 성사 / 실패), 메모 필드가 있는 간단한 폼. 지금은 플레이스홀더 데이터 사용. 깔끔하고 전문적으로 만들어줘."

구조적으로 맞아 보이는가? 좋다. 계속 진행하라.

4단계: 데이터 연결 > "리드 추가 폼이 실제로 작동하게 만들어줘. 제출하면 새 리드가 테이블에 나타나야 해. 지금은 앱 상태에 저장해줘."

테스트. 리드를 추가해라. 나타나는가? 예 → 계속. 아니오 → 먼저 디버그.

5단계: 필터링 추가 > "상태로 리드 테이블을 필터링하는 기능을 추가해줘. 상태를 클릭하면 그 상태의 리드만 보여. '전체 보기' 옵션도 추가해줘."

필터가 올바르게 작동하는지 테스트하라.

6단계: 배포 Bolt에는 내장 배포 버튼이 있다. 클릭하라. 즉시 라이브 URL. 팀과 바로 공유 가능.

총 시간: 2~3시간. 당신이 쓴 코드: 없음. 팀이 오늘 사용할 수 있는 작동하는 배포된 앱.

솔직한 한계

바이브 코딩은 강력하다. 하지만 이 벽들을 만날 것이고 미리 알아야 한다:

단순 CRUD 앱에서는 탁월하지만, 복잡한 조건부 로직이나 다단계 계산이 있는 앱은 만들기도 어렵고 무언가 잘못됐을 때 유지하기 훨씬 어렵다.

내부 사용이나 프로토타입이라면 보안 문제는 크지 않다. 하지만 민감한 사용자 데이터나 결제를 처리한다면 개발자가 관여하거나 관리형 보안 플랫폼(Supabase Auth, Clerk, Firebase)이 필요하다.

프로토타입에는 인메모리 상태로 충분하다. 세션 간 실제 영속성이나 사용자 계정이 필요하면 제대로 된 데이터베이스를 연결하는 것이 의미 있는 복잡성 단계 상승이다.

마지막으로, 바이브 코딩된 앱은 기술 부채가 쌓인다. 많은 반복 후에 코드가 일관성이 없어지고 확장하기 어려울 수 있다. 앱을 집중적으로 유지해라. 상당한 성장이 필요하면 깔끔한 기반으로 재작성을 고려해라.

이것이 실제로 무엇을 여는가

바이브 코딩이 나타내는 더 깊은 전환은 개발자를 대체하는 것에 관한 게 아니다. 누가 만들 수 있는지에 관한 것이다.

전: 소프트웨어 솔루션이 필요한 문제가 있으면, 개발자를 고용하거나, 정확히 맞지 않는 기성 도구를 사용하거나, 기다렸다.

후: 정확한 상황에 맞는 도구를 직접 만들 수 있다. 팀이 실제로 필요로 하는 내부 도구. 투자자에게 만들고 있는 것을 보여주는 프로토타입. 몇 달째 요청해온 고객용 기능. 오후 안에.

이것을 가장 많이 활용하는 사람들은 도메인 전문가들이다. 문제를 깊이 이해하지만 이전에는 그 이해를 소프트웨어로 변환할 수 없었던 사람들. 고객이 정확히 무엇을 필요로 하는지 아는 컨설턴트. 매일 비효율성을 보는 운영 매니저. 비전은 있지만 아직 기술 공동창업자가 없는 창업자.

그게 바로 당신이다. 그리고 지금 도구들이 충분히 좋다.

핵심만 짚자면

바이브 코딩은 실재한다. 하지만 도구 선택보다 프로세스가 더 중요하다. 작동하는 결과와 답답한 결과를 가르는 것은 규율이다. 만들기 전에 정의하고, 레이어로 구축하고, 진행하며 테스트해라.

가장 작은 유용한 버전으로 시작해라. 다른 모든 것은 버전 2다. 첫 번째 버전을 과도하게 범위화하는 것이 바이브 코딩 프로젝트가 막히는 가장 흔한 이유다.

레이어로 구축하고 각각을 테스트해라. 뼈대 → 데이터 → 핵심 액션 → 부가 기능. 무작위가 아닌 정밀하게 디버그해라. 문제를 명확하게 설명하고, 정확한 오류를 보여주고, AI에게 수정 전에 설명을 요청해라.

그리고 기억해라. 바이브 코딩의 진짜 힘은 개발자를 대체하는 것이 아니라, 문제를 깊이 이해하는 사람들이 직접 솔루션을 만들 수 있게 하는 것이다.

> 데모까지 만들었다면? 후속 가이드 바이브 코딩 이후의 실전 가이드가 그 다음을 이어받아요 — 바이브 코딩으로 만든 앱을 "내 노트북에서만 돌아가는" 상태에서 진짜 돈을 버는 서비스로 한 단계씩 키우는 법.

더 깊이 배우고 싶다면

이 글은 바이브 코딩 워크플로우를 다뤘다. 송재희의 *AI Development Guide*는 더 깊이 들어간다 — 바이브 코딩된 앱을 유지하고 확장하는 방법, 데이터베이스와 인증을 연결하는 방법, 프로토타입에서 프로덕션으로 이동하는 방법, 노코드 도구의 한계에 부딪혔을 때 개발자와 협력하는 방법 포함.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

다음 편: "AI 코드 어시스턴트 — 새로운 페어 프로그래밍 파트너" — 프로젝트가 요구할 때 개발자와 비 개발자 모두 바이브 코딩보다 더 깊이 들어가기 위해 AI 코딩 도구를 사용하는 방법.

AI 코드 어시스턴트

12 min read · 구축하기

"AI로 만들기" 시리즈 9편

> **핵심 요약:** AI 코드 어시스턴트 — Cursor, Windsurf, GitHub Copilot, Claude Code — 는 당신의 코드 에디터 안에서 코드 수준으로 협업한다. 설명하고, 리팩터링하고, 디버깅하고, 생성하고, 검토하지만 통제권은 당신에게 있다. 코드를 감추는 바이브 코딩 도구와 달리 어시스턴트는 코드를 그대로 보여 준다 — 그래서 무엇이 만들어지는지 이해하고, 유지보수하고, 신뢰할 수 있다.

8편에서 바이브 코딩을 다뤘다. 원하는 것을 설명하면 AI가 만드는 경험. 많은 사용 사례에서 그걸로 충분하다. 코드를 직접 건드리지 않고 오후 안에 작동하는 앱을 얻는다.

하지만 한계가 있다.

프로젝트가 복잡해지면 — 미묘한 것을 디버깅하거나, 기존 코드베이스를 확장하거나, 성능을 최적화하거나, 공유된 기준을 가진 팀에서 작업할 때 — 순수한 "설명하고 받는" 워크플로우가 한계를 드러내기 시작한다. 더 많은 제어, 실제 일어나고 있는 것에 대한 더 많은 가시성, 코드 자체와의 더 긴밀한 피드백 루프가 필요하다.

여기서 AI 코드 어시스턴트가 등장한다.

바이브 코딩의 대체물이 아니라, 그것의 자연스러운 확장이다. AI를 완전히 포기하지 않고 코드에 더 가까이 가고 싶을 때 코드 어시스턴트가 다린다. 2026년에 이 도구들은 비개발자들도 생산적으로 사용할 만큼 유능해졌다. 코드를 처음부터 작성하기 위해서가 아니라, 만든 것을 이해하고, 확장하고, 유지하기 위해.

코드 어시스턴트의 다른 점

바이브 코딩 도구들(Bolt, Lovable, v0)은 설명에서 전체 애플리케이션을 생성한다. 아키텍처, 파일, 배포를 관리한다. 기본 코드는 거의 보이지 않는다.

코드 어시스턴트는 다르게 작동한다. 개발 환경, 즉 코드 편집기 안에 자리 잡고 코드 레벨에서 협력한다. 코드가 보인다. 직접 편집할 수 있다. AI가 설명하고, 리팩터링하고, 디버그하고, 생성하고, 검토할 수 있다. 하지만 실제로 무엇이 작성되는지는 당신이 제어한다.

구분이 중요한 이유:

- 가시성 — 코드베이스에 무엇이 있는지 이해하면 그것에 대해 추론하고, 문제를 발견하고, 의도적인 변경을 할 수 있다
- 정밀도 — 전체 기능이 아닌 특정 함수, 특정 버그, 특정 리팩터 레벨에서 AI에게 지시할 수 있다
- 유지보수성 — 이해하는 코드가 유지할 수 있는 코드다. 마법처럼 나타난 코드는 신비롭게 망가진다
- 팀 호환성 — 개발자와 함께 작업할 때 새로운 조각을 생성하는 것이 아닌 공유 코드를 이해하고 수정해야 한다

2026년의 주요 플레이어들

Cursor는 2026년 현재 지배적인 코드 어시스턴트다. 완전한 VS Code 포크로, 세계에서 가장 인기 있는 코드 편집기와 똑같이 보이고 작동하며 AI가 경험의 모든 부분에 깊이 내장되어 있다. 할 수 있는 것:

- 코드의 어떤 부분이든 선택해서 AI에게 설명하거나, 리팩터링하거나, 버그를 찾아달라고 요청
- 자연어로 변경을 설명하면 Cursor가 여러 파일에 걸쳐 구현
- "Composer"로 전체 코드베이스에 걸쳐 대규모 조율된 변경
- 모든 파일의 전체 맥락으로 프로젝트에 대해 AI와 대화

Cursor의 특별한 강점은 코드베이스 전반적인 이해다. 편집 중인 파일만 보는 게 아니라 전체 프로젝트를 보고, 조각들이 어떻게 관련되는지 이해하고, 시스템 전반에 걸쳐 일관된 변경을 만들 수 있다.

Windsurf (Codeium 제)는 약간 다른 강점을 가진 강력한 Cursor 대안이다. 특히 다단계 코딩 작업을 자율적으로 계획하고 실행할 수 있는 "Cascade" 에이전틱 모드가 있다. 많은 개발자들이 작업에 따라 Cursor와 Windsurf를 전환한다.

GitHub Copilot (Microsoft/GitHub 제)는 기업 환경에서 가장 널리 배포된 코드 어시스턴트다. "에이전트 모드"가 이제 줄 단위 제안만이 아닌 멀티파일 편집과 전체 작업 완성을 처리한다. 팀이 이미 GitHub를 사용한다면 Copilot이 자연스럽게 통합된다.

Claude Code (Anthropic 제)는 터미널 기반 에이전틱 코딩 도구다. 복잡하고 다단계의 자율 코딩 작업에 가장 유능하다. 편집기 기반 도구와 달리 Claude Code는 명령줄에서 작동하며 전체 작업 스프린트를 계획하고, 실행하고, 결과를 테스트하고, 반복할 수 있다. 초보자에게 덜 친화적이지만 맞는 작업에서 탁월하게 강력하다.

OpenAI Codex는 OpenAI의 클라우드 기반 코딩 에이전트로, 위의 에디터 도구들과 구별된다. IDE 안에 자리 잡는 대신 완전히 클라우드에서 작동한다. 작업을 할당하면 GitHub 저장소가 사전 로드된 안전한 샌드박스 환경을 구동해서 자율적으로 작동하고 (파일 읽기 및 편집, 테스트 실행, 타입 확인), 검토를 위한 완성된 풀 리퀘스트를 반환한다. 작업 완료는 일반적으로 1-30분 걸린다. 소프트웨어 엔지니어링에 특화된 GPT-5 버전인 GPT-5-Codex로 구동된다. 핵심 사용 사례: 집중력을 방해하지 않고 잘 정의된 반복 작업 — 리팩터링, 테스트 작성, 버그 수정, 문서 생성 — 을 맡기는 것. 2026년 3월 기준 주

간 활성 사용자 200만 명을 돌파했고, 점점 더 광범위한 기업 에이전트 플랫폼으로 자리매김하고 있다. ChatGPT Plus, Pro, Business, Enterprise, Edu 구독자가 이용 가능.

Google Antigravity는 Google의 에이전트 우선 IDE로, 2025년 11월 발표됐으며 현재 이 분야에서 가장 많이 회자되는 도구 중 하나다. Cursor나 Copilot이 기존 에디터 프레임워크에 에이전트 기능을 레이어로 추가한 것과 달리, Antigravity는 처음부터 멀티 에이전트 자율 실행을 중심으로 설계됐다. 코드베이스의 다른 부분에서 동시에 작업하는 여러 에이전트를 파견할 수 있는 "Manager View"를 도입했으며, 각각은 감사 가능한 Artifacts — 작업 계획, 구현 계획, 스크린샷, 브라우저 녹화 — 를 생성해 무슨 일이 일어났는지 검토할 수 있다. 두드러지는 기능: 창을 전환하지 않고 웹 UI를 자율적으로 테스트하고 검증할 수 있는 네이티브 브라우저 에이전트. Gemini 3.1 Pro, Claude Opus 4.6, Claude Sonnet 4.6, GPT-OSS-120B를 지원하며, 현재 관대한 모델 쿼터로 공개 프리뷰에서 무료다. 솔직한 평가: 초기 프리뷰 허니문 후 속도 제한이 강화됐고 일부 사용자가 신뢰성 문제를 보고했다. 실험과 1인 개발에는 탁월하지만, 프로덕션 팀 워크플로우에서는 검증이 덜 됐다.

JetBrains AI는 JetBrains 제품군(IntelliJ, PyCharm, WebStorm)에 통합된다. 팀이 기업 Java와 Kotlin 개발에서 여전히 일반적인 JetBrains IDE를 사용한다면 관련이 있다.

비개발자가 코드 어시스턴트를 사용하는 5가지 방법

개발자가 아니어도 코드 어시스턴트에서 가치를 얻을 수 있다. 비개발자와 바이브 코더들이 실제로 사용하는 5가지 방법:

1. 직접 쓰지 않은 코드 이해하기

코드의 어떤 부분이든 선택하고 물어봐라: > "이 코드가 일반 언어로 무엇을 하는지 설명해줘. 입력으로 무엇을 받고 무엇을 반환하는가? X를 바꾸면 어떻게 되는가?"

이것이 가장 과소평가된 사용이다. 바이브 코딩된 앱이 예상치 못한 동작을 만들 때 코드를 읽고 이해할 수 있는 것, 불완전하게도, 이 효과적으로 디버깅하는 것과 맹목적으로 패치하는 것의 차이다.

2. 기존 기능 확장하기

새로 만드는 대신 확장해라. 기존 코드의 관련 부분을 선택하고 프롬프트해라: > "사용자가 이 테이블을 CSV 파일로 내보낼 수 있는 기능을 추가하고 싶어. 테이블이 현재 어떻게 구축됐는지 보면, 이것을 추가하는 최선의 방법은 무엇인가? 코드를 작성해줘."

AI가 이미 존재하는 것을 보고 그것과 작동하는 코드를 생성한다. 기존 구조와 충돌할 수 있는 것을 생성하는 대신.

3. 맥락으로 디버깅하기

무언가 망가지면 망가진 코드와 오류 메시지를 붙여넣고 물어봐라: > "이 함수가 이 오류를 발생시키고 있어: [오류 붙여넣기]. 여기 함수가 있어: [코드 붙여넣기]. 이것이 무엇을 일으키는지 설명하고 수정을

보여줘."

한 단계 제거된 노코드 도구를 통한 디버깅과 달리 문제의 정확한 설명을 얻는다. 패치만이 아니라 무엇이 잘못됐는지에 대한 이해.

4. 코드 리뷰와 품질 개선

코드의 일부를 선택하고 물어봐라: > "이 코드를 검토해줘. 잠재적인 문제는 무엇인가? 이것을 작성하는 더 깔끔하거나 효율적인 방법이 있는가? 놓칠 수 있는 엣지 케이스는 무엇인가?"

개발자에게 코드를 보여주기 직전에 특히 가치 있다. 실제 검토 전에 명백한 문제를 정리할 수 있다.

5. 형식 간 변환

코드 어시스턴트는 변환에 탁월하다: > "이 JavaScript 함수를 Python으로 변환해줘." > "이 SQL 쿼리를 ORM의 쿼리 빌더 문법을 사용하도록 변환해줘." > "이 함수는 배열을 입력으로 받는데 — 같은 데이터를 JSON 객체로 받도록 재작성해줘."

이런 변환은 사람에게 지루하고 AI에게 사소하다.

실제로 작동하는 워크플로우

코드 어시스턴트에서 좋은 결과를 얻기 위한 가장 중요한 단 하나의 습관: **항상 맥락을 보여줘라, 독립적으로 설명하지 마라.**

나쁜 프롬프트: > "이메일을 보내는 함수를 작성해줘."

좋은 프롬프트: > "여기 내 기존 알림 서비스가 있어: [코드 붙여넣기]. 사용자의 구독이 만료될 때 이메일을 보내는 함수를 추가해야 해. 이 파일 상단에서 이미 임포트하고 있는 같은 이메일 클라이언트를 사용하고, 여기 다른 함수들과 같은 오류 처리 패턴을 따라야 해."

차이는 맥락이다. 기존 코드를 보는 AI가 시스템에 맞는 것을 만든다. 기존 코드를 보지 않는 AI가 통합하는 데 한 시간 걸릴 일반적인 것을 만든다.

3단계 디버그 루프

무언가 망가지면:

- 식별 — 문제를 신뢰할 수 있게 재현해라. 어떤 입력이 어떤 잘못된 출력을 만드는지 정확히 알아라.
- 보여줘 — AI에게 정확한 오류, 정확한 코드, 정확한 예상 동작을 줘라:

> "이 함수가 [Y]를 전달하면 [X]를 반환해. [Z]를 기대했어. 여기 코드가 있어. 왜 X를 반환하고 무엇을 바꿔야 하는가?"

- 검증 — 그냥 수정을 받아들이지 마라. 이해해라. 적용하기 전에 "왜 이 수정이 작동하는가?"를 물어봐라. 이해하지 못한 수정은 찾을 수 없을 미래의 버그다.

프롬프트 대신 주석을 쓸 때

활용이 부족한 기법 하나: 원하는 것을 코드 주석으로 써라, 그 다음 AI에게 구현하도록 요청해라.

```
// TODO: 데이터베이스에 저장하기 전에 이메일 필드가 유효한 // 이메일 형식인지 검증하라. 유효하지 않으면
```

> "이 함수의 TODO 주석을 구현해줘."

이 접근법은 프롬프트하기 전에 원하는 것에 대해 정밀하게 생각하도록 강요하고, 코드에 각 부분이 무엇을 해야 하는지에 대한 흔적을 남긴다.

더 좋은 결과를 위한 파워 유저 팁

코드 어시스턴트에서 일관되게 좋은 결과를 얻는 사람과 계속 벽에 부딪히는 사람을 가르는 습관들이다.

먼저 플랜 모드를 사용하라, 코드를 작성하기 전에. 대부분의 코드 어시스턴트(Cursor의 Composer, Antigravity의 Manager View, Claude Code)는 코드를 건드리기 *전에* 단계별 구현 계획을 생성하는 계획 또는 "생각" 모드가 있다. 사소하지 않은 모든 것에 항상 이것을 사용하라. 도구에게 요청해라: "이것을 어떻게 구현할지 계획해줘. 아직 코드는 작성하지 마, 접근 방식, 어떤 파일이 변경될지, 어떤 결정이 필요한지만 설명해줘." 계획을 검토해라. 오해를 수정해라. 그 다음 실행해라. 이 단 하나의 습관이 대부분의 작업 중 탈선을 없앤다.

CLAUDE.md 또는 AGENTS.md 파일을 사용하라. Cursor, Claude Code, OpenAI Codex는 모두 프로젝트 루트의 특별한 파일(`.cursorrules` , `CLAUDE.md` , `AGENTS.md`)을 지원한다. 이 파일에 프로젝트가 어떻게 작동하는지, 규칙, 사용할 라이브러리, 따를 패턴, 피할 것들을 문서화한다. AI는 매 세션 시작 시 이것을 읽는다. 좋은 프로젝트 파일은 이렇게 말할 수 있다: "이것은 스타일링에 Tailwind를 사용하는 Next.js 앱이다. 인증에는 항상 기존 `useAuth` 혹은 사용하라 — 절대 직접 만들지 마라. TypeScript strict 모드를 사용하라. 컴포넌트는 150줄 이내로 유지해라." 이 맥락은 시간이 지나면서 복리로 쌓인다. 매 세션이 더 스마트하게 시작된다.

작업을 작고 원자적으로 유지해라. "전체 사용자 설정 페이지 만들기" 같은 작업은 너무 크다. 분해해라: "표시 이름과 이메일 필드가 있는 폼 추가" → 테스트 → "폼에 유효성 검사 추가" → 테스트 → "폼을 저장 엔드포인트에 연결" → 테스트. 작은 작업이 더 신뢰할 수 있는 결과를 만들고, 검토하기 쉽고, 더 복구 가능한 방식으로 실패한다. AI에게 큰 작업을 주고 싶은 유혹이 강하다, 도구가 할 수 있기 때문에. 하지만 작은 작업이 더 나은 코드베이스로 복리 누적된다.

다른 작업에 다른 모델을 사용하라. 단일 모델이 모든 것에서 최고는 아니다. 일반적인 파워 워크플로우: 빠른 편집, 설명, 작은 생성 작업에 빠른 모델(Claude Sonnet 4.6, GPT-5.2)을 사용하라. 아키텍처 결정, 까다로운 버그, 또는 신중한 사고가 중요한 것에 강력한 추론 모델(Claude Opus 4.6)을 사용하라. Antigravity에서는 말 그대로 다른 병렬 에이전트에 다른 모델을 할당할 수 있다. 모든 것에 같은 모델을 기본값으로 사용하는 대신 작업에 모델을 맞춰라.

긴 세션 후 요약으로 새로 시작해라. 코드 어시스턴트 컨텍스트 윈도우는 긴 세션 동안 채워진다. 세션이 길어지고 AI가 이전에 하지 않던 실수를 하기 시작하면 그게 신호다. 계속 밀어붙이지 마라. 새 세션을 시작해라. 새 세션을 요약으로 열어라: "사용자 인증 시스템을 구축하고 있었어. 지금까지 완료한 것: [목록]. 다음 단계는 [작업]. 핵심 파일들이야: [관련 코드 붙여넣기]." 좋은 요약을 가진 새 모델이 꼭 찬 컨텍스트 윈도우를 가진 지친 모델을 능가한다.

AI가 같은 자리를 맴돌 때 리셋해라. 모든 코드 어시스턴트에는 어떤 형태의 컨텍스트 리셋이 있다. AI가 혼란스러워 보일 때, 같은 실수를 반복하거나, 이전 결정과 모순되거나, 이미 고친 것을 망가뜨릴 변경을 제안하거나 할 때는 혼란에서 벗어나도록 설득하려 하지 말고 컨텍스트를 리셋해라. 관련 코드만 붙여넣고 문제를 깔끔하게 다시 진술해라. 대화 줄이고, 정밀도 높이기.

검토하지 않은 변경을 수락하지 마라. 코드 어시스턴트의 가장 큰 위험은 맹목적 수락이다. 이해하지 않고 변경을 적용하는 것. 모든 diff를 적용하기 전에 읽는 습관을 들여라. 모든 줄을 반드시 이해할 필요는 없지만, 무엇이 변경됐는지와 왜는 알아야 한다. 수락하기 전에 어시스턴트에게 각 변경을 일반 언어로 요약해달라고 요청해라. 이것이 당신을 운전석에 유지시키고 작은 오류가 큰 오류로 복리 증가하는 것을 방지한다.

만드는 코드 이해하기

모든 바이브 코더가 결국 맞닥뜨리는 솔직한 질문이 있다: AI가 생성하는 코드를 이해해야 하는가?

답은 미묘하다.

모든 줄을 이해할 필요는 없다. 라우팅 라이브러리가 내부적으로 어떻게 작동하는지, 데이터베이스 ORM이 SQL을 어떻게 생성하는지, 인증 라이브러리가 토큰 검증을 어떻게 처리하는지 알 필요가 없다. 이것들은 해결된 문제다.

시스템의 형태를 이해해야 한다. 데이터는 어디서 오는가? 애플리케이션을 통해 어떻게 흐르는가? 사용자가 양식을 제출하면 무슨 일이 일어나는가? 무엇이 무엇을 호출하는가? 이 개념적 이해, 구현이 아닌 아키텍처가 확장하고, 디버그하고, 코드에 대해 좋은 결정을 내릴 수 있게 해주는 것이다.

변경하는 코드를 특히 이해해야 한다. 이해하지 못하는 코드를 변경하는 것이 바이브 코딩 프로젝트가 부채를 쌓고 유지 불가능해지는 방법이다. 변경을 할 때마다, 들어가기 전에 그것이 무엇을 하는지 이해해라. 필요하다면 코드 어시스턴트에게 설명해달라고 해라.

목표는 코드 작성의 유창함이 아니다. 읽고 추론하는 능력이다. 그건 배울 수 있다. 그리고 코드 어시스턴트는 생성하는 대신 설명을 요청할 때 놀랍도록 좋은 선생님이다.

언제 개발자에게 넘겨야 하는가

코드 어시스턴트는 비개발자가 얼마나 멀리 갈 수 있는지를 확장한다. 개발자의 필요를 완전히 없애지는 않는다. 언제 개발자를 데려올지:

인증, 권한 부여, 데이터 암호화, 또는 결제 처리와 관련된 것은 공격 표면을 이해하는 누군가에게 검토받아야 한다.

10,000명의 동시 사용자를 위한 최적화는 진짜 전문성이 필요한 데이터베이스 인덱싱, 캐싱 전략, 인프라 결정을 포함한다.

기업 API, 레거시 시스템, 복잡한 데이터 파이프라인 연결은 종종 문서화되지 않은 엣지 케이스와 어떤 AI도 가지지 않은 조직 지식을 포함한다.

코드베이스에 대해 더 이상 추론할 수 없을 때, 즉 시스템이 무엇을 하는지, 조각들이 어떻게 관련되는지, 무언가가 왜 실패하고 있는지 설명할 수 없다면 그것이 신호다. 개발자를 데려와라. 인수인계하기 위해서가 아니라 그 이해를 회복하는 데 도움을 받기 위해서.

인수인계는 협력적이지, 포기가 아니다. 바이브 코딩된 기반을 검토하고 확장하는 개발자는 처음부터 구축하는 것보다 훨씬 생산적이다. 특히 만든 것과 왜 그렇게 했는지 설명할 수 있을 때.

핵심만 짚자면

코드 어시스턴트는 바이브 코딩과 전문 개발 사이의 다리다. 둘 중 어느 것의 대체물도 아니다. 바이브 코딩이 제공하는 것보다 더 많은 가시성과 제어가 필요할 때의 자연스러운 다음 단계다.

맥락이 전부다. 새 코드를 요청하기 전에 항상 기존 코드를 보여줘라. 시스템을 보는 AI가 그것에 맞는 코드를 생성한다. 변경하는 것을 이해해라. 이해하지 못하는 코드를 확장하는 것이 프로젝트를 유지 불가능하게 만드는 방법이다. 적용하기 전에 AI에게 설명을 요청해라.

코드 유창함보다 코드 리터러시가 중요하다. 코드를 작성할 필요가 없다. 읽고, 추론하고, 시스템의 형태를 이해해야 한다. 코드 어시스턴트는 설명을 요청할 때 탁월한 선생님이다.

그리고 언제 개발자를 데려올지 알아라. 보안, 규모, 복잡한 통합, 더 이상 추론할 수 없는 코드베이스. 이것들이 신호다. 인수인계는 협력적이지, 실패가 아니다.

더 깊이 배우고 싶다면

이 글은 코드 어시스턴트 작업을 다뤘다. 송재희의 *AI Development Guide*는 더 깊이 들어간다 — 효과적인 AI 보조 개발 워크플로우 설정 방법, 특정 도메인(데이터 파이프라인, API, 프론트엔드, 모바일)에 코드 어시스턴트를 사용하는 방법, AI가 대부분을 생성할 때 시간이 지나면서 코드 품질을 유지하는 방법 포함.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

다음 편: "데모에서 프로덕션으로" — 만든 것을 신뢰할 수 있고, 비용 효율적이고, 실제 규모에서 실제 사용자를 위해 준비된 것으로 만드는 방법.

데모에서 프로덕션으로

9 min read · 배포하기

"AI로 만들기" 시리즈 10편

> **핵심 요약:** "데모에서 작동했다"는 "프로덕션에서 작동한다"와 다르다. 데모는 최선의 경우에 맞춰져 있지만, 프로덕션은 예측 불가능한 입력, 실제 사용자의 데이터, 비용 한계, 신뢰를 무너뜨리는 실패까지 견뎌야 한다. 프로덕션 준비는 통제된 프로토타입에서 현실이 제멋대로 변해도 버티는 시스템까지, 다섯 단계를 거친다.

아무도 잘 이야기하지 않는 묘지가 있다.

데모에서 완벽하게 작동했던 AI 프로젝트들로 가득 차 있다. 창업자가 투자자에게 보여줬다. 팀이 박수를 쳤다. 준비된 데이터셋과 세 명의 테스트 사용자로 프로토타입이 완벽하게 돌아갔다. 모두가 흥분했다.

그리고 라이브로 갔다. 실제 사용자. 실제 데이터. 실제 규모. 실제 엣지 케이스. 그리고 하나씩 균열이 나타났다.

AI가 결제 고객에게 자신 있게 틀린 답을 줬다. 하루 1만 건의 API 호출이 얼마나 드는지 아무도 계산하지 않아서 비용이 폭등했다. 실제 부하에서 시스템이 느릿느릿 기어갔다. 사용자가 전체를 망가뜨리는 입력을 찾아냈다. 테스트에서 좋게 들렸던 응답이 프로덕션에서는 어색하게 들렸다. 실제 사용자들이 팀이 예상한 것과 다른 질문을 했기 때문에.

데모는 작동했다. 제품은 그렇지 않았다.

이것이 대부분의 AI 빌더들이 빠지는 간극이다. 잘 못 만들어서가 아니다. 데모 사고방식과 프로덕션 사고방식이 근본적으로 다른 분야이기 때문이다. 이 글은 프로덕션 사고방식에 관한 것이다.

데모-프로덕션 간극

데모는 최선의 경우를 보여주도록 최적화된다. 프로덕션 시스템은 예상하지 못한 것들을 포함한 모든 경우를 처리해야 한다.

데모에서 프로덕션으로 갈 때 변하는 것:

| 데모 | 프로덕션 | |-----|-----| | 엄선된 입력 | 예측 불가능한 입력 | | 나의 데이터 | 사용자의 데이터 | | 테스트 사용자 3명 | 수천 명의 사용자 | | 예상하는 것을 안다 | 사용자가 끊임없이 놀라게 한다 | | 비용은 중요하지 않다 | 비용이 실행 가능성을 결정한다 | | 실패는 보이지 않는다 | 실패가 신뢰를 손상시킨다 | | 맥락을 제어한다 | 맥락이 크게 다르다 | | 속도는 수용 가능하다 | 속도는 기능이다 |

이 전환들 중 불가능한 것은 없다. 하지만 각각은 의도적인 설계가 필요하다. 데모가 작동했다고 자동으로 일어나는 것은 없다.

1단계: 프로토타입에서 파일럿으로

첫 번째 전환은 데모에서 전체 프로덕션으로가 아니다. 데모에서 실제 사용자와의 통제된 파일럿으로다.

파일럿은 모든 것을 지켜볼 수 있을 만큼 작고, 망가졌을 때 수동으로 고칠 수 있고, 규모에 헌신하기 전에 프로덕션이 실제로 어떤 모습인지 배울 수 있다.

좋은 파일럿의 모습:

- 테스트 계정이 아닌 실제 사용자 10-50명
- 엄선된 예시가 아닌 실제 데이터
- 스크립트된 시나리오가 아닌 실제 작업
- 무언가 잘못됐을 때 사용자가 알려줄 수 있는 피드백 메커니즘
- 당신이나 팀 누군가가 정기적으로 결과물을 지켜보는 것

파일럿에서 배우는 것:

- 실제 사용자가 실제로 무엇을 묻거나 입력하는가? (거의 항상 예상과 다름)
- AI가 실패하거나, 환각을 일으키거나, 잘못 느껴지는 결과물을 만드는 곳은?
- 얼마나 빨라야 하는가? (사용자는 개발자보다 훨씬 인내심이 없다)
- 실제 사용량에서 실제로 얼마가 드는가?
- 예상하지 못한 어떤 엇지 케이스가 나타나는가?

전체 출시로 바로 가는 것으로 파일럿을 건너뛰지 마라. 파일럿은 프로덕션이 실제로 무엇을 요구하는지 배우는 곳이다. 아직 흡수할 수 있는 비용으로.

2단계: 신뢰성 엔지니어링

프로덕션 시스템은 실패한다. 문제는 언제 실패하느냐가 아니라 언제, 어떻게, 그리고 실패했을 때 무슨 일이 일어나는가다.

AI 실패를 우아하게 처리하기

AI 결과물은 확률적이다. 때로는 틀리거나, 불완전하거나, 부적절할 것이다. 시스템이 망가지거나 당황스럽게 만들지 않고 이것을 처리해야 한다.

중요한 것에 대해 원시 AI 결과물을 사용자에게 직접 전달하지 마라. 결과물이 전달되기 전에 의미가 있는지 확인하는 검증을 구축하라. 예상한 형식인가? 존재하는 것들을 참조하는가? 길이가 합리적인가?

AI가 실패하거나, 타임아웃되거나, 유효하지 않은 것을 반환하면 무슨 일이 일어나는가? 계획을 가져라. 우아한 폴백 — "응답을 생성할 수 없었습니다, 직접 연락하는 방법이 있습니다" — 이 망가진 경험보다 무한히 낫다.

틀렸을 때 실제 결과가 있는 것, 의료 질문, 법적 세부사항, 재무 권고에는 인간 검토 단계를 구축하라. AI가 초안을 작성한다. 인간이 승인한다.

레이턴시: 속도는 기능이다

사용자는 참을성이 없다. 응답하는 데 3-4초 이상 걸리면 평균적인 사람은 프로세스를 포기한다.

전략들:

- 스트리밍 응답 — 응답이 생성되는 대로, 글자 하나씩 보여줘라. 사용자는 총 생성 시간이 같아도 스트리밍 응답을 더 빠르게 인식한다.
- 캐싱 — 같거나 비슷한 질문이 반복적으로 질문되면 응답을 캐시해라. 이미 생성한 것을 다시 생성하지 마라.
- 모델 티어링 — 단순한 작업에는 빠르고 저렴한 모델을 써라. 가장 유능한(그리고 가장 느린) 모델은 진정으로 필요한 작업을 위해 남겨둬라.
- 비동기 처리 — 즉각적인 응답이 필요하지 않은 작업(보고서 생성, 배치 분석)은 백그라운드에서 처리하고 완료되면 알림을 보내라.

모니터링: 볼 수 없는 것을 고칠 수 없다

프로덕션에서는 항상 무슨 일이 일어나는지 가시성이 필요하다.

모니터링할 것:

- 응답 레이턴시 (요청이 얼마나 오래 걸리는가?)
- 오류율 (얼마나 자주 실패하는가?)
- AI 결과물 품질 (응답이 실제로 좋은가? 자동화된 지표만이 아닌 샘플링과 인간 검토가 필요하다)
- 요청당 비용 (예상한 만큼 지출하고 있는가?)
- 사용 패턴 (누가, 무엇을, 언제, 어떻게 사용하는가?)

Langfuse, Helicone, Langsmith 같은 도구들이 AI 애플리케이션 모니터링을 위해 특별히 만들어졌다. 모든 프롬프트, 모든 응답, 모든 레이턴시 측정을 기록하고 사용자가 알리기 전에 문제를 발견할 수

있는 대시보드를 제공한다.

규칙: 출시 전에 모든 것을 계측하라. 라이브 시스템에 모니터링을 나중에 추가하는 것은 고통스럽고 가장 가시성이 필요한 기간에 눈을 감고 운영했다는 의미다.

3단계: 비용 아키텍처

이것이 가장 많은 AI 프로젝트를 죽이는 단계다. 제품이 작동하지 않아서가 아니라 경제성이 안 맞아서.

AI API 호출은 돈이 든다. 데모 규모에서 비용은 사소하다. 월 몇 달러. 수천 명의 사용자가 각각 수십 건의 요청을 하는 프로덕션 규모에서 숫자는 매우 불편하게 빠를 수 있다.

아무도 미리 하지 않는 비용 계산

출시 전에 사용자 1인당 월 비용을 계산하라.

요청당 비용 = (입력 토큰 × 입력 가격) + (출력 토큰 × 출력 가격) 사용자당 일일 요청 수 × 30 = 사용자

그리고 물어보라: 계획된 가격으로 실행 가능한 마진이 남는가?

실제 숫자 (근사치, 2026년 중반):

- Claude Sonnet 4.6: 입력 토큰 백만당 약 \$3, 출력 토큰 백만당 약 \$15
- GPT-5.2: 입력 토큰 백만당 약 \$10, 출력 토큰 백만당 약 \$40
- Claude Haiku 4.5: 입력 토큰 백만당 약 \$0.25, 출력 토큰 백만당 약 \$1.25

일반적인 채팅 상호작용은 입력 토큰 1,000개와 출력 토큰 500개를 사용할 수 있다. Sonnet 4.6 가격으로 상호작용당 약 \$0.003, 100번 상호작용에 \$0.30. 관리 가능하다.

하지만 시스템에 큰 컨텍스트(긴 문서, 긴 대화 이력), 복잡한 추론 작업, 또는 많은 순차적 에이전트 단계가 포함되면 비용이 빠르게 배가된다. 항상 헌신하기 전에 숫자를 실행해보라.

비용 제어 전략

프롬프트 압축: 짧은 프롬프트는 비용이 덜 든다. 불필요한 맥락을 제거해라. 긴 이력을 전부 전달하는 대신 요약해라. 절약되는 모든 토큰이 절약되는 돈이다.

모델 티어링: 단순한 작업은 저렴한 모델로, 복잡한 작업은 유능한 모델로 라우팅해라. Haiku에서 \$0.001이 드는 분류 작업이 Opus에서 실행될 필요가 없다.

캐싱: 일반적인 응답을 캐시해라. 비싼 계산을 캐시해라. 반복적으로 요청되는 모든 것을 캐시해라.

배치 처리: 많은 API가 비실시간 요청에 50% 할인된 배치 가격을 제공한다. 사용 사례가 즉각적인 응답이 필요하지 않다면 모든 것을 배치 처리해라.

사용 한도: 사용자당 한도를 설정해라. 인색해서가 아니라, 단일 폭주 사용 패턴이 알아채기 전에 예상치 못한 비용을 만들 수 있기 때문이다.

4단계: 확장 아키텍처

100명의 사용자에게 작동하는 것이 10,000명에게는 종종 망가진다. AI가 달라서가 아니라, 주변 인프라가 부하를 위해 설계되지 않았기 때문이다.

상태 비저장 vs 상태 저장 설계

AI 상호작용은 가장 단순하게 상태 비저장으로 설계된다. 각 요청이 독립적이고, 필요한 모든 맥락을 포함하고, 이전 요청의 서버 측 상태에 의존하지 않는다. 상태 비저장 시스템은 수평으로 확장된다. 서버를 더 추가하면 더 많은 요청을 처리한다.

상태 저장 시스템, 즉 서버가 대화 이력, 사용자 맥락, 또는 진행 중인 에이전트 상태를 유지하는 시스템은 확장하기 어렵다. 상태 저장 AI 기능을 구축한다면 서버 메모리가 아닌 적절한 데이터베이스를 상태로 사용해라.

속도 제한은 친구다

모든 AI 제공자는 속도 제한을 시행한다. 분당 최대 요청 수. 낮은 사용량에서는 절대 도달하지 않는다. 프로덕션 규모에서는 도달할 수 있다.

속도 제한 오류를 우아하게 처리하도록 시스템을 설계해라:

- 한도 근처일 때 요청을 큐에 넣어라
- 지수 백오프를 구현해라 (1초 후 재시도, 그 다음 2초, 4초, 8초...)

프롬프트는 인프라다

프로덕션에서 프롬프트는 단순한 지시가 아니다. 코드다. 애플리케이션 코드와 마찬가지로 버전 제어, 테스트, 배포 프로세스가 필요하다.

프롬프트를 변경할 때:

- 실제 입력의 대표 샘플에 대해 변경을 테스트해라
- 새 결과물을 이전 것과 비교해라
- 점진적으로 배포해라. 완전히 출시하기 전에 트래픽의 일부에서 A/B 테스트
- 무언가 잘못되면 즉시 롤백할 수 있어야 한다

프롬프트를 일회성 텍스트로 취급하고 프로덕션에서 무심코 변경하는 팀은 디버그하기 어려운 방식으로 것들을 망가뜨린다.

프로덕션 체크리스트

데모에서 프로덕션으로 이동하기 전에 이것을 검토하라:

신뢰성

- ☐ AI 결과물에 검증 레이어 있음
- ☐ AI 실패 시 우아한 폴백 있음
- ☐ 고위험 결과물에 인간 검토 있음
- ☐ 의미 있는 사용자 메시지로 오류 처리

성능

- ☐ 해당하는 곳에 스트리밍 응답 적용
- ☐ 응답 캐싱 전략 있음
- ☐ 다른 작업 복잡성에 모델 티어링 적용
- ☐ 레이턴시 목표 정의 및 테스트됨

비용

- ☐ 사용자당 월 비용 계산됨
- ☐ 목표 규모에서 마진 검증됨
- ☐ 비용/품질에 맞게 모델 선택 최적화됨
- ☐ 사용 한도 및 알림 설정됨

모니터링

- ☐ 레이턴시 모니터링 활성화
- ☐ 오류율 모니터링 활성화
- ☐ 알림과 함께 비용 모니터링 활성화
- ☐ 결과물 품질 샘플링 프로세스 정의됨

확장

- ☐ 상태 비저장 설계 검증됨
- ☐ 속도 제한 처리 구현됨
- ☐ 프롬프트 버전 제어 중

- [] 배포 및 롤백 프로세스 테스트됨

핵심만 짚자면

데모 사고방식과 프로덕션 사고방식은 근본적으로 다른 분야다. 데모는 최선의 경우를 보여준다. 프로덕션은 모든 경우를 처리한다. 처음부터 모든 경우를 위해 설계해라.

출시 전에 항상 파일럿을 해라. 실제 데이터로 실제 사용자 10-50명이 엄선된 입력으로 1,000시간의 테스트보다 프로덕션이 요구하는 것에 대해 더 많은 것을 가르쳐준다.

모델에 헌신하기 전에 비용을 계산하라. 사용자당 월 비용, 계획된 가격으로, 현실적인 사용량으로. 놀라운 청구서를 받은 후가 아닌 출시 전에 이 숫자를 실행해라.

출시 전에 모든 것을 계측해라. 레이턴시, 오류, 비용, 결과물 품질. 볼 수 없는 것을 고칠 수 없다. 모니터링을 나중에 추가하는 것은 고통스럽다. 미리 해라.

그리고 프롬프트를 지시가 아닌 인프라로 취급해라. 버전 제어, 테스트, 점진적 배포, 롤백. 프로덕션에서 무심코 변경되는 프롬프트는 기다리고 있는 프로덕션 인시던트다.

> **데모를 프로덕션으로 올리는 중이라면?** 후속 가이드가 바로 거기서 시작해요. [바이브 코딩 이후의 실전 가이드](#)는 호스팅, 시크릿 관리, 법적 기본, 분석, 그리고 진짜 서비스를 굴리는 운영 체계 — 나아가 수요 검증과 첫 결제까지 다룹니다.

더 깊이 배우고 싶다면

이 글은 프로덕션 라이프사이클을 다뤘다. 송재희의 *AI Development Guide*는 더 깊이 들어간다 — 다양한 유형의 AI 애플리케이션을 위한 특정 아키텍처 패턴, AI 결과물 품질을 위한 평가 프레임워크 구축 방법, 첫 번째 100명 사용자에서 첫 번째 10만 명으로 확장하는 방법 포함.

 [Apple Books](#)  [Google Play Books](#)  전 플랫폼 구매 (Books2Read)

다음 편: "리스크, 환각, 책임 있는 AI" — 모든 빌더가 AI의 실패 모드, 법적 노출, 그리고 설계에 의해 신뢰할 수 있는 시스템을 구축하는 방법에 대해 알아야 하는 것.

위험, 할루시네이션과 책임 있는 AI

10 min read · 배포하기

"AI로 만들기" 시리즈 11편

> **핵심 요약:** 할루시네이션은 AI가 거짓되거나 지어내거나 근거 없는 내용을 정확한 정보와 똑같이 자신 있는 어조로 말하는 것이다. AI에는 불확실성을 표시하는 내부 신호가 없고, 그저 패턴을 완성할 뿐이다. 이 글은 AI의 실패 양상을 네 가지 할루시네이션 유형으로 나누고, 배포 전에 이를 잡아내는 실용적인 안전장치 툴킷을 제공한다.

모든 AI 빌더가 결국 맞닥뜨리는 순간이 있다.

사용자가 찾아와 말한다: "당신 AI가 [틀린 것]을 말했어요. 그걸 믿고 행동했는데, 손해를 봤습니다."

자신감 있게 말했지만 거짓으로 드러난 사실이었을 수 있다. 그 사람의 상황에 맞지 않는 조언이었을 수 있다. 테스트에서는 괜찮아 보였지만 이 특정 사람, 이 특정 순간에는 깊이 부적절한 결과물이었을 수 있다.

당신이 무언가를 만들었다. 그것이 해를 끼쳤다. AI가 말했지, 당신이 말한 게 아니더라도 — 당신이 책임이다.

이 글은 출시 후가 아닌 출시 전에 그 책임을 진지하게 받아들이는 것에 관한 것이다. AI로 구축하는 것이 위험하기 때문이 아니다. 대부분의 경우 그렇지 않다. 하지만 리스크에 대해 신중하게 생각하는 빌더가 더 나은 제품을 만들기 때문이다. 사용자가 필요할 일이 없기 때문에 절대 알아채지 못하는 안전장치를 설계한다. AI가 시스템에 어디에 속하는지, 어디에 인간이 루프에 있어야 하는지에 대해 미리 판단한다.

이것이 이 글이 다루는 것이다.

환각 이해하기: 핵심 리스크

2편에서 환각에 대해 잠깐 다뤘다. 여기서 더 깊이 들어간다. 실제 무언가를 구축하고 있다면, 정확히 무엇을 다루고 있는지 이해해야 하기 때문이다.

환각은 AI 모델이 사실적으로 부정확하거나, 조작되거나, 지지되지 않는 내용을 생성하면서 정확한 정보에 사용하는 것과 같은 자신감 있는 말투로 제시할 때다. 모델은 자신이 틀렸다는 것을 모른다. "여기서 불확실합니다"라는 내부 플래그가 없다. 결과물은 그냥 흘러나온다.

왜 일어나는가

언어 모델은 지금까지 온 모든 것을 고려해 통계적으로 가장 가능성 있는 다음 토큰을 예측한다. 훈련 데이터에 패턴의 충분한 예시가 있으면 모델이 그 패턴을 자신 있게 완성한다. 없을 때, 즉 모호하거나, 최근 이거나, 훈련 범위 밖의 것에 대해 질문받으면 문법적으로나 맥락적으로 맞는 것으로 패턴을 완성하는데, 이것이 현실과 거의 관계가 없을 수 있다.

거짓말하는 게 아니다. 인간이 진실 대 거짓을 이해하는 방식으로 진실이나 거짓에 대한 개념이 없다. 패턴을 완성하고 있는 것이다.

네 가지 환각 유형

사실적 환각은 가장 흔하다. 낱자, 통계, 인용, 이름, 사건들을 자신감 있게 틀리게 말한다. 확인하면 잡아내기 가장 쉽다.

추론 환각은 잡아내기 어렵다. 논리적으로 결함이 있는 추론 사슬을 만들어내 잘못된 결론에 도달하지만, 건전한 논리처럼 제시한다.

출처 환각은 존재하지 않는 인용, 논문, 기사, 인용문을 발명한다. 리서치, 법적, 의학적 맥락에서 특히 위험하다.

맥락적 환각은 독립적으로는 기술적으로 정확하지만 특정 사용자, 특정 상황, 특정 관할권이나 도메인에 대해 잘못된 내용을 생성한다.

환각이 위험한 때

환각이 모든 애플리케이션에서 똑같이 위험한 건 아니다.

마케팅 카피를 생성하는 AI가 가끔 약간 어긋난 문구를 만든다면? 잡아내기 쉽다. 낮은 이해관계.

의료 정보를 제공하고 자신 있게 틀린 용량을 말하는 AI? 잠재적으로 생명을 위협한다.

법적 지침을 제공하고 존재하지 않는 판례를 인용하는 AI? 그것에 의존하는 사람에게 직업적으로 파멸적이다.

재무 조언을 제공하고 조작된 수익률을 말하는 AI? 재정적으로 손상을 준다.

환각의 리스크는 두 가지 요소에 직접 비례한다: 결과물이 얼마나 중요한가, 그리고 사용자가 독립적으로 검증할 가능성이 얼마나 되는가. 높은 이해관계에 낮은 검증일 때, 즉 의료, 법률, 재무, 안전 핵심 영역에서 가장 강력한 안전장치가 필요하다. 낮은 이해관계에 쉬운 검증일 때, 즉 마케팅 카피, 브레인스토밍, 초안 작성 수준이라면 리스크는 관리 가능하다.

리스크 스펙트럼

모든 AI 애플리케이션이 같은 리스크를 가지는 건 아니다. 당신의 것이 어디에 위치하는지 이해하라.

낮은 리스크

- 창의적 콘텐츠 생성 (마케팅, 글쓰기, 아이디어 발굴)
- 사용자가 제공한 콘텐츠 요약
- 결과물을 검토할 전문가 사용자가 있는 내부 도구
- 결과물이 출발점인 브레인스토밍과 아이디어 발굴
- 엔터테인먼트와 낮은 이해관계 추천

필요한 안전장치: 기본 품질 점검, 사용자의 편집 능력, AI 생성임을 투명하게 표시.

중간 리스크

- 고객 대면 정보 시스템
- 리서치 지원 도구
- 자동화된 커뮤니케이션 (실제 사람에게 보내는 이메일, 메시지)
- 실제 비즈니스 프로세스의 분류와 라우팅
- 직접 결정을 내리지 않고 결정에 영향을 미치는 도구

필요한 안전장치: 검증 레이어, 엡지 케이스에 인간 검토, 명확한 면책 조항, 감사 로깅, 사용자 피드백 메커니즘.

높은 리스크

- 의료, 법률, 또는 재무 지침
- 안전 핵심 정보 (비상 절차, 위험 경고)
- 중요한 결과를 가진 자동화된 결정 (채용, 대출, 의료 분류)
- 취약한 인구가 사용하는 도구 (정신 건강, 위기 지원)
- 인간 검토 없이 직접 고객에게 전달되는 콘텐츠

필요한 안전장치: 모든 중요한 결과물에 인간 포함, 명시적 면책 조항, 전문가 감독 요구사항, 포괄적 로깅, 결과물 품질 정기 감사, 명확한 에스컬레이션 경로.

안전장치 구축: 실용적 툴킷

1. 그라운드닝

그라운드닝은 AI 결과물을 검증된 권위 있는 출처에 연결하는 것을 의미한다. 모델이 혼자 훈련 데이터에서 생성하도록 놔두는 대신.

RAG(검색 증강 생성)이 가장 일반적인 구현이다. 모델에게 사실을 상기하도록 요청하는 대신 신뢰할 수 있는 출처에서 먼저 관련 문서를 검색한 다음 그 문서에만 기반해 답하도록 요청한다.

프롬프트 패턴: > "제공된 문서의 정보만을 사용해 다음 질문에 답하세요. 답이 문서에 없다면 '이 질문에 답할 충분한 정보가 없습니다'라고 말하세요. 일반 지식을 사용하지 마세요. > > 문서: [검색된 콘텐츠]
> 질문: [사용자 질문]"

이것이 사실적 환각을 극적으로 줄인다. 모델이 출처 자료에서 찾을 수 없는 것을 조작할 수 없다. 완벽하지는 않지만 개방형 생성보다 훨씬 신뢰할 수 있다.

2. 신뢰도 신호

불확실성을 솔직하게 전달하는 시스템을 구축해라. 사용자가 언제 검증에 더 주의해야 하는지 알 수 있도록.

모델이 불확실할 때 그것을 표현하도록 프롬프트하라: > "답변의 어떤 부분에 대해 불확실하다면 명시적으로 말하세요. 불확실한 것을 자신 있게 말하는 대신 '확실하지 않지만...' 또는 '이것은 확인이 필요하지만...' 같은 문구를 사용하세요."

불확실한 영역에서 AI가 어떻게 행동하는지도 명확하게 정의하라: > "질문이 의료, 법률 또는 재무 조언과 관련이 있는지 결정하세요. 있다면 적절한 전문가에게 안내하는 것으로만 응답하고 질문에 직접 답하려 하지 마세요."

3. 범위 제한

AI가 할 것과 하지 않을 것에 대한 명확한 경계를 정의하라. 그리고 일관되게 시행해라.

인스코프/아웃오브스코프 프롬프팅: > "당신은 [회사]의 고객 지원 어시스턴트입니다. 도움을 드릴 수 있는 것: 계정 질문, 제품 기능, 청구, 플랫폼 기술 지원. 이 주제 밖의 것 — 일반 조언, 다른 회사 제품, 의료나 법률 질문 포함 — 에 대해 정중하게 안내하고 [회사] 관련 질문만 도움을 드릴 수 있다고 설명하세요."

4. 결과물 검증

중요한 것에 대해 원시 AI 결과물을 사용자에게 직접 전달하지 마라. 검증 단계를 구축해라.

형식 검증: 결과물이 예상한 구조를 가졌는가? AI가 항상 특정 필드를 가진 JSON 객체를 반환해야 한다면 사용하기 전에 확인해라.

내용 검증: 결과물에 예상한 요소가 있는가? 금지된 내용을 피하는가? 이차 확인을 실행해라: > "다음 응답을 검토하세요. (1) 주제 범위 내에 있는지, (2) 용량, 법적 결과, 또는 재무 수익에 대한 특정 주장을 피하는지, (3) 불확실한 정보에 대한 적절한 주의 사항을 포함하는지 확인하세요. 이 중 하나라도 실패하면 FAIL을 반환하고 이유를 설명하세요."

5. 감사 로깅

항상 모든 것을 로그해라. 모든 프로덕션 AI 시스템에 대해:

- 모든 입력 (사용자가 보낸 것)
- 모든 결과물 (AI가 반환한 것)
- 타임스탬프와 사용자 식별자
- 사용된 모델과 프롬프트 버전
- 검증 레이어가 제기한 모든 플래그

이건 좋은 관행만이 아니다. 규제된 산업에서는 법적으로 요구될 수 있다. 그리고 실용적으로, 문제 발생 후 조사하고, 체계적 실패 패턴을 식별하고, 무언가 잘못됐을 때 적절한 주의를 입증하는 유일한 방법이다.

법적, 윤리적 환경

AI 주변의 규제 환경은 빠르게 진화하고 있다. 2026년 빌더가 알아야 할 것:

책임

대부분의 관할권에서 AI 책임에 대한 법적 프레임워크는 여전히 발전 중이다. 하지만 실용적 현실은 이미 명확하다. AI 제품이 해를 끼치면 당신이 책임이다. 해가 기술적으로 AI의 결과물에 의해 야기됐더라도.

법원과 규제기관들은 AI 결과물을 전문적 조언이나 제품 권고와 점점 더 동등하게 취급하고 있다.

실용적 함의: AI 결과물에 자신의 전문적 조언에 적용할 것과 같은 수준의 주의를 적용하라. 자격 없이 직접 말하지 않을 것이라면 AI도 말해서는 안 된다.

투명성

사용자는 AI와 상호작용하고 있다는 것을 알 권리가 있다. 이것은 여러 관할권에서 점점 더 법으로 성문화되고 있다.

실용적 요구사항:

- AI 생성 콘텐츠를 AI 생성으로 명확하게 표시하라
- 사용자가 인간이라고 생각하도록 속이는 AI 페르소나를 설계하지 마라
- AI가 사용자에게 영향을 미치는 결정에 사용될 때 공개하라

데이터 프라이버시

사용자가 AI 시스템과 상호작용할 때 종종 개인 정보를 공유한다. 때로는 민감한 개인 정보를. 이것은 의무를 만든다:

- 어떤 데이터가 얼마나 오래 보존되는가?
- 사용자 데이터가 모델 훈련이나 파인튜닝에 사용되는가?
- GDPR, 한국 개인정보보호법, 또는 기타 적용 가능한 규정을 준수하는가?
- 사용자가 데이터 삭제를 요청할 권리가 있는가?

사용하는 모든 AI API 제공자의 서비스 약관을 읽어라. 일부 제공자는 기본적으로 모델 훈련에 데이터를 사용하도록 허용한다. 허용되지 않는다면 옵트아웃해야 한다.

편향과 공정성

역사적 데이터로 훈련된 AI 모델은 역사적 편향을 재현하고 증폭할 수 있다. 채용 도구, 대출 승인, 콘텐츠 조정 같은 중요한 애플리케이션에서 이것은 윤리적 문제와 법적 리스크 모두를 만든다.

실용적 단계:

- 배포 전에 다양한 사용자 그룹에 걸쳐 시스템을 테스트해라
- 인구 통계 그룹에 걸쳐 체계적인 차이에 대한 결과물을 모니터링해라
- 사용자가 문제 있는 결과물을 플래그할 수 있는 피드백 메커니즘을 구축해라
- 오류 패턴을 이해하지 않고 고위험 의사결정에 AI를 배포하지 마라

책임 있는 빌더의 마인드셋

기술적, 법적 요구사항 너머에 모든 AI 빌더가 깊이 생각해봐야 할 더 깊은 질문이 있다:

내가 만들고 있는 것에서 무엇이 잘못될 수 있는가, 그리고 그것에 편안한가?

"혹시라도 해를 끼칠 방법이 있는가"가 아니다. 거의 모든 것에 있다. 하지만: 현실적인 실패 모드에 대해 신중하게 생각했는가? 그것들을 위해 설계했는가? 내가 만드는 이점이 내가 만드는 리스크보다 크다고 확신하는가?

이것은 의사가 약을 처방하기 전에, 변호사가 조언을 주기 전에, 엔지니어가 설계를 승인하기 전에 하는 것과 같은 질문이다. 전문적 책임.

AI 분야는 이 책임 문화가 아직 형성되고 있을 만큼 젊다. 잘 형성하는 빌더들, 실패 모드를 진지하게 받아들이고, 강력한 안전장치를 설계하고, 한계에 대해 솔직하게 소통하는 이들이 지속되고 사용자가 실제로 신뢰할 수 있는 것을 만들 것이다.

신중하게 생각하지 않고 빠르게 출시하는 사람들은 규제를 정당화하는 경고의 이야기가 될 것이다.

첫 번째 그룹처럼 구축하라.

레드 플래그 체크리스트

출시 전에 스스로에게 물어보라:

결과물에 대해:

- [] 시스템을 망가뜨리거나 오도하도록 설계된 입력으로 테스트했는가?
- [] 시스템이 불확실할 때 무슨 말을 하는지 아는가?
- [] 엡지 케이스 사용자(비원어민, 접근성 필요 사용자, 위기 상황 사용자)로 테스트했는가?
- [] 결과물이 사용자에게 전달되기 전에 검증되는가?
- [] 고위험 결과물에 인간 검토 단계가 있는가?

신뢰와 투명성에 대해:

- [] 사용자가 AI와 상호작용하고 있다는 것을 아는가?
- [] 한계와 면책 조항이 명확하게 전달되는가?
- [] 무언가 잘못됐을 때 명확한 피드백 경로가 있는가?
- [] 중요한 AI 결정이 사용자에게 설명 가능한가?

데이터에 대해:

- [] 어떤 데이터가 얼마나 오래 보존되는지 아는가?
- [] 사용하는 모든 API 제공자의 데이터 정책을 읽었는가?
- [] 사용자 데이터가 적용 가능한 규정을 준수하여 처리되는가?
- [] 사용자가 자신의 데이터에 대한 적절한 통제권을 가지는가?

실패에 대해:

- [] 모든 것이 로그되는가?
- [] 무언가 잘못됐을 때 알려줄 모니터링이 있는가?
- [] 문제를 조사하고 해결하는 프로세스가 있는가?
- [] 필요시 시스템을 빠르게 롤백하거나 비활성화할 수 있는가?

핵심만 짚자면

환각은 고쳐야 할 버그가 아닌 구조적인 것이다. 모델은 가장 가능성 있는 다음 토큰을 생성한다. 진실을 검증하지 않는다. 결과물이 때로는 틀릴 것이라고 가정하고 그에 맞게 시스템을 설계하라.

리스크는 결과와 검증 가능성에 따라 확대된다. 의료, 법률, 재무가 가장 높은 리스크다. 창의적 작업, 브레인스토밍, 내부 도구는 가장 낮은 리스크다. 당신의 것이 어디에 위치하는지 알고 이해관계에 비례하는

안전장치를 설계하라.

그라운드, 범위 제한, 검증을 조합하라. RAG로 사실적 그라운딩을. 명확한 인스코프/아웃오브스코프 정의. 사용자 전달 전 결과물 검증. 이 세 가지가 함께 대부분의 일반적인 실패 모드를 제거한다.

항상 모든 것을 로그해라. 기록하지 않은 것을 조사할 수 없다. 로깅은 책임, 품질 개선, 법적 방어 가능성의 기반이다.

그리고 책임 있는 빌더가 물어야 할 질문은 이것이다: 무엇이 잘못될 수 있는지 신중하게 생각했는가? 리스크가 있는지가 아니라, 현실적인 실패 모드를 위해 설계했는가? 이점이 리스크를 정당화하는가? 내가 출시하는 것에 편안한가?

더 깊이 배우고 싶다면

이 글은 리스크와 책임 레이어를 다뤘다. 송재희의 *AI Development Guide*는 더 깊이 들어간다 — 다양한 애플리케이션 유형을 위한 특정 안전장치 아키텍처, AI 품질을 위한 평가 프레임워크 구축 방법, 권한에 걸쳐 진화하는 규제 환경 탐색 방법 포함.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

시리즈 마지막 편: "다음 물결 — AI 에이전트, 피지컬 AI, 그리고 그 이후" — 기술이 향하는 곳과 빌더가 지금 무엇을 준비해야 하는지.

다음 물결: AI 에이전트, 물리 AI, 그리고 그 이후

9 min read · 미래

"AI로 만들기" 시리즈 12편 — 마지막

> **핵심 요약:** AI의 다음 물결은 이미 형성되고 있다. 여러 전문 에이전트를 조율하는 자율 에이전트 네트워크, 현실 세계에서 행동하는 물리 AI와 휴머노이드 로봇, 그리고 초기 단계의 자기 개선 시스템이다. 오늘의 기준선 — 운영 중인 에이전트, 좁은 영역의 AI "직원", 거의 닫힌 소프트웨어 개발 루프 — 은 종착점이 아니라 출발점이다.

이 시리즈는 답답함으로 시작했다. AI가 할 수 있다는 것과 대부분의 사람들이 실제로 얻고 있는 것 사이의 간극. 11편에 걸쳐 그 간극을 좁혀왔다: AI가 어떻게 생각하는지 이해하고, 데이터와 프롬프트로 구축하고, 에이전트로 자동화하고, 프로덕션으로 출시하고, 책임감 있게 구축하는 것.

이제 앞을 바라본다.

간극은 가만히 서 있지 않기 때문이다. 도구들은 역사상 거의 모든 기술보다 빠르게 진화하고 있다. 그리고 다음 물결은 우리가 논의해온 것의 개선이 아니다. 완전히 다른 범주의 역량이다.

이 글은 무엇이 오고 있는지에 관한 것이다. 거짓 정밀함으로 미래를 예측하기 위함이 아니라, 당신이 지능적으로 자리를 잡을 수 있도록 영역의 지도를 제공하기 위함이다. 지금 일어나고 있는 것을 이해하는 빌더들이 다음 물결이 도착했을 때 진짜 우위를 가지게 될 것이다.

지금 실제로 어디에 있는가

앞을 보기 전에 오늘 실제로 사실인 것에 닳을 내리는 것이 중요하다. 과대포장도, 추측도 아닌.

에이전틱 AI는 운영 중이다. 7편에서 논의한 다단계, 도구 사용, 의사결정 에이전트들은 실험적이지 않다. 기업 워크플로우, 고객 서비스, 소프트웨어 개발, 영업, 리서치에서 대규모로 배포되고 있다.

AI 직원은 좁은 도메인에서 실재한다. Artisan, 11x, Relevance AI 같은 회사들이 AI SDR을 배포했다. 잠재 고객을 발굴하고, 리서치하고, 아웃리치를 작성하고, 팔로업하고, 미팅을 예약하는 영업 개발 담

당자다. 완벽하지 않고 엇지 케이스에서 망가진다. 하지만 실제 회사들이 실제 수익을 내며 프로덕션에서 운영 중이다.

소프트웨어 개발 루프가 닫히고 있다. 2026년 초 현재 Claude Code는 전체 코드베이스, 멀티파일 편집, 1년 전에는 시니어 엔지니어가 필요했던 에이전틱 코딩 작업을 처리하는 최고의 AI 코딩 어시스턴트로 널리 인식된다.

이것이 기준선이다. 다음은 무슨 일이 일어나고 있는지도.

이미 밀려오는 물결 1: 자율 에이전트 네트워크

이미 진행 중인 첫 번째 전환은 단일 에이전트에서 에이전트 네트워크로의 이동이다. 여러 AI 에이전트가 함께 작동하고, 각각이 전문화되어 공유 목표를 향해 조율하는 시스템.

패턴: 오케스트레이터 에이전트가 목표를 받고 하위 작업으로 나누어 각각을 전문가 에이전트에게 할당하고, 결과를 조율하고, 결과물을 전달한다. 전문가 에이전트들은 리서치, 작성, 사실 확인, 포매팅, 품질 검토를 처리할 수 있다. 각각 작업에 최적화되어 병렬로 실행된다.

실용적으로 이것이 여는 것:

수십 개의 소스를 동시에 검색하고, 각각에서 구조화된 데이터를 추출하고, 일관성을 위해 교차 참조하고, 보고서로 합성하는 에이전트 스웸은 인간 연구자가 며칠 걸릴 것을 분 단위로 완성한다.

요구사항 분석, 아키텍처 계획, 구현, 테스트, 디버깅, 문서화, 배포의 전체 소프트웨어 개발 루프를 처리하는 에이전트 네트워크는 인간이 핵심 체크포인트에서 검토하고 승인하는 방식으로 작동한다.

단순히 이메일을 라우팅하거나 양식을 채우는 것이 아니라, 비즈니스 프로세스의 전체 라이프사이클을 처리하는 것도 가능해진다. 접수, 분석, 결정, 행동, 팔로업, 예외 처리를 인간은 진정으로 새로운 상황에만 개입해서 담당한다.

빌더를 위한 실용적 함의: 오늘 에이전트 시스템을 설계한다면 조합 가능하게 설계해라. 다른 에이전트를 호출할 수 있는 에이전트를 구축해라. 그들을 조율할 수 있는 오케스트레이션 레이어를 구축해라. 확장될 시스템은 모놀리스가 아닌 네트워크로 구축된 것이다.

구축 중인 물결 2: 피지컬 AI

2026년의 가장 극적인 발전이고 가장 긴 도달 범위를 가진 것은 AI 지능과 물리적 하드웨어의 수렴이다. 피지컬 AI.

CES에서 NVIDIA CEO 젠슨 황은 "피지컬 AI의 ChatGPT 순간이 왔다"고 말했다.

이것이 실제로 무엇을 의미하는가?

수십 년 동안 로봇은 스크립트된 기계였다. 사전 프로그래밍된 루틴을 따른다. 정의된 매개변수 내에서 정밀하고 신뢰할 수 있으며, 그 밖에서는 쓸모없다. 하루에 같은 용접을 1만 번 하는 공장 로봇은 그 정확한 작업에 탁월하고 다른 모든 것에는 무능하다.

전환: AI 기반 로봇은 이제 환경을 인식하고, 자연어 지시를 이해하고, 다음에 무엇을 할지 추론하고, 전에 한 번도 만난 적 없는 상황에 적응할 수 있다. 스크립트를 따르는 게 아니다. 생각하고 있다.

핵심 기술은 VLA 모델이다. 시각적 인식(환경 보기), 언어 이해(음성 명령 처리), 행동 실행(물리 세계에서 이동)을 통합하는 비전-언어-행동 모델.

지금 피지컬 AI를 구축하는 주체들

NVIDIA의 Isaac GR00T N1.6은 휴머노이드 로봇을 위해 특별히 만들어진 오픈 추론 비전 언어 행동 모델로, 전신 제어를 가능하게 한다. ABB Robotics, AGIBOT, Agility, Figure, KUKA, Universal Robots, YASKAWA 등이 대규모로 피지컬 AI를 배포하기 위해 NVIDIA 기술 위에 구축하고 있다.

Tesla의 Optimus Gen 2는 반복적인 작업을 처리하고, 제조를 보조하고, 실제 데이터에서 학습하면서 홈 자동화 기능을 수행하도록 설계됐다. Boston Dynamics는 CES 2026에서 자재 취급부터 주문 이행까지 광범위한 산업 작업을 위해 설계된 고성능 엔터프라이즈급 휴머노이드인 Electric Atlas를 공개했다. 1X는 NEO 홈 휴머노이드의 사전 주문을 열었고, 2026년 첫 번째 고객 배달이 계획됐다.

AGIBOT은 10,000번째 휴머노이드 로봇의 출시를 발표했으며, 이 규모로 이 이정표에 도달한 최초의 회사 중 하나가 됐다. 2026년 3월 서울 COEX에서 열린 스마트 팩토리 & 자동화 세계 전시회에는 2,300개 부스가 채워졌으며, 중국 휴머노이드 로봇 제조업체들이 전시했다.

피지컬 AI가 이미 작동하는 곳

Amazon의 창고 로봇 군단은 2025년 100만 대를 돌파했으며, 새로운 DeepFleet AI 모델이 네트워크 전반의 이동 효율을 10% 향상시킬 것으로 기대된다. 수술 로봇은 대형 병원에서 60% 도입에 달했으며, 로봇 보조 수술이 선진국 복잡 수술의 55%를 차지한다.

3,200명 이상의 글로벌 비즈니스 리더를 대상으로 한 딜로이트(Deloitte) 조사에 따르면, 약 58%가 현재 어느 정도 피지컬 AI를 사용하고 있으며, 향후 2년 내 그 비율이 80%로 늘어날 전망이다.

피지컬 AI가 여전히 어려워하는 곳

여기서 솔직함이 중요하다. 대부분의 휴머노이드 플랫폼은 여전히 3-4시간의 배터리 수명이라는 "운영 상한"과 씨름하며, 바늘 실 꿰기나 부서지기 쉬운 물건 취급 같은 섬세한 작업의 정교함은 여전히 인간 능력에 미치지 못한다.

피지컬 AI는 실재하고 가속화되고 있다. 하지만 실험실 성능과 현장 성능 사이의 간극은 여전히 크다. 솔직한 그림: 피지컬 AI는 지금 창고, 공장, 수술실을 변환하고 있다. 향후 5년 내에 서비스 산업과 가정으로 온다. 하지만 인간이 할 수 있는 모든 것을 하는 완전히 유능한 범용 로봇의 비전은 10년 이상 남아 있다.

피지컬 AI가 한국에 의미하는 것

한국 기술 생태계에서 구축하는 독자들에게 특히 관련이 있다.

한국은 세계 로봇 제조, 반도체 기술, 스마트 팩토리 배포에서 선두 국가 중 하나다. 한국 스타트업 RLWORLD는 자동화된 학습과 인간 전문성 모방을 통해 전통적인 수작업 집약적 프로세스가 로봇에 의해 자율적으로 수행될 수 있도록 하는 기반 AI 모델을 개발하고 있다.

한국 정부의 스마트시티와 스마트 팩토리 이니셔티브, 이 시리즈가 함께 작성된 바로 그 생태계의 많은 부분이 피지컬 AI 물결로의 직접적인 진입점이다. K-스타트업 생태계는 특별한 기회를 가지고 있다. 깊은 하드웨어 제조 전문성, 스마트 인프라에 대한 정부 지원, 피지컬 AI 배포가 가장 빠르게 확장될 아시아 시장과의 근접성.

지평선의 물결 3: 스스로 개선하는 AI

세 번째 물결은 가장 추측적이지만, 일부가 믿는 대로 도착한다면 가장 결과적이기도 하다.

현재 AI 시스템은 훈련되고, 배포되고, 그 다음 정적이다. Claude Opus 4.6은 대화에서 더 똑똑해지지 않는다. GPT-5.2는 배포에서의 상호작용에서 아무것도 배우지 않는다. 오늘 사용하는 모델은 6개월 후 사용할 것과 같다. Anthropic이나 OpenAI가 업데이트를 출시하지 않는 한.

다음 프론티어: 상호작용을 통해 스스로 개선되는 AI 시스템. 각 배포, 각 실패, 각 새로운 정보 조각에서 배우고, 처음부터 재훈련 없이 시간이 지나면서 측정 가능하게 더 나아진다.

이것은 아직 대규모로 일어나지 않고 있다. 하지만 연구 방향은 명확하다:

지속적 학습은 이미 알고 있는 것의 파국적 망각 없이 새로운 데이터에서 가중치를 업데이트하는 시스템을 목표로 한다.

자체 플레이와 자기 개선은 이전 버전의 자신과 경쟁함으로써 자체 훈련 신호를 생성하는 시스템이다. AlphaGo를 체스에서 초인적으로 만든 접근법이 추론에도 적용된다.

재귀적 개선은 더 나은 데이터, 더 나은 평가 기준, 더 나은 파인튜닝 접근법을 생성하며 자체 훈련 프로세스를 개선하는 데 도움이 되는 시스템이다.

이것이 오늘의 빌더에게 의미하는 것

위의 모든 것에서 세 가지 구체적인 함의:

1. 도메인 지식이 해자다

도구들이 일반적인 부분, 즉 코드 작성, 콘텐츠 생성, 문제 추론에서 더 나아질수록 가치는 오직 당신만이 아는 것으로 이동한다. 업계 전문성. 고객 관계. 특정 도메인의 뉘앙스에 대한 이해. 독점 데이터.

일반 AI 시스템은 누구도 마케팅 이메일을 작성하는 데 도움을 줄 수 있다. 20년의 엔터프라이즈 데이터 엔지니어링 경험과 500명 이상의 바이브 코딩 교육 과정으로 훈련된 시스템, 그건 일반적이지 않다. 이기는 빌더는 강력한 일반 AI와 깊은 도메인 특수성을 결합하는 사람들이다.

2. 지금만을 위해서가 아닌 적응성을 위해 구축하라

도구들은 변할 것이다. 오늘 Claude Opus 4.6으로 구축하는 워크플로우가 2년 후 10배 더 유능한 모델에서 실행 가능하다. 시스템을 "모델 교체"가 쉽도록 설계해라. 모델 특정 특성으로 프롬프트를 하드코딩하지 마라. 한 제공자의 API로만 작동하는 워크플로우를 구축하지 마라.

추상화 레이어, 즉 로직, 데이터, 워크플로우 설계는 당신 것이다. 아래의 모델은 시간이 지나면서 더 나아지는 상품이다. 그렇게 다뤄라.

3. AI 능력을 쌓을 최고의 시간은 지금이다

다음 물결에 대한 솔직한 진실: 오늘 가진 것보다 훨씬 더 강력할 것이다. 그리고 이미 기본을 이해하는 사람들, 즉 문제 정의, 데이터 품질, 프롬프팅, 에이전트, 프로덕션 배포, 책임 있는 구축을 아는 이들을 제로에서 시작하는 사람들보다 훨씬 더 효과적으로 만들 것이다.

"AI 기본을 이해한다"와 "이해하지 못한다" 사이의 간극은 도구가 더 강력해질수록 좁아지는 게 아니라 넓어진다. 어떻게 지시하는지 이해하지 못하는 사람의 손에 있는 더 강력한 도구는 그냥 더 빠른 평범함을 만든다. 이해하는 사람의 손에 있는 더 강력한 도구는 우위를 복리로 쌓는다.

이 시리즈를 읽었다. 더 이상 제로에서 시작하지 않는다.

이것이 의미하지 않는 것

다음 물결은 흥분과 함께 많은 두려움을 만들어낼 것이다. 대체되는 일자리. 집중되는 권력. 인간의 감독 범위를 벗어나는 시스템.

그 우려 중 일부는 정당하고 진지한 주목이 필요하다. 여기서 무시할 장소가 아니다. 11편의 책임 있는 AI 글이 이 환경에서 빌더의 의무를 다룬다.

하지만 이것을 읽는 빌더에게: 기술 자체가 물결이 좋은지 나쁜지를 결정하지 않는다. 어떻게 배포되는지가 결정한다. 누구를 섬기느냐가 결정한다. 어떤 안전장치가 내장되는지가 결정한다. 이것들은 빌더가 내리는 결정이다, 당신이.

이 질문들에 대해 신중하게 생각하는 빌더들, 이 시리즈의 나머지에서 나온 기술적 역량과 함께 11편의 책임 프레임워크로 구축하는 빌더들이 만들 가치 있는 것들을 만드는 사람들이다.

그렇게 구축하라.

전체 그림

시작한 곳과 도달한 곳을 명명하며 마무리하자.

답답함으로 시작했다. AI가 약속을 이행하지 않고, 데모 간극이 있고, 비교 나선이 있고, 뒤처지는 느낌.

다뤄온 것들:

- 사고 모델 전환 (1편)
- AI가 실제로 할 수 있는 것과 못 하는 것 (2편)
- 스택이 어떻게 맞물리는지 (3편)
- 데이터가 진짜 병목인 이유 (4편)
- 문제 정의가 진짜 기술인 이유 (5편)
- 정밀하게 프롬프팅하는 방법 (6편)
- 에이전트를 구축하고 사용하는 방법 (7편)
- 실제 제품을 바이브 코딩하는 방법 (8편)
- 데모에서 프로덕션으로 가는 방법 (10편)
- 책임감 있게 구축하는 방법 (11편)
- 그리고 이제: 다음 물결이 우리를 어디로 데려가는지 (12편)

이 모든 것을 관통하는 실: 이것은 배울 수 있다. 도구는 계속 개선된다. 기본은 같다. AI에서 일관되게 좋은 결과를 얻는 사람들, 지금과 미래에, 은 문제에 대해 명확하게 생각하고, 사용하는 도구를 이해하고, 주의를 기울여 구축하는 사람들이다.

그게 당신이다. 가서 구축하라.

> **다음 단계:** 배운 것을 진짜 제품으로 만들 준비가 됐다면, 후속 트랙 바이브 코딩 이후의 실전 가이드가 실천적인 다음 걸음이에요 — 띄우기부터 첫 매출, 그리고 이 프로젝트를 무엇으로 만들지 결정하기까지.

완전한 시리즈

이 시리즈가 가치 있었다면 송재희의 *AI Development Guide*가 그것이 그리는 책이다. 여기서 다룬 모든 주제를 더 깊이 다루며, YouTube 튜토리얼과 블로그 글에 흩어져 있지 않은 실용적 예시, 아키텍처, 프레임워크를 담고 있다.

이것은 전체 아크를 다루는 유일한 책이다. AI 기본에서 프로덕션 배포까지, 1인 빌더에서 엔터프라이즈 워크플로우까지, 오늘의 도구에서 분야가 향하는 곳까지.

 Apple Books  Google Play Books  전 플랫폼 구매 (Books2Read)

"AI로 만들기" 시리즈를 읽어주셔서 감사합니다. 이 글들이 도움이 됐다면 아직 왜 답답한지 모르는 채 AI에 좌절하고 있는 누군가에게 공유해주세요.